

파이토치
1.0 대응!

이미지
처리

자연어
처리

앱
만들기

추천
시스템

딥러닝 모델 생성에서 애플리케이션 개발까지

파이토치 첫걸음

주피터
노트북 형식

GPU
구글
클라우드
대응

두세교 지음 | 김완섭 옮김

Jpub
제이펍

두세교 杜世橋

도로공업대학에서 계산 기기를 활용한 분자생물학을 연구했으며, 졸업 후에는 IT 기업에서 소프트웨어 개발 및 데이터 분석을 담당하고 있다. 대학원 시절에 아직 유명해지기 전이었던 파이썬과 NumPy를 접했고, 스터디 모임이나 집필 등을 통해 파이썬을 전파했다. 요근래 몇 년 동안은 스타트업 중심으로 데이터 분석이나 머신러닝 개발 지원 등을 해왔으며, 2018년 4월부터 물류 IT 관련 스타트업에서 근무하고 있다. 머신러닝, 빅데이터 분석, 서버 개발 등에 관심이 많으나, 지금은 자녀 교육을 위해 육아 휴직 중인 아빠 엔지니어이다.

옮긴이

김완섭 jinsilto@gmail.com

네덜란드 ITCO에서 Geoinformation for Disaster Risk Management 석사 학위를 취득했다. 약 9년간 일본과 한국의 기업에서 IT 및 GIS/LBS 분야 업무를 담당했으며, 일본에서는 세콤(SECOM) 계열사인 파스코(PASCO)에서 일본 외무부, 국토지리정보원 같은 정부기관을 대상으로 한 시스템 통합(SI) 업무를 담당했다. 이후 야후 재팬으로 직장을 옮겨 야후맵 개발 담당 시니어 엔지니어로 근무했으며, 한국으로 돌아와 SK에서 내비게이션 지도 데이터 담당 매니저로 근무했다. 현재는 싱가포르에 있는 일본계 회사에서 은행 관련 IT 프로젝트를 담당하고 있다. 저서로는 《나는 도쿄 롯폰기 서 은행 관련 IT 프로젝트를 담당하고 있다》, 《나는 도쿄 롯폰기 서 출근한다》가 있으며, 역서로는 《알고리즘 도감》, 《처음 만나는 HTML5 & CSS3》, 《인공지능 70》, 《처음 만나는 자바스크립트》, 《다양한 언어로 배우는 정규표현식》, 《그림으로 공부하는 IT 인프라 구조》, 《그림으로 공부하는 시스템 성능 구조》 등 20여 종이 있다. 블로그를 통해 IT 번역 관련 이야기와 싱가포르 직장 생활을 소개하고 있다.

독자 A/S 안내

이 책의 예제 코드는 다음 사이트에서 다운로드할 수 있으며, 책 내용 중 궁금한 부분은 독자 Q&A로 문의 바랍니다.



예제 코드

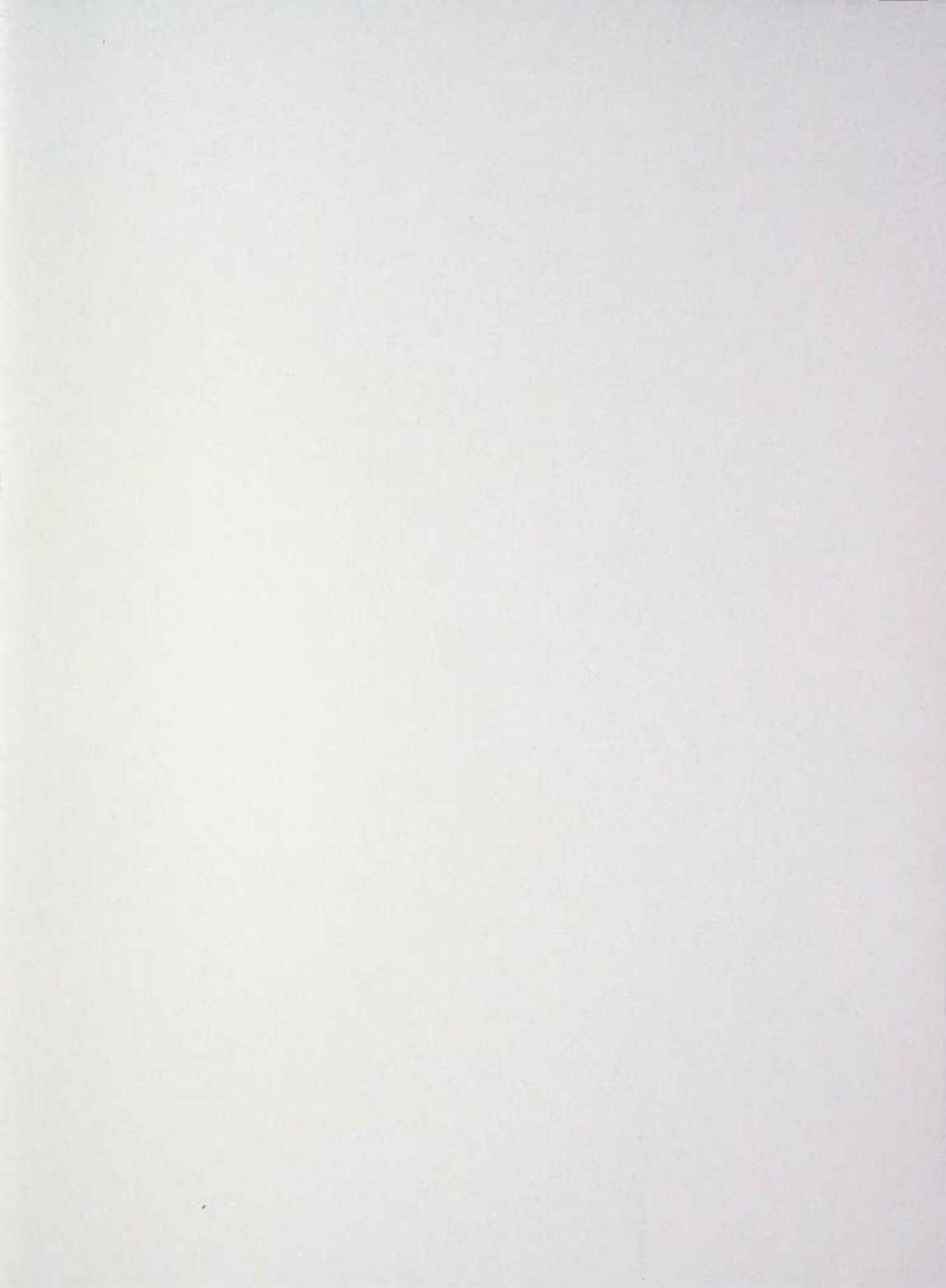
https://github.com/Jpub/PyTorch_firststep



독자 Q&A

jinsilto@gmail.com 또는 readers.jpub@gmail.com





7
E
C
S
I
I
I
I

1

파이트치 첫걸음

現場で使える! PyTorch開発入門

(Genba de Tukaeru! PyTorch Kaihatu Nyumon : 5718-4)

Copyright © 2018 by Du Shiqiao.

Original Japanese edition published by SHOEISHA Co., Ltd.

Korean translation rights arranged with SHOEISHA Co., Ltd. in care of The English Agency (Japan) Ltd.
through Danny Hong Agency.

Korean translation copyright © 2019 J-Pub Co., Ltd.

이 책의 한국어판 저작권은 대니홍 에이전시를 통한 저작권사와의 독점 계약으로 제이펍에 있습니다.
저작권법에 의해 한국 내에서 보호를 받는 저작물이므로 무단전제와 복제를 금합니다.

과학도들 상상에서 에볼루션이신 개발까지

파이토치 첫걸음

1쇄 발행 2019년 5월 9일

자문이 두세교
옮긴이 김완섭
펴낸이 장성두
펴낸곳 주식회사 제이펍

출판신고 2009년 11월 10일 제406-2009-000087호

주소 경기도 파주시 회동길 159 3층 3-B호

전화 070-8201-9010 / 팩스 02-6280-0405

홈페이지 www.jpup.kr / 원고투고 jeipub@gmail.com

독자문의 readers.jpup@gmail.com / 교재문의 jeipubmarketer@gmail.com

편집부 이종무, 황혜나, 최병찬, 이슬, 이주원 / 소통·기획팀 민지환, 송찬수 / 회계팀 김유미
교정·교열 배규호 / 본문디자인 북아이 / 표지디자인 미디어웍스
용지 에스에이치페이퍼 / 인쇄 한승인쇄 / 제본 광우제책사

ISBN 979-11-88621-59-0

값 24,000원

※ 이 책은 저작권법에 따라 보호를 받는 저작물이므로 무단전제와 무단복제를 금지하며,

이 책 내용의 전부 또는 일부를 이용하려면 반드시 저작권자와 제이펍의 서면 동의를 받아야 합니다.

※ 잘못된 책은 구입하신 서점에서 바꾸어 드립니다.

제이펍은 독자 여러분의 책에 관한 아이디어와 원고 투고를 기다리고 있습니다. 책으로 펴내고자 하는 아이디어나 원고가 있는 분께서는 책의 간단한 개요와 차례, 구성과 저역자 연락처 등을 메일로 보내 주세요.

jeipub@gmail.com

딥러닝 모델 생성에서 애플리케이션 개발까지

파이토치 첫걸음



두세교 지음 | 김완섭 옮김

pub
제이펍

※ 드리는 말씀

- 이 책에 기재된 내용을 기반으로 한 운용 결과에 대해 역자, 소프트웨어 개발자 및 제공자, 제이펍 출판사는 일체의 책임을 지지 않으므로 양해 바랍니다.
- 이 책에 등장하는 각 회사명, 제품명은 일반적으로 각 회사의 등록 상표 또는 상표입니다. 본문 중에는 ™, ©, ® 마크 등이 표시되어 있지 않습니다.
- 이 책에서 사용하고 있는 제품 버전은 파이토치 1.0 버전을 기준으로 작성되었으며, 독자의 학습 시점이나 환경에 따라 책의 내용과 다를 수 있습니다.
- 책 내용과 관련된 문의사항은 옮긴이나 출판사로 연락 주시기 바랍니다.

- 옮긴이: jinsilto@gmail.com

- 출판사: readers.jpub@gmail.com

웁긴이 머리말	x
시작하며	xii
이 책의 대상 독자와 필요한 사전 지식	xiv
이 책의 구성	xvi
About the SAMPLE: 이 책의 개발 환경과 예제 프로그램	xviii
베타리더 후기	xx

PROLOGUE 개발 환경 준비

1

0.1 이 책의 검증 환경	2
0.1.1 OS 환경: 우분투 16.04	2
0.1.2 엔비디아의 GPU	2
0.1.3 클라우드에서 GPU를 탑재한 인스턴스 실행하기	3
0.2 개발 환경 구축	5
0.2.1 미니콘다 설치	5
0.2.2 가상 환경 구축	7

CHAPTER 1 파이토치의 기본

11

1.1 파이토치의 구성	12
1.1.1 파이토치의 전반적인 구성	12

1.2 텐서	13
1.2.1 텐서 생성과 변환	13
1.2.2 텐서의 인덱스 조작	15
1.2.3 텐서 연산 16	
1.3 텐서와 자동 미분	20
1.4 정리	22

CHAPTER 2

최대 우도 추정과 선형 모델

23

2.1 확률 모델과 최대 우도 추정	24
2.2 확률적 경사 하강법	26
2.3 선형 회귀 모델	28
2.3.1 선형 회귀 모델의 최대 우도 추정	28
2.3.2 파이토치로 선형 회귀 모델 만들기(직접 만들기)	30
2.3.3 파이토치로 선형 회귀 모델 만들기(nn, optim 모듈 사용)	32
2.4 로지스틱 회귀	35
2.4.1 로지스틱 회귀의 최대 우도 추정	35
2.4.2 파이토치를 사용한 로지스틱 회귀 분석	36
2.4.3 다중 분류를 위한 로지스틱 회귀 분석	40
2.5 정리	42

CHAPTER 3

다층 퍼셉트론

43

3.1 MLP 구축과 학습	44
3.2 Dataset과 DataLoader	48
3.2.1 Dataset과 DataLoader	48

3.3	학습 효율화 팁	50
3.3.1	Dropout을 사용한 정규화	50
3.3.2	Batch Normalization를 사용한 학습 가속	53
3.4	신경망의 모듈화	55
3.4.1	자체 신경망 계층(커스텀 계층) 만들기	55
3.5	정리	57

CHAPTER 4 이미지 처리와 합성곱 신경망

59

4.1	이미지와 합성곱 계산	60
4.2	CNN을 사용한 이미지 분류	62
4.2.1	Fashion-MNIST	62
4.2.2	CNN 구축과 학습	65
4.3	전이 학습	69
4.3.1	데이터 준비	72
4.3.2	파이토치를 사용한 전이 학습	75
4.4	CNN 회귀 모델을 사용한 이미지 해상도 향상	80
4.4.1	데이터 준비	80
4.4.2	모델 작성 83	
4.5	DCGAN을 사용한 이미지 생성	89
4.5.1	GAN이란 89	
4.5.2	데이터 준비	90
4.5.3	파이토치를 사용한 DCGAN	91
4.6	정리	101

5.1 RNN이란?	104
5.2 텍스트 데이터의 수치화	106
5.3 RNN과 문장 분류	109
5.3.1 IMDb 리뷰 데이터	109
5.3.2 신경망 정의와 훈련	113
5.3.3 가변 길이 계열 처리	118
5.4 RNN을 사용한 문장 생성	121
5.4.1 데이터 준비	122
5.4.2 모델 정의 및 학습	124
5.5 인코더-디코더 모델을 사용한 기계 번역	129
5.5.1 인코더-디코더 모델이란	130
5.5.2 데이터 준비	131
5.5.3 파이토치를 사용한 인코더-디코더 모델	135
5.6 정리	142

6.1 행렬 인수분해	144
6.1.1 이론적 배경	144
6.1.2 MovieLens 데이터	145
6.1.3 파이토치에서 행렬 인수분해하기	147
6.2 신경망 행렬 인수분해	151
6.2.1 행렬 인수분해를 비선형화	151
6.2.2 부속 정보 이용	153
6.3 정리	160

CHAPTER 7 애플리케이션 적용**161**

7.1 모델 저장과 불러오기	162
7.2 플라스크를 사용한 웹 API화	164
7.3 도커를 이용한 배포	173
7.3.1 nvidia-docker 설치	174
7.3.2 파이토치의 도커 이미지 작성	175
7.3.3 웹 API 배포	176
7.4 ONNX를 사용한 다른 프레임워크와의 연계	179
7.4.1 ONNX란	179
7.4.2 파이토치 모델 익스포트	181
7.4.3 Caffe2에서 ONNX 모델 사용하기	183
7.4.4 ONNX 모델을 Caffe2 모델로 저장	184
7.5 정리	186

APPENDIX A 후련 상태 가시화**187**

A1.1 텐서보드를 사용한 가시화	188
--------------------	-----

APPENDIX B 컬래버레이터를 사용한 파이토치 개발 환경 구축**93**

B1.1 컬래버레이터를 사용한 파이토치 개발 환경 구축 방법	194
B1.1.1 컬래버레이터란	194
B1.1.2 장비 사양	194
B1.1.3 파이토치 환경 구축	195
B1.1.4 데이터 처리	201

찾아보기	205
------	-----



파이토치는 요즘 많이 사용하는 파이썬을 사용해서 쉽게 머신러닝을 구현할 수 있게 해준다. 머신러닝 등의 AI 관련 지식은 대부분 수학적 지식이 많이 필요해서 접근하기 쉽지 않은 것이 사실이다. 하지만 이 책은 최소한의 수학적 지식만 설명하고 나머지는 실습을 통해 머신러닝을 구현하도록 구성되어 있어서 초보자도 쉽게 접근할 수 있으리라 생각된다. 특히, 후반부에는 클라우드 서버를 기반으로 한 머신러닝 플랫폼을 만드는 방법도 설명하고 있어서 클라우드와 AI를 함께 접할 좋은 기회가 되리라 본다.

파이토치로 머신러닝을 구현하기 위해서는 고사양의 PC(GPU를 탑재한)가 필요하지만, 이 책은 구글이 제공하는 가상 파이토치 개발 환경을 이용하여 실습을 진행하므로 고 사양 PC(GPU)가 없어도 문제없이 실습을 진행할 수 있다. 실제로 코드를 실행해서 컴퓨터가 그림을 그려 내거나 외국어를 번역해 내는 것을 보다 보면 재미있게 실습할 수 있을 것이다.

10년 전에도 그랬지만, 여전히 IT 분야는 늘 새로운 기술들이 등장하고 있다. 엔지니어뿐만 아니라 데이터 과학자, 데이터 엔지니어 등의 새로운 직종도 생겨나고 있다. 엔지니어로서는 어쩌면 더 많은 진로를 선택할 수 있게 되어 다행일 수도 있지만, 그만큼 더 많은 분야를 공부해야 한다는 부담으로 다가올 수도 있다. 하지만 남보다 한발 앞서 새로운 기술을 익힌다면 자신을 내세울 수 있는 큰 장점이 될 것이다.

여담이지만 내가 처음 번역한 책은 서버 부하분산 책이다(2012년경으로 기억한다). 하지만 이 부하분산은 클라우드 시대가 되면서 중요도가 많이 떨어진 상태다. 아마 또 5년 후면 파이토치나 머신러닝보다 상위 개념의 기술이 나오지 않을까 싶기도 하다. 어찌 됐든 세상은 빠르게 변화하면서 기술도 빠르게 진화하고 있으니, 아마 IT 분야에서 계속 종사한다면 끊임없이 공부해야 하는 것이 우리의宿命일 것 같다.

싱가포르에서
윤진이 김완섭



딥러닝(deep learning), 신경망(neural network), 머신러닝(machine learning)은 이미지 처리(CV, Computer Vision)나 자연어 처리(NLP, Natural Language Processing), 음성 인식 등의 응용 분야에서 가장 주목받는 기술이다. IT 분야와 관련 없는 일을 하더라도 이 용어들은 어디선가 들어본 적이 있을 것이다. 특히 최근에는 AI나 인공지능이라는 키워드와 함께 빠르게 퍼져 나가고 있다.

딥러닝이나 신경망에 관심을 가지고 직접 사용해 보고 싶다는 생각에 공부를 시작하지만, 생각했던 것과 달리 실제로는 수많은 수식으로 구성돼 있어서 어디부터 손을 대야 할지 몰라 헤매는 경우가 많을 것이다.

이 책에서는 신경망의 기초인 선형 모델부터 시작해서 확률 모델(신경망을 포함한)의 기본적인 원리를 설명한다. 그리고 파이토치(PyTorch)를 사용해서 직접 프로그램을 만들어 보고, 그 원리를 깊이 있게 이해할 수 있도록 구성했다.

선형 모델이라는 매우 중요한 기초 개념을 설명할 때를 제외하고는 가능한 한 수식을 사용하지 않고 코드와 실제 예를 통해 설명하도록 노력했다. 간단한 데이터 식별부터 이미지 분류 및 생성, 문장 분류 및 생성, 번역, 그리고 추천 시스템 등 다양한 응용 예제를 사용해 실제 데이터가 움직이는 것을 보면서 이해할 수 있도록 주의를 기울였다.

이 책에서는 파이토치라는 파이썬(Python) 프레임워크를 사용한다. 파이썬이라는 프로그래밍 언어 자체는 데이터 과학 분야에서 가장 많이 사용되는 언어로 자리잡은 지 오래지만, 딥러닝 분야에서도 같은 역할을 하고 있다.

딥러닝의 파이썬 라이브러리로는 구글이 개발한 텐서플로(TensorFlow) 프레임워크가 가장 유명하지만, 심벌을 사용하는 프로그래밍 스타일 때문에 초보자가 접근하기 어렵다는 의견도 있다. 반면 이 책에서 다루는 파이토치는 페이스북을 중심으로 개발된 오픈소스 프로젝트로, 동적 네트워크라는 구조를 도입했으며, 일반적인 파이썬 프로그램과 같은 환경에서 간단하게 신경망을 구축할 수 있다는 점에서 많은 관심을 받고 있다. 특히, 해외 연구자들로부터 많은 지지를 받음으로 인해 최신 연구들이 파이토치를 사용해 구현되는 중이다. 연구 결과들도 깃허브(GitHub)를 통해 빠르게 공개되는 것이 당연시되고 있다. 아직 한글 자료는 부족하지만, 사용하기 쉽고 최신 연구 결과를 바로 적용할 수 있어서 서비스에 딥러닝을 곧바로 적용하고 싶은 사람에게는 최적의 프레임워크가 될 것이다.

이 책을 통해서 독자 여러분이 신경망이나 딥러닝, 그리고 머신러닝 등에 흥미를 가지고 실제로 자신의 업무에 적용할 수 있게 되기를 바란다.

지은이 두세교



MEMO

파이토치(PyTorch)

딥러닝이 유명해지기 전에는 루아(Lua)라는 스크립트 언어에서 사용할 수 있는 토치(Torch)라는 프레임워크가 존재했다. 파이토치는 이 토치와 똑같은 기반 코드를 사용하고 있지만, 토치를 사용해 보지 않았더라도 문제는 전혀 없다.



이 책은 다음과 같이 전반부(1장부터 3장)와 후반부(4장부터 7장)로 구성되어 있다.

1장에서는 파이토치의 패키지 구성을 살펴보고 어떤 기능이 있는지 대략적으로 정리해 본다. 또한, 텐서(Tensor)라는 파이토치의 가장 기본적인 데이터 구조를 실제로 사용해 본다.

2장에서는 선형 모델을 다룬다. 선형 모델을 학습하는 구조는 신경망에서도 동일하게 사용할 수 있으므로 매우 중요하다. 이 장의 전반부는 이 책에서는 유일하게 수식을 사용해 이론을 설명한다. 후반부에선 실제로 파이토치를 사용해 선형 회귀 모델과 로지스틱 회귀 모델을 구현해 보고 실제 데이터의 파라미터를 학습한다.

3장에서는 드디어 신경망을 다룬다. 여기서는 가장 기본적인 신경망인 다층 퍼셉트론을 파이토치를 사용해서 작성해 본다. 2장과 3장에서 배운 것은 뒤의 4장, 5장, 6장의 기초가 된다.

4장에서는 합성곱 신경망(CNN, Convolutional Neural Network)을 사용한 이미지 처리를 다룬다. CNN을 이용한 단순한 이미지 분류뿐만 아니라 저해상도의 이미지를 고해상도로 변환하거나 DCGAN(Deep Convolutional Generative Adversarial Networks)을 사용해서 새로운 이미지를 생성해 본다. 또한, 이미 학습이 완료된 모델을 자신의 데이터에 적용하는 전이 학습에 대해서 배운다.

5장에서는 순환 신경망(RNN, Recurrent Neural Network)을 사용한 자연어 처리에 대해 다룬다. RNN을 사용한 문장 분류, 문장 생성 방법과 두 개의 RNN을 결합한 인코더-디코더(Encoder-Decoder) 모델을 통해 영어와 스페인어 번역에도 도전해 본다.

6장에서는 신경망을 사용한 추천(recommendation) 시스템 구축에 대해 다룬다. 이미지나 자연어 등의 대표적인 예 이외에도 신경망이 사용되는 다양한 분야를 살펴보고, 파이토치 자체가 신경망 외에 다양한 모델을 기술할 수 있다는 것을 확인한다.

7장에서는 파이토치를 실제 애플리케이션에 내장하는 방법을 소개하며, 머신러닝 분야와 웹 분야의 다양한 파이썬 라이브러리가 제공되고 있는 것을 확인한다. 여기서는 파이토치와 플라스크(Flask)라는 웹 애플리케이션 프레임워크를 사용해서 웹 API를 실제로 만들고 이것을 도커(Docker)([메모](#) 참고)를 사용해서 패키징하는 방법을 배운다. 또한, 신경망의 최신 포맷인 ONNX(Open Neural Network Exchange)를 사용해서 파이토치로 작성한 모델을 다른 답러닝 프레임워크에 적용하는 방법을 소개한다.



MEMO

도커(Docker)

도커는 컨테이너형 가상화 기술로 최근 가장 활발히 사용되는 물이다. 컨테이너형 가상 환경은 버추얼박스(VirtualBox)나 VMWare 등과 비교해 실행 시간, 메모리, 디스크, 처리 속도 등 모든 측면에서 오버헤드가 적다는 장점이 있다. 또한, Dockerfile이라는 텍스트 파일로 도커 이미지를 작성하고 관리할 수 있어서 깃허브 등을 통해 공유하기 쉽다는 장점도 있다.

도커는 매우 간단한 명령만으로도 컨테이너를 사용해 서버를 가상화할 수 있다. 이 책의 7장에서 도커를 사용한 웹 API 개발에 대해 설명한다.

먼저, 이 책의 1장부터 3장까지를 읽은 후에 4장부터 7장 중에서 관심 있는 순서로 읽도록 하자. 7장은 4장에서 작성한 모델을 사용하지만, 4장의 지식이 반드시 필요한 것은 아니다.

참고로, 이 책에서 사용하고 있는 것 중에 필자가 직접 수집한 데이터나 학습 완료된 모델 등은 다음 깃허브 리포지토리를 통해 공개하고 있다. 질문이나 수정 사항 등이 있으면 여기서 이슈를 작성해 주면 가능한 한 답변을 달도록 하겠다.

- 해당 도서 리포지토리

<https://github.com/lucidfrontier45/PyTorch-Book>



About the SAMPLE: 이 책의 개발 환경과 예제 프로그램

각 장에서 사용하는 예제는 표 1에 있는 환경에서 문제없이 실행되는 것을 확인했다.

표 1 실행 환경

항목	우분투	컬래버러터리
OS	Ubuntu 16.04	Windows 10 (64bit)
CPU	Intel Core i7 7700K	Intel Core i5
메모리	16GB	8GB
GPU	NVIDIA GeForce GTX 1060(6GB)	Intel HD
파이썬	3.6	3.6
파이토치	1.0	1.0

주의: 이 책의 예제 데이터 테스트 환경과 GPU

이 책의 예제를 실행하려면 GPU(엔비디아의 GeForce GTX 1060과 동급)가 필요하다. GPU가 없는 경우 CPU에 큰 부담을 줄 수 있어서 권장하지 않는다. GPU가 없는 독자를 위해서 브라우저를 통해 동일 환경을 제공하는 구글 컬래버러터리(Google Colaboratory)에서도 테스트가 가능하도록 구성했다(단, 7장의 전반부 및 부록 A는 실행이 안 된다).

부록 데이터

부록 데이터(이 책에 수록된 예제 코드)는 다음 사이트를 통해 다운로드할 수 있다.

- 부록 데이터 다운로드 사이트

URL https://github.com/Jpub/PyTorch_firststep

주의

부록 데이터와 관련된 권리는 지은이 및 주식회사 쇼에이사가 소유한다. 허가 없이 배포하거나 웹사이트에 게재할 수 없다. 부록 데이터의 제공은 예고 없이 종료될 수 있다.

면책 사항

부록 데이터 내용은 2018년 8월 현재의 법령 등에 근거한다. 부록 데이터에 기재된 URL 등은 예고 없이 변경될 수 있다. 부록 데이터는 지은이 및 출판사의 확인을 거쳤지만, 그 내용에 대해서 보증할 수 없으며, 내용이나 예제를 사용한 운용 결과에 대해서는 책임을 지지 않는다.

부록 데이터에 기재된 회사명, 제품명은 모두 각 사의 상표 및 등록 상표다.

저작권에 대해서

예제 데이터의 저작권은 지은이 및 주식회사 쇼에이사, 제이펍이 소유하고 있다. 개인의 학습 목적 이외의 용도로는 사용할 수 없다. 허가 없이 인터넷을 통해 배포할 수 없다. 개인적인 용도로 사용하는 경우에는 코드 변경 등을 자유롭게 할 수 있다. 상업적인 목적으로 사용하고자 하는 경우에는 제이펍과 논의해야 한다.



✧ **곽상영(NeuRobo)**

딥러닝을 이제 막 시작한 사람으로서 텐서플로(TensorFlow)나 케라스(Keras) 외에 파이토치(PyTorch)에 대해 알게 된 좋은 계기가 된 듯합니다. 코드도 케라스 코드를 봤을 때의 느낌처럼 텐서플로보다 직관적이라는 생각이 듭니다. 다양한 예제 또한 파이토치를 알아가는 데 한몫했구요. 다른 책에서 볼 수 없었던 팁도 많이 얻어 갑니다.

✧ **김용천(Microsoft MVP)**

기본적인 파이썬 문법에 익숙하고 딥러닝 알고리즘을 간략히 알고 있는 독자가 대상입니다. 다양한 실용 예제를 통해 인공지능 알고리즘을 종류별로 쉽게 실습해 볼 수 있으며, 딥러닝의 실무 업무의 가이드라인을 얻을 수 있습니다. 전자기기 없는 도시의 삶은 생각하기 어렵듯이 이제 딥러닝 역시 필수적인 서비스로 우리 생활의 일부가 되어 가고 있습니다. 현재의 기술로 어떠한 서비스가 가능한지 쉽고 빠르게 정리해 보고픈 개발자에게 이 책을 적극 추천합니다.

✧ **송근(네이버)**

현재 파이토치가 왜 주목받고 있는지 쉽게 풀어 설명하는 책입니다. 파이토치의 장점처럼 간결한 설명과 예제로 하나씩 풀어 나가고, 중간중간 시각화된 결과를 통해 빠르게 습득할 수 있습니다. 이런 이유로 이 책은 파이토치 입문서로 적합한 책입니다. 이번 책은 정말 깔끔한 번역과 구성으로 읽기 쉬웠습니다.

송현(규슈대 대학원)

내용이 정말 알맞게 들어가 있었습니다. 딥러닝에 관한 기초적인 내용으로 많은 공간을 차지하지도 않았으며, 파이토치를 각 상황에서 어떻게 써야 하는지를 제대로 보여 준 책이라고 생각합니다. 코드 또한 상당히 깔끔하고 가독성 좋게 적혀 있었고, 번역도 훌륭해서 어색한 부분 없이 이해가 잘 되었습니다.

신성기(투비소프트)

이 책은 요즘 많이 출판되는 딥러닝 초보자를 위한 책들과는 다른 점이 세 가지 있습니다. 첫째, 파이토치를 다룬다는 것, 둘째, 사용해 볼 수 있는 예제가 많다는 것, 셋째, 구글 컬래버레이터 환경에서 실습할 수 있다는 것입니다. 파이토치는 구글의 텐서플로와 함께 아주 널리 쓰이는 딥러닝 프레임워크입니다. 특히, 대부분의 실습을 구글 클라우드인 컬래버레이터에서 하면서 사용법을 익히는 것도 많은 도움이 될 것이라고 생각합니다. 평소 관심이 있던 분야여서 그런지 책의 내용이 전반적으로 재미있고 친근하게 다가와서 좋았습니다. 텐서플로를 주로 사용하다 요즘 들어 파이토치로 된 코드들이 많아져서 관심 있게 보고 있던 차에 파이토치 코드 실습을 많이 해 볼 수 있어서 정말 좋았습니다.



제이펍은 책에 대한 애정과 기술에 대한 열정이 뜨거운 베타리더들로 하여금
출간되는 모든 서적에 사전 검증을 시행하고 있습니다.

1. The first part of the chapter discusses the importance of the...
 2. The second part of the chapter discusses the importance of the...
 3. The third part of the chapter discusses the importance of the...

1. The first part of the chapter discusses the importance of the...
 2. The second part of the chapter discusses the importance of the...
 3. The third part of the chapter discusses the importance of the...
 4. The fourth part of the chapter discusses the importance of the...
 5. The fifth part of the chapter discusses the importance of the...

6. The sixth part of the chapter discusses the importance of the...
 7. The seventh part of the chapter discusses the importance of the...
 8. The eighth part of the chapter discusses the importance of the...
 9. The ninth part of the chapter discusses the importance of the...
 10. The tenth part of the chapter discusses the importance of the...

11. The eleventh part of the chapter discusses the importance of the...
 12. The twelfth part of the chapter discusses the importance of the...
 13. The thirteenth part of the chapter discusses the importance of the...
 14. The fourteenth part of the chapter discusses the importance of the...
 15. The fifteenth part of the chapter discusses the importance of the...



PROLOGUE

개발 환경 준비

이 책의 검증 환경

이 책의 검증 환경에 대해 설명한다.

0.1.1 OS 환경: 우분투 16.04

필자는 우분투(Ubuntu 16.04)를 사용해 내용을 검증 및 집필했다. 파이토치뿐만 아니라 머신러닝이나 데이터 분석 시에도 우분투가 최선의 환경이라고 생각하고 있으므로 우분투 16.04를 전제로 한다. 참고로, 파이토치는 0.4부터 윈도우를 지원한다. 윈도우나 맥 OS를 사용하는 경우도 설치 방법은 우분투와 거의 같다. 뒤에서 설명하겠지만 개발 환경으로 미니콘다(Miniconda)를 사용하면 편리하다. 다음 도커 이미지를 도커 허브(Docker Hub)를 통해 공개하고 있으니 빠른 개발 환경 구축을 원하는 경우 사용하도록 하자.

- 도커 허브

URL <https://hub.docker.com/r/lucidfrontier45/pytorch/>

0.1.2 엔비디아의 GPU

4장부터 6장까지의 내용은 무거운 처리를 실행하므로 CUDA(**메모** 참고)를 사용할 수 있는 엔비디아(NVIDIA)의 GPU가 필요하다. CPU로도 실행할 수 있지만, 심한 경우 계산 시간이 10배 정도 오래 걸릴 수도 있다. 따라서 GPU 사용을 권장한다. 필자는 지포

스(GeForce) GTX 1060(6GB)라는 GPU를 사용하고 있으며, 30만 원 정도(2018년 8월 시점)에 구입할 수 있다.



MEMO

쿠다(CUDA)

쿠다는 엔비디아가 제공하는 GPU용의 과학 계산 플랫폼으로 전용 컴파일러나 라이브러리로 구성돼 있다. 쿠다는 C언어로 작성할 수 있을 뿐 아니라 행렬 계산이나 FFT(Fast Fourier Transform, 고속 푸리에 변환) 등에 자주 사용되는 라이브러리도 제공한다. 또한, cuDNN이라는 심화 학습용 라이브러리도 최근 들어 공개했다. 유명한 딥러닝 프레임워크는 모두 쿠다를 지원하고 있다.

0.1.3 클라우드에서 GPU를 탑재한 인스턴스 실행하기

본인 PC에 GPU를 탑재할 수 없는 경우는 AWS(Amazon Web Service)나 Azure, GCP(Google Cloud Platform) 등의 클라우드에서 GPU를 탑재한 인스턴스를 사용하는 방법도 있다. 한 시간에 1달러 정도 한다. 참고로, 파이토치에 쿠다가 포함돼 있으므로 엔비디아의 드라이버만 설치해 주면 사용할 수 있다. apt-get 등으로 nvidia-<version>의 패키지를 설치하면 된다. 필자는 nvidia-384를 사용해서 검증했다.

콘솔 모드에서 다음 명령을 실행한다(필자의 환경).



우분투 환경

```
$ sudo apt-get install nvidia-384
$ sudo reboot
```

드라이버가 설치됐다면 다음 명령을 실행해서 확인할 수 있다.

우분투 환경

```
$ nvidia-smi
```

```
Sat Aug 4 22: 51: 00 2018
```

```
+-----+  
| NVIDIA-SMI 384. 130 Driver Version: 384. 130 |  
+-----+  
| GPU Name Persistence-M | Bus-Id Disp. A | Volatile Uncorr. ECC |  
| Fan Temp Perf Pwr: Usage/ Cap | Memory-Usage | GPU-Util Compute M. |  
+-----+  
| 0 GeForce GTX 106... Off | 00000000: 01: 00. 0 On | N/ A |  
| 35% 31 C P 8 ERR! / 75 W | 283 MiB / 4035 MiB | 0% Default |  
+-----+  
(... 생략...)
```

개발 환경 구축

이 책의 개발 환경 구축 방법을 소개한다.

0.2.1 미니콘다 설치

파이썬 개발 환경은 미니콘다라는 배포 버전을 사용한다. 다음 URL을 통해 리눅스 (Linux)용 64비트 파이썬 3.6 이상을 다운로드한다(그림 0.1 → 그림 0.2 → 그림 0.3). 여기서는 'Miniconda3-latest-Linux-x86_64.sh'를 다운로드한다.

- 미니콘다

URL <https://docs.conda.io/en/latest/miniconda.html>

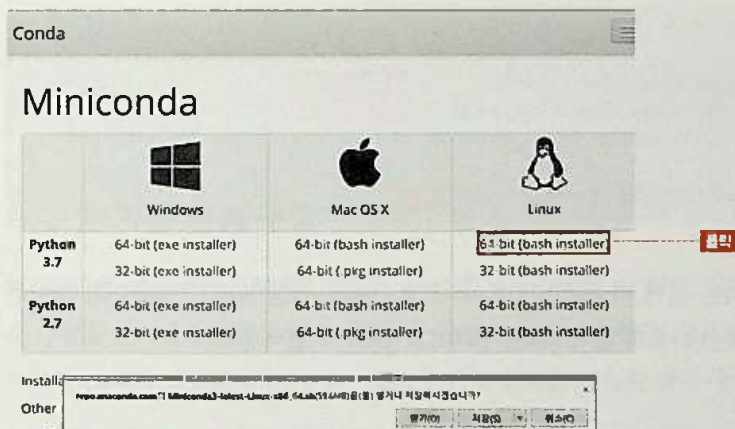


그림 0.1 미니콘다 웹사이트

Miniconda

	 Windows	 Mac OS X	 Linux
Python 3.7	64-bit (exe installer) 32-bit (exe installer)	64-bit (bash installer) 64-bit (.pkg installer)	64-bit (bash installer) 32-bit (bash installer)
Python 2.7	64-bit (exe installer) 32-bit (exe installer)	64-bit (bash installer) 64-bit (.pkg installer)	64-bit (bash installer) 32-bit (bash installer)

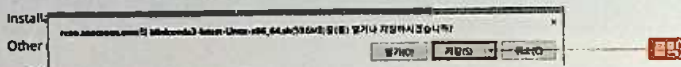


그림 0.2 미니콘다 다운로드(이 화면 이후의 저장 화면은 생략)

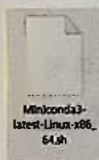


그림 0.3 다운로드한 파일 선택

파일을 다운로드했으면 다음 명령을 실행한다.

② 우분투 환경

```
$ cd /home/(사용자명)/(다운로드 디렉터리)
$ sh Miniconda3-latest-linux-x86_64.sh
```

In order to continue the installation process, please review the license agreement.

Please, press ENTER to continue. — **Enter 키 누름**

그러면 More라는 곳에 라이선스가 표시되므로 **(Space)** 키를 눌러서 진행한다. 라이선스 허가 화면에선 다음과 같이 'yes'를 입력하고 **(Enter)** 키를 누른다.

② 우분투 환경

```
Do you accept the license terms? [yes | no] [no] >>> -- 'yes' 입력하고 Enter 키 누름
```

설치 위치를 확인하는 메시지가 표시되므로 변경을 원하지 않으면 그냥 **Enter** 키를 누르면 된다. 이 책에서는 `/home/(사용자명)/miniconda3`에 설치하고 있다.

우분투 환경

```
Miniconda3 will now be installed into this location: /home/(사용자명)/ miniconda3
```

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

```
[/home/(사용자명)/miniconda3] >>> Enter 키 누름
```

경로를 지정하면 설치할지 묻는 메시지가 표시된다. 'yes'를 입력하고 **Enter** 키를 누른다.

우분투 환경

```
Do you wish the installer to prepend the Miniconda3 install location  
to PATH in your /home/(사용자명)/.bashrc ? [yes | no]
```

```
[no] >>> yes 키 누름
```

설치가 끝나면 `exit` 명령으로 빠져나온다.

우분투 환경

```
For this change to become active, you have to open a new terminal.  
Thank you for installing Miniconda3!  
$ exit
```

0.2.2 가상 환경 구축

미니콘다를 사용하면 프로젝트 단위로 가상 환경을 구축할 수 있다. 구체적으로는 다음 명령을 순서대로 실행해서 파이토치용 가상 환경을 구축한다. 참고로, 이 책은 파이토치 1.0에서 검증했다. 먼저, 파이토치용 가상 환경을 다음과 같이 생성한다.

우분투 환경

```
$ /home/(사용자명)/miniconda3/bin/conda create -n pytorch python=3.6
Proceed ([y]/ n)? — y 입력하고 Enter 키 누름
Downloading and Extracting Packages
sqlite-3. 24. 0      | 1. 8 MB | ##### | 100%
openssl-1. 0. 2 o    | 3. 4 MB | ##### | 100%
python-3. 6. 6       | 29. 4 MB | ##### | 100%
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use:
# > source activate pytorch
#
# To deactivate an active environment, use:
# > source deactivate
#
```

파이토치 가상 환경을 다음 명령을 통해 실행한다.

우분투 환경

```
$ source /home/(사용자명)/miniconda3/bin/activate pytorch
(pytorch) $
```

데이터 분석 시에 자주 사용되는 라이브러리들(pandas, jupyter, matplotlib, scipy, scikit-learn, pillow, tqdm, cython)을 설치한다.

우분투 환경

```
(pytorch) $ conda install pandas jupyter matplotlib \
> scipy scikit-learn \
> pillow tqdm cython
Proceed ([y]/ n)? — y 입력하고 Enter 키 누름
(_종료_)
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

다음은 파이토치 0.4를 설치한다.

우분투 환경

```
(pytorch) $ conda install pytorch=0.4 torchvision -c pytorch// -- PyTorch=0.4.1
Proceed ([y]/ n)? y
(중략...)
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

7장에서 사용하는 웹 API 관련 라이브러리(flask, smart_getenv, gunicorn)를 다음과 같이 설치한다.

우분투 환경

```
(pytorch)$ pip install flask smart_getenv gunicorn
Successfully installed Werkzeug-0.14.1 click-6.7 flask-1.0.2 gunicorn-19.9.0 itsdangerous-0.24 smart-getenv-1.1.0
You are using pip version 10.0.1, however version 18.0 is available.
You should consider upgrading via the 'pip install --upgrade pip' command.
```

7장에서 사용할 caffe2를 다음과 같이 설치한다.

우분투 환경

```
(pytorch)$ conda install -c caffe2 caffe2 protobuf
Proceed ([y]/n)? y
(중략...)
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

7장에서 사용할 onnx 관련 라이브러리를 다음과 같이 설치한다.

우분투 환경

```
(pytorch)$ sudo apt install git build-essential g++ cmake
(pytorch)$ pip install git+https://github.com/onnx/onnx.git@367995b1439e→478122780ffc9d4e3ee8918fb7ad
Successfully built onnx
Installing collected packages: typing-extensions, onnx
Successfully installed onnx-1.2.1 typing-extensions-3.6.5
```


③ 설치한 가상 환경 사용하기

이후에 설치한 환경을 사용할 때는 다음 명령을 실행한다.

❶ 우분투 환경

```
$ source /home/(사용자명)/miniconda3/bin/activate pytorch
```

❷ 주피터 노트북에 대해

주피터 노트북(Jupyter Notebook)은 브라우저에서 인터랙티브하게 파이썬을 실행할 수 있게 해주는 프로그램이다. 이것은 가상 환경 구축 과정에서 설치된다. 이 책의 코드를 실행할 때는 이 주피터 노트북의 사용을 권한다. 이것을 실행할 때는 단말에서 다음과 같은 순서로 명령을 실행해야 한다.

먼저, 사전에 노트북 파일을 저장할 디렉터리를 다음 명령으로 생성한다.

❶ 우분투 환경

```
$ cd /home/(사용자명)/  
$ mkdir notebooks
```

이어서 주피터 노트북을 다음과 같이 실행한다.

❶ 우분투 환경

```
$ jupyter notebook --port=8888 --ip="0.0.0.0" \  
--notebook-dir=notebooks
```

PC에서는 위 명령으로 브라우저가 실행되면 그대로 사용하면 된다. 가상 머신이나 클라우드 등에서 사용할 때는 실제 IP 주소와 터미널에 표시되는 토큰을 조합한 URL을 브라우저에 직접 입력해서 접속해야 한다.

그외 구글의 컬래버레이터라는 주피터 노트북을 무료로 제공하는 클라우드 서비스를 사용하는 방법도 있다. 이에 대해서는 부록 2에 정리했으니 참고하도록 하자. 윈도우 환경의 독자는 이 컬래버레이터를 사용하도록 하자.



CHAPTER

1

파이토치의 기본

이 장에서는 파이토치의 패키지 구성을 전체적으로 살펴보고, 기본 데이터 구조인 텐서 사용법을 알아본다. 그리고 텐서의 핵심 중 하나인 autograd를 사용해서 자동 미분에 대해 설명한다.

파이토치의 구성

먼저, 파이토치 패키지 구성을 살펴보자.

표 1.1의 패키지를 사용하게 되는데 개요 정도만 이해해 두면 된다.

1.1.1 파이토치의 전반적인 구성

파이토치 구성은 **표 1.1**과 같다.

표 1.1 파이토치의 패키지 구성

구성 내용	설명
torch	메인 네임스페이스로 텐서 등의 다양한 수학 함수가 이 패키지에 포함돼 있다. NumPy와 같은 구조를 가지고 있다
torch.autograd	자동 미분을 위한 함수가 포함돼 있다. 자동 미분의 on/off를 제어하는 컨텍스트 매니저(enable_grad/no_grad)나 자체 미분 가능 함수를 정의할 때 사용하는 기반 클래스인 'Function' 등이 포함돼 있다
torch.nn	신경망을 구축하기 위한 다양한 데이터 구조나 레이어 등이 정의돼 있다. 예를 들어, Convolution이나 LSTM, ReLU 등의 활성화 함수나 MSELoss 등의 손실 함수도 포함된다
torch.optim	확률적 경사 하강법(SGD, Stochastic Gradient Descent)을 중심으로 한 파라미터 최적화 알고리즘이 구현돼 있다
torch.utils.data	SGD의 반복 연산을 실행할 때 사용하는 미니 배치용 유틸리티 함수가 포함돼 있다
torch.onnx	ONNX(Open Neural Network Exchange) 포맷으로 모델을 익스포트(export)할 때 사용한다. ONNX는 서로 다른 딥러닝 프레임워크 간에 모델을 공유할 때 사용하는 새로운 포맷이다

활성화 함수, 손실 함수, SGD 등의 용어가 익숙하지 않을 수 있지만, 2장, 3장에서 자세히 다루니 걱정하지 말자. 또한, ONNX에 대해서는 7장에서 설명한다.

1.2

텐서

여기선 텐서(Tensor)라는 파이토치의 가장 기본이 되는
데이터 구조와 그 기능에 관해 설명한다.

텐서는 그 명칭에서도 알 수 있듯이 다차원 배열을 처리하기 위한 데이터 구조다. NumPy의 ndarray와 거의 같은 API를 지니고 있으며, GPU를 사용한 계산도 지원한다. 텐서는 각 데이터형별로 정의돼 있다. 예를 들어, 32비트의 유동소수점은 `torch.FloatTensor`를, 64비트의 부호 있는 정수라면 `torch.LongTensor`를 사용한다.

또한, GPU상에서 계산할 때는 `torch.cuda.FloatTensor` 등을 사용한다. 참고로, `Tensor`는 `FloatTensor`의 별칭(alias)이다. 어떤 형의 텐서이건 `torch.tensor`라는 함수로 작성할 수 있다.

1.2.1 텐서 생성과 변환

텐서는 다양한 방법으로 만들 수 있다. `torch.tensor` 함수에 중첩 구조의 다차원 list나 ndarray를 지정하는 방법 외에도 NumPy처럼 `arange`, `linspace`, `logspace`, `zeros`, `ones` 등의 상수를 사용해 만드는 방법도 있다. **리스트 1.1**에서는 몇 가지 텐서 생성 예를 보여준다.

리스트 1.1 텐서 생성 예(이후, 특별한 언급이 없다면 모두 주피터 노트북의 입력(In)과 출력(Out)이다)

In

```
import numpy as np
import torch

# 중첩 list를 지정
t = torch.tensor([[1, 2], [3, 4.]])

# device를 지정하면 GPU에 텐서를 만들 수 있다
t = torch.tensor([[1, 2], [3, 4.]], device="cuda:0")

# dtype을 사용해 데이터형을 지정하여 텐서를 만들 수 있다
t = torch.tensor([[1, 2], [3, 4.]], dtype=torch.float64)

# 0부터 9까지의 수치로 초기화된 1차원 텐서
t = torch.arange(0, 10)

# 모든 값이 0인 100 × 10의 텐서를
# 작성해서 to 메서드로 GPU에 전송
t = torch.zeros(100, 10).to("cuda:0")

# 정규 난수로 100 × 10의 텐서를 작성
t = torch.randn(100, 10)

# 텐서의 shape은 size 메서드로 확인 가능
t.size()
```

Out

```
torch.Size([100, 10])
```

텐서는 NumPy의 ndarray로 쉽게 변환할 수 있다. 단, GPU상의 텐서는 그대로 변환할 수 없으며, CPU로 이동 후에 변환해야 한다(**리스트 1.2**).

리스트 1.2 텐서 변환 예

In

```
# numpy 메서드를 사용해 ndarray로 변환
t = torch.tensor([[1, 2], [3, 4.]])
x = t.numpy()

# GPU상의 텐서는 to 메서드로,
# CPU의 텐서로 이동(변환)할 필요가 있다
t = torch.tensor([[1, 2], [3, 4.]], device="cuda:0")
x = t.to("cpu").numpy()
```


1.2.2 텐서의 인덱스 조작

텐서는 ndarray와 마찬가지로 인덱스 조작을 지원한다. 배열 $A[i,j]$ 의 i, j 를 첨자 인덱스(Index)라고 하며, 이 값을 지정해서 배열의 값을 가져오거나 변경하는 것을 '인덱스 조작'이라고 한다(리스트 1.3). 스칼라를 사용한 지정 외에도 슬라이스, 첨자 리스트, ByteTensor의 마스크 배열(메모 참고) 등을 지원한다.

텐서의 인덱스 조작 예



```
t = torch.tensor([[1,2,3], [4,5,6.]])

# 스칼라 첨자 지정
t[0, 2]

# 슬라이스로 지정
t[:, :2]

# 리스트로 지정
t[:, [1,2]]

# 마스크 배열을 사용해서 3보다 큰 부분만 선택
t[t > 3]

# [0, 1]의 요소를 100으로 설정
t[0, 1] = 100

# 슬라이스를 사용한 일괄 대입
t[:, 1] = 200

# 마스크 배열을 사용해서 특정 조건의 요소만 치환
t[t > 10] = 20
```



MEMO

마스크 배열

마스크 배열이란 원 배열과 크기는 같으면서 각 요소가 True/False로 설정돼 있는 배열을 가리킨다. 예를 들어, $a = [1, 2, 3]$ 라는 배열에서 $a > 2$ 를 실행하면 $[False, False, True]$ 라는 마스크 배열이 생성된다.


```
# 벡터와 스칼라의 덧셈
v2 = v + 10
# 제곱도 같은 방식
v2 = v ** 2
# 동일 길이의 벡터 간 뺄셈
z = v - w
# 여러 가지 조합
u = 2 * v - w / 10 + 6.0

# 행렬과 스칼라
m2 = m * 2.0
# 행렬과 벡터
# (2, 3)인 행렬과 (3,)인 벡터이므로 브로드캐스트가 작동
m3 = m + v
# 행렬 간 처리
m4 = m + m
```



MEMO

브로드캐스트

- SciPy.org: broadcasting

URL <https://docs.scipy.org/doc/numpy-1.13.0/user/basics.broadcasting.html>

파이토치는 텐서를 통해 다양한 함수를 제공한다. abs, sin, cos, exp, log, sqrt 등 텐서의 각 요소에 사용할 수 있는 것도 있고, sum, max, min, mean, std 등의 집계 함수도 제공하므로 ndarray와 동일하게 사용할 수 있다. 또한, 텐서에 사용되는 대부분의 함수는 텐서의 메서드로도 제공된다(**리스트 1.5**).

리스트 1.5 수학 함수



```
# 100 × 10의 테스트 데이터 생성
X = torch.randn(100, 10)

# 수학 함수를 포함하는 수식
y = X * 2 + torch.abs(X)
# 평균치 구하기
m = torch.mean(X)
# 함수가 아닌 메서드로도 사용할 수 있다
m = X.mean()
```

```
# 집계 결과는 0차원의 텐서로 item 메서드를 사용해서
# 값을 추출할 수 있다
m_value = m.item()
# 집계는 차원을 지정할 수도 있다. 다음은 행 방향으로 집계해서,
# 열 단위로 평균값을 계산한다
m2 = X.mean(0)
```

수학 함수 외에도 텐서의 차원을 변경하는 view나 텐서를 결합하는 cat, stack, 그리고 차원을 교환하는 (나 transpose도 자주 사용된다. view는 ndarray의 reshape 함수와 동일하며, cat는 다른 길이의 복수의 텐서를 하나로 묶을 때 사용한다. transpose는 행렬의 전치 외에도 이미지 데이터의 데이터 형식을 HWC(높이, 너비, 색)순에서 CHW(색, 높이, 너비)순으로 변경할 때도 사용된다([리스트 1.6](#)).

리스트 1.6 텐서의 인덱스 조작 예



```
x1 = torch.tensor([[1, 2], [3, 4.]]) # 2x2
x2 = torch.tensor([[10, 20, 30], [40, 50, 60.]]) # 2x3

# 2x2를 4x1로 보여 준다
x1.view(4, 1)
# -1은 표현할 수 있는 자동화된 값으로 대체되며, 한 번만 사용할 수 있다
# 아래 예에선 -1을 사용하면 자동으로 4가 된다
x1.view(1, -1)

# 2x3을 전치해서 3x2로 만든다
x2.t()

# dim=1로 결합하면 2x5의 텐서를 만든다
torch.cat([x1, x2], dim=1)

# HWC를 CHW로 변환
# 64x32x3의 데이터가 100개
hwc_img_data = torch.rand(100, 64, 32, 3)
chw_img_data = hwc_img_data.transpose(1, 2).transpose(1, 3)
```

선형 대수는 [표 1.2](#)의 연산자를 사용할 수 있으며, [리스트 1.7](#)의 연산을 할 수 있다. 특히, 큰 행렬의 곱이나 특이값 분해 등 계산량이 많은 것은 GPU상에서 실행하면 큰 폭으로 계산 시간을 단축할 수 있다.

표 1.2 선형 대수의 연산자

연산자	설명
dot	벡터 내적
mv	행렬과 벡터의 곱
mm	행렬과 행렬의 곱
matmul	인수의 종류에 따라 자동으로 dot, mv, mm을 선택해서 실행
gesv	LU 분해를 사용한 연립 방정식의 해
eig, symeig	고유값 분해. symeig는 대칭 행렬보다 효율이 좋은 알고리즘
svd	특이값 분해

연산 예



```

m = torch.randn(100, 10)
v = torch.randn(10)

# 내적
d = torch.dot(v, v)

# 100 × 10의 행렬과 길이 10인 벡터의 곱
# 결과는 길이 100인 벡터
v2 = torch.mv(m, v)

# 행렬곱
m2 = torch.mm(m.t(), m)

# 특이값 분해
u, s, v = torch.svd(m)

```


1.3

텐서와 자동 미분

여기서는 텐서와 자동 미분에 대해 설명한다.

텐서에는 `requires_grad`라는 속성이 있어서 이것을 `True`로 설정하면 자동 미분 기능이 활성화된다(메모 참고). 신경망을 다루는 경우 파라미터나 데이터가 모두 이 기능을 사용한다. `requires_grad`가 적용된 텐서에 다양한 계산을 하게 되면 계산 그래프가 만들어지며, 여기에 `backward` 메서드를 호출하면 그래프로부터 자동으로 미분을 계산한다. 다음은 L 을 계산해서 a_k 로 미분한다.

$$y_i = \mathbf{a} \cdot \mathbf{x}_i, \quad L = \sum_i y_i$$

이 간단한 예는 해석적으로 풀면 다음과 같다.

$$\frac{\partial L}{\partial a_k} = \sum_i x_{ik}$$

이것을 자동 미분으로 구해 보겠다(리스트 1.8).



MEMO

자동 미분과 Variable

파이토치 0.3 이전 버전에선 자동 미분을 사용하기 위해 `Variable`이라는 클래스로 텐서를 래핑(wrapping)해야 했지만, 0.4부터는 텐서와 `Variable`이 통합됐다.

리스트 1.8 자동 미분

 In

```
x = torch.randn(100, 3)
# 미분의 변수로 사용하는 경우는 requires_grad를 True로 설정
a = torch.tensor([1, 2, 3.], requires_grad=True)

# 계산을 통해 자동으로 계산 그래프가 구축된다
y = torch.mv(x, a)
o = y.sum()

# 미분을 실행
o.backward()

# 분석 답과 비교
a.grad != x.sum(0)
```

 Out

```
tensor([ 0,  0,  0], dtype=torch.uint8)
```

 In

```
# x는 requires_grad가 False이므로 미분이 계산되지 않는다
x.grad is None
```

 Out

```
True
```

분석 답과 자동 미분으로 구한 답이 일치하는 것을 알 수 있다. 이처럼 간단한 예에서는 자동 미분의 장점을 느끼기 어려울 수도 있지만, 신경망처럼 복잡한 함수에서 미분의 체인 규칙이 연속되는 경우에는 매우 중요한 역할을 한다.

정리

이 장에서 설명한 내용을 정리한다.

텐서는 NumPy의 ndarray와 똑같은 방식으로 사용할 수 있는 다차원 배열로 GPU를 사용한 계산도 지원하고 있어서 규모가 큰 행렬의 연산 등에 그 위력을 발휘한다. 텐서는 자동으로 미분을 계산할 수 있어서 신경망 최적화에 중요한 역할을 한다.



CHAPTER

2

최대 우도 추정과 선형 모델

이 장에서는 신경망이나 딥러닝의 기초인 최대 우도 추정과 선형 모델에 대해 설명하고, 자주 사용되는 선형 모델인 선형 회귀와 로지스틱 회귀를 통해 파이토치 사용법을 살펴본다. 이 장은 수식을 사용한 설명이 많이 나오지만, 반대로 여기서 다루는 식만 이해하면 신경망 뿐만 아니라 다양한 확률 모델의 가장 중요한 개념을 이해할 수 있게 된다. 이 장 이후에는 수식이 거의 등장하지 않으니 조금만 힘을 내서 읽어 보도록 하자.

확률 모델과 최대 우도 추정

확률 모델과 최대 우도 추정에 대해 설명한다.

확률 모델과 최대 우도 추정은 딥러닝 모델 중에서도 가장 많이 사용되는 중요한 프레임워크다. 신경망이 이 프레임워크를 이용한다.

확률 모델이란, 다음과 같이 변수 x 가 파라미터 θ 를 지니는 확률 분포 $P(x|\theta)$ 로부터 생성된다고 가정하는 모델이다.

$$x \sim P(x|\theta)$$

$P(x|\theta)$ 는 x 가 연속 변수인 경우는 정규 분포가 될 수 있다.

$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(x-\mu)^2}{2\sigma^2} \right]$$

이산 변수, 특히 동전 던지기처럼 $[0, 1]$ 인 경우는 다음과 같이 베르누이 분포(Bernoulli Distribution)가 될 수 있다.

$$B(x|p) = p^x (1-p)^{1-x}$$

서로 독립된 N 개의 데이터 $X = (x_0, x_1, \dots)$ 이 주어졌을 때 다음과 같이 각 데이터의 확률 함수 값의 곱을 θ 의 함수라고 하면, 이때 θ 가 가능 정도가 되며 우도(Likelihood)라고 부른다.

$$L(\theta) = \prod_n P(x_n|\theta)$$

우도는 확률 모델에서 가장 중요한 양으로, 우도를 최대로 만드는 파라미터 θ 를 구하는 것을 **최대 우도 추정(MLE, Maximum Likelihood Estimation)**이라고 한다. 보통은 계산이 쉽다는 이유로 대수 우도(log likelihood)의 형태를 취하기도 한다.

$$\ln L(\theta) = \sum_n \ln P(x_n|\theta)$$

예를 들어, 정규 분포의 기댓값 파라미터 μ 의 최대 우도 추정은 다음과 같이 대수 우도를 μ 에 대한 편미분으로 두고, 미분이 0이 되는 방정식을 풀어서 구한다. 결과적으로 기댓값 파라미터 μ 의 최대 우도 추정은 모든 x 의 평균값이 된다.

$$\begin{aligned}\ln L(\theta) &= -\frac{N}{2} \ln 2\pi\sigma^2 - \frac{1}{2\sigma^2} \sum_n (x_n - \mu)^2 \\ \frac{\partial}{\partial \mu} \ln L(\theta) &= -\frac{1}{\sigma^2} \sum_n (x_n - \mu) = 0 \\ \mu &= \frac{1}{N} \sum_n x_n = \bar{x}\end{aligned}$$

마찬가지로 베르누이 분포에 대해서도 p 의 최대 우도 추정을 구하면 다음과 같이 된다. 여기서 $x = 1$ 의 개수를 M 이라고 하면 다음과 같이 된다.

$$\begin{aligned}\sum_n x_n &= M \\ \ln L(\theta) &= \sum_n x_n \ln p + (1 - x_n) \ln(1 - p) \\ &= N \ln p + (N - M) \ln(1 - p) \\ \frac{\partial}{\partial p} \ln L(\theta) &= -\frac{M}{p} + \frac{N - M}{1 - p} = 0 \\ p &= \frac{M}{N}\end{aligned}$$

p 는 $x = 1$ 인 횟수의 비율이라는 것을 알 수 있다.

확률적 경사 하강법

최대 우도 추정을 수치적으로 구하는 방법인 경사 하강법과 그것을 발전시킨 형태인 확률적 경사 하강법에 대해 설명한다.

대수 우도 함수의 미분이 0이 되는 방정식에 해석해(analytic solution)가 없는 경우에는 수치적으로 최적화해야 한다. 또한, 이 분야에서는 일반적으로 목적 함수를 최소화하는 것을 목표로 한다. 이처럼 최소화하는 경우의 목적 함수를 **손실 함수(Loss Function)**라고 한다. 따라서 대수 우도 함수를 최대화하는 것이 아니라 부호를 반대로 한 것을 최소화하는 것이 목표가 된다.

$$\begin{aligned}\theta_{MLE} &= \operatorname{argmin}_{\theta} E(\theta) \\ E(\theta) &= -\ln L(\theta)\end{aligned}$$

이런 미분 가능한 함수의 수치 최적화를 구하는 가장 간단한 방법이 경사 하강법(Gradient Descent)으로, 다음과 같이 경사(미분 계수)를 이용해서 반복적으로 최적화해 나가는 방법이다.

$$\theta^{t+1} = \theta^t - \gamma \frac{\partial}{\partial \theta} E(\theta^t)$$

여기서 γ 는 학습률 파라미터로 양의 값이다. 학습률이 크면 손실 함수의 감소 속도가 빠르지만, 잘 수렴하지 않으면 진동이 발생할 가능성이 있다. 반면 학습률이 적으면 손실 함수의 감소가 느리며, 수렴할 때까지 많은 계산을 해야 한다. 특히, 대수 우도 함수처럼 목적 함수가 동일 형태의 함수의 합으로 분해할 수 있을 때는 모든 값을 사용

하지 않고, 랜덤으로 일부 값(미니 배치)만 사용하는 확률적 경사 하강법(SGD, Stochastic Gradient Descent)을 이용할 수 있다. 따라서 빅데이터 등 데이터 규모가 큰 경우에 매우 효과적이다.

$$E(\theta) = \sum_n E_n(\theta)$$

$$\theta^{t+1} = \theta^t - \gamma \sum_{n \in \text{batch}} E_n(\theta)$$

이 경사 하강법과 자동 미분을 조합하면 '복잡한 우도 함수라도 시스템적으로 최적화할 수 있게' 된다.

선형 회귀 모델

대표적인 확률 모델인 선형 회귀 모델에 대해 설명한다. 신경망도 사실은 선형 회귀 모델을 확장한 것이므로 이 모델을 잘 이해하면 신경망도 쉽게 이해할 수 있다.

2.3.1 선형 회귀 모델의 최대 우도 추정

선형 회귀(Linear Regression) 모델은 여러 변수로부터 하나 또는 여러 개의 값을 예측하는 기법이다. 다음과 같은 식으로 표현할 수 있다.

$$y = a \cdot x + b + \epsilon = \sum_i a_i x_i + b + \epsilon$$

여기서 x 는 독립 변수 또는 특이량이라고 하며, y 는 예측하고 싶은 목적 변수, a , b 는 모델의 파라미터(회귀 계수), ϵ 는 정규 분포 $N(0, \sigma^2)$ 의 오차항이다. x 로부터 y 를 예측하는 것이 목적이다. 특히, y 에 연속 변수를 사용하므로 '회귀 문제'라고 부른다. 변수 x 에 1을 포함해서 $(1, x_1, x_2, \dots)$ 라고 하면 절편 b 도 회귀 계수 a 에 포함할 수 있어서 다음과 같이 식이 간단해진다.

$$y = a \cdot x$$

선형 회귀 모델이라는 이름은 목적 변수 y 가 파라미터 a 의 1차식으로 표현된다는 데서 온 것이다. 한편 선형 회귀 모델은 확률 모델이기도 하므로 다음과 같이 표현할 수 있다.

$$y \sim \mathcal{N}(x|a \cdot x, \sigma^2)$$

확률 모델의 파라미터를 구하려면 2.1절에서 다룬 최대 우도 추정이 필요하다. N 개의 데이터가 주어졌을 때 선형 회귀 모델의 대수 우도 함수는 다음과 같이 된다.

$$\ln L(\mathbf{a}) = -\frac{N}{2} \ln 2\pi\sigma^2 - \frac{1}{2\sigma^2} \sum_n (y_n - m_n)^2$$

$$m_n = \sum_i a_i x_{ni}$$

$\mathbf{a}$ 에서 미분을 만들 때에 의미가 있는 항만 남기면 실제로는 다음과 같이 평균 제곱 오차(MSE, Mean Squared Error)를 최소화하는 문제가 된다.

$$E(\mathbf{a}) = \sum_n E_n(\mathbf{a}) = \frac{1}{N} \sum_n (y_n - \mathbf{a}_n \cdot \mathbf{x}_n)^2$$

이 $E(\mathbf{a})$ 에 SGD 최소화를 적용하면 모델의 $\mathbf{a}$ 를 최대 우도 추정을 할 수 있다. 이것으로 이론적 배경은 쌓였으므로 이제 파이토치를 실제로 사용해서 선형 회귀 모델의 파라미터를 추정해 보도록 하자. 또한, 다음 장 이후에 다룰 신경망을 위해서 선형 회귀 모델을 신경망으로 표현해 보았다(그림 2.1).

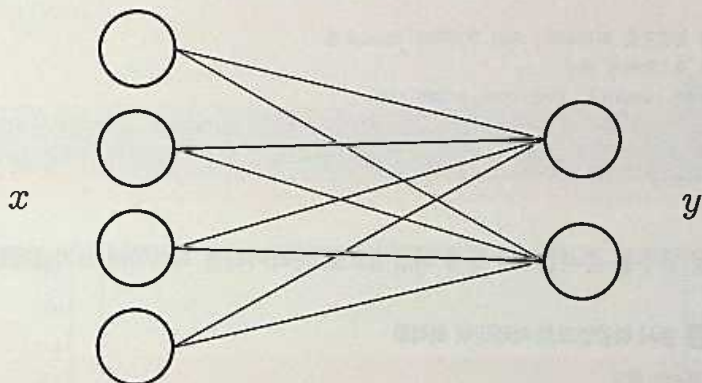


그림 2.1 선형 회귀 모델의 네트워크. 특이량 x 의 여러 차원을 조합해서 y 의 예측값을 만들고 있다. 이 예에서 4차원의 x 에서 2차원의 y 로 변환하고 있다.

2.3.2 파이토치로 선형 회귀 모델 만들기(직접 만들기)

그러면 바로 파이토치를 사용해서 선형 회귀 모델의 파라미터를 구해 보자. 다음과 같이 변수가 두 개인 모델을 생각할 수 있다.

$$y = 1 + 2x_1 + 3x_2$$

리스트 2.1의 코드에서는 테스트 데이터를 생성해서 파라미터 학습을 위한 변수를 준비하고 있다.

리스트 2.1 테스트 데이터 생성 및 파라미터 학습을 위한 변수 정의



```
import torch

# 참(true)의 계수
w_true = torch.Tensor([1, 2, 3])

# X 데이터 준비. 절편을 회귀 계수에 포함시키기 위해
# X의 최초 차원에 1을 추가해 둔다
X = torch.cat([torch.ones(100, 1), torch.randn(100, 2)], 1)

# 참의 계수와 각 X의 내적을 행렬과 벡터의 곱으로 모아서 계산
y = torch.mv(X, w_true) + torch.randn(100) * 0.5

# 기울기 하강으로 최적화하기 위해 파라미터 Tensor를
# 난수로 초기화해서 생성
w = torch.randn(3, requires_grad=True)

# 학습률
gamma = 0.1
```

데이터 및 변수를 준비했으니 경사 하강법으로 파라미터를 최적화해 보자(**리스트 2.2**).

리스트 2.2 경사 하강법으로 파라미터 최적화



```
# 손실 함수의 로그
losses = []

# 100회 반복
for epoc in range(100):
    # 전회의 backward 메서드로 계산된 경사값을 초기화
    w.grad = None
```



```

# 선형 모델로 y 예측값을 계산
y_pred = torch.mv(X, w)

# MSE loss와 w에 의한 미분을 계산
loss = torch.mean((y - y_pred)**2)
loss.backward()

# 경사를 갱신한다
# w를 그대로 대입해서 갱신하면 다른 텐서가 돼서
# 계산 그래프가 망가진다. 따라서 data만 갱신한다
w.data = w.data - gamma * w.grad.data

# 수렴 확인을 위한 loss를 기록해 둔다
losses.append(loss.item())

```

최적화가 제대로 이루어지면 손실 함수가 수렴하는지 확인할 수 있다. 주피터 노트북(컬래버레이터도 동일)을 사용하고 있는 독자라면 **리스트 2.3**처럼 matplotlib을 사용해서 그래프로 확인할 수 있다.

리스트 2.3 matplotlib으로 그래프 그리기

In

```

%matplotlib inline
from matplotlib import pyplot as plt
plt.plot(losses)

```

Out

[<matplotlib.lines.Line2D at 0x7f2bf5ff7b70>]

리스트 2.3를 참고(X, y가 난수를 포함하고 있으므로 세로축의 값은 매번 달라진다)

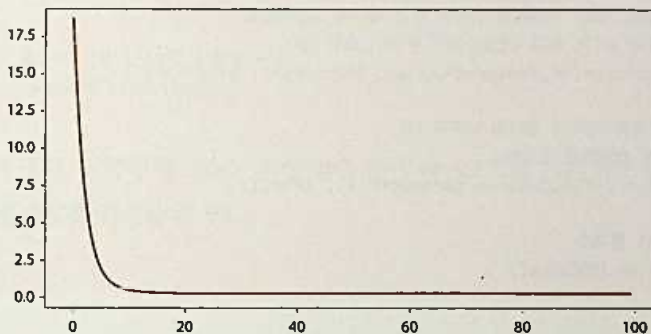


그림 2.2 손실 함수의 수렴 확인

이처럼 반복을 진행함에 따라 손실 함수의 값이 작아져서 일정 값으로 수렴하면 최적화가 제대로 된 것이다. 회귀 계수도 맞는지 확인해 보자(리스트 2.4). 참고로, 프로그램 상에 난수가 포함돼 있어서 값이 다를 수도 있다.

리스트 2.4 회귀 계수의 확인

 In

W

 Out

```
tensor([0.9524, 1.9783, 2.9360], requires_grad=True)
```

a = (1, 2, 3)으로 제대로 학습돼 있는 것을 알 수 있다.

2.3.3 파이토치로 선형 회귀 모델 만들기(nn, optim 모듈 사용)

2.2절에서는 자동 미분을 사용할 때 이외에는 모델 구축이나 경사 하강법 계산을 모두 직접 했었다. 이들은 신경망을 풀 때 공통적으로 사용되는 계산으로, 파이토치에는 이 계산들을 쉽게 작성할 수 있게 해주는 모듈이 포함돼 있다.

모델의 구축은 torch.nn, 최적화는 torch.optim을 사용하면 된다. 이 둘을 활용하면 선형 회귀 모델을 더 간단하게 작성할 수 있다. 리스트 2.5를 보도록 하자.

리스트 2.5 선형 회귀 모델의 구축과 최적화 준비

 In

```
from torch import nn, optim

# Linear층을 작성. 이번에는 절편은 회귀 계수에 포함하므로
# 입력 차원을 3으로 하고 bias(절편)을 False로 한다
net = nn.Linear(in_features=3, out_features=1, bias=False)

# SGD의 최적화기상에서 정의한 네트워크의
# 파라미터를 전달해서 초기화
optimizer = optim.SGD(net.parameters(), lr=0.1)

# MSE loss 클래스
loss_fn = nn.MSELoss()
```

`nn.Linear`는 그 이름에서도 알 수 있듯이, 선형 결합을 계산하는 클래스로 회귀 계수나 절편 등의 파라미터를 포함하고 있다. 또한, `nn.Linear`는 3장에서 자세하게 다룰 `nn.Module`의 서브 클래스로 SGD 등의 최적화기(optimizer)와 연계하거나 학습 결과 파라미터를 저장하는 등 다양한 처리를 할 수 있다.

`nn.MSELoss`는 명칭 그대로 MSE를 계산하기 위한 클래스다. MSE 정도의 간단한 손실 함수라면 직접 작성해도 괜찮지만, 파이토치가 기본으로 제공하니 사용하도록 하자.

이상으로 모든 준비를 마쳤다. 다음은 최적화 루프(반복)를 돌려 보자(**리스트 2.6**).

리스트 2.6 최적화 루프(반복 루프) 돌리기



```
# 손실 함수 로그
losses = []

# 100회 반복
for epoc in range(100):
    # 전회의 backward 메서드로 계산된 경사값을 초기화
    optimizer.zero_grad()

    # 선형 모델으로 y 예측값을 계산
    y_pred = net(X)

    # MSE loss 계산
    # y_pred는 (n,1)과 같은 shape를 지니고 있으므로 (n,)으로 변경할 필요가 있다
    loss = loss_fn(y_pred.view_as(y), y)

    # loss의 w를 사용한 미분 계산
    loss.backward()

    # 경사를 갱신한다
    optimizer.step()

    # 수렴 확인을 위한 loss를 기록해 둔다
    losses.append(loss.item())
```

이처럼 모델 정의, 최적화 알고리즘, 손실 함수 등에 파이토치의 기본 함수를 사용하면 더 간단하게 코드를 작성할 수 있다.

그러면 수렴한 모델의 파라미터를 확인해 보자(리스트 2.7). 참고로 프로그램상에 난수를 사용하고 있으므로 출력값이 다를 수 있다.

리스트 2.7 수렴한 모델의 파라미터 확인

↗ In

```
list(net.parameters())
```

↖ Out

```
[Parameter containing:  
tensor([[0.9524, 1.9783, 2.9360]], requires_grad=True)]
```

리스트 2.4와 똑같은 결과다. 성공이다!

로지스틱 회귀

유명한 선형 모델 중 하나인 로지스틱 회귀
(Logistic Regression, 분류 문제를 풀 때 사용함)에 대해 설명한다.

2.4.1 로지스틱 회귀의 최대 우도 추정

로지스틱 회귀는 회귀라는 이름이 붙어 있긴 하지만, 실제로는 분류를 위한 선형 모델이다. 선형 회귀는 목적 변수 y 가 연속값이지만, 로지스틱 회귀에서는 y 는 $[0, 1]$ 의 이산값으로 분류 대상이 두 개인 문제가 된다.

변수의 선형 결합은 $[-\infty, \infty]$ 사이의 값을 임의로 취하므로 로지스틱 회귀에서는 선형 결합을 취한 후에 추가로 시그모이드(sigmoid) 함수 $\sigma(x)$ 를 적용해서 $[0, 1]$ 사이의 값으로 변환한다.

$$h = \mathbf{a} \cdot \mathbf{x}, \quad z = \sigma(h) = \frac{1}{1 + e^{-h}}$$

시그모이드 함수는 **그림 2.3**처럼 S자 곡선을 그린다.

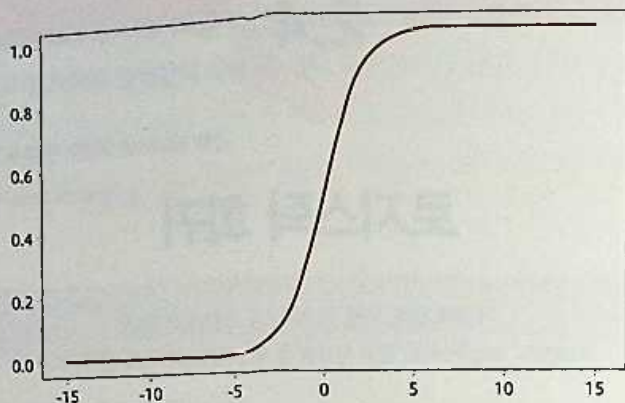


그림 2.3 시그모이드 함수

선형 회귀 모델과 마찬가지로 로지스틱 회귀도 확률 모델이다. 이 모델에서는 y 가 파라미터 z 의 베르누이 분포를 따른다고 가정한다($y \sim B(z)$). 따라서 최대 우도 추정에 사용하기 위한 손실 함수는 2.1절에서 등장한 베르누이 분포의 우도 식을 적용하면 다음과 같은 크로스 엔트로피(cross entropy)라는 양이 된다.

$$E(a) = - \sum_n y_n \ln z_n + (1 - y_n) \ln(1 - z_n)$$

이 손실 함수를 a 에 대해 미분하려면 식을 변형해야 해서 쉽지 않다. 하지만 우리에게 는 파이토치의 자동 미분이 있다. 다음 절에서 보도록 하겠다.

2.4.2 파이토치를 사용한 로지스틱 회귀 분석

여기서는 분류 처리 예로 자주 사용되는 iris(꽃이름) 데이터를 이용해서 로지스틱 회귀를 실행해 보도록 하겠다. iris 데이터는 **리스트 2.8**처럼 scikit-learn(**메모** 참고)에 포함되어 있는 것을 사용한다.



MEMO

scikit-learn

대부분이 파이썬으로 기술된 오픈 소스 머신 러닝 라이브러리다. NumPy나 SciPy와도 연동된다.

리스트 2.8 iris 데이터 준비



```
import torch
from torch import nn, optim
from sklearn.datasets import load_iris
iris = load_iris()

# iris는 (0,1,2)의 세 가지 종류를 분류하는 문제이므로
# (0,1)의 두 개 데이터만 사용한다
# 원래는 학습용과 테스트용으로 나누어야 하지만 여기선 생략한다
X = iris.data[:100]
y = iris.target[:100]

# NumPy의 ndarray를 PyTorch의 Tensor로 변환
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)
```

다음은 모델을 만들도록 한다(리스트 2.9).

리스트 2.9 모델 작성



```
# iris 데이터는 4차원
net = nn.Linear(4, 1)

# 시그모이드 함수를 적용해서 두 클래스의 분류를 위한
# 크로스 엔트로피를 계산
loss_fn = nn.BCEWithLogitsLoss()

# SGD(약간 큰 학습률)
optimizer = optim.SGD(net.parameters(), lr=0.25)
```

선형 회귀와 다른 것은 손실 함수밖에 없다. 나머지 처리는 선형 회귀와 똑같이 파라미터 최적화를 위한 반복 루프를 돌리면 된다(리스트 2.10).

리스트 2.10 파라미터 최적화를 위한 반복 루프

```
# 손실 함수 로그
losses = []

# 100회 반복
for epoc in range(100):
    # 전회의 backward 메서드로 계산된 경사값을 초기화
    optimizer.zero_grad()

    # 선형 모델로 y 예측값을 계산
    y_pred = net(X)

    # MSE loss를 사용한 미분 계산
    loss = loss_fn(y_pred.view_as(y), y)
    loss.backward()

    # 경사를 갱신한다
    optimizer.step()

    # 수렴 확인을 위한 loss를 기록해 둔다
    losses.append(loss.item())
```

📄 In

```
%matplotlib inline
from matplotlib import pyplot as plt
plt.plot(losses)
```

📄 Out

```
[<matplotlib.lines.Line2D at 0x7f72290dee10>]
# 그림 23을 참고(X, y가 난수를 포함하고 있어서 세로축의 값이 다를 수 있다)
```

손실 함수는 **그림 24**에서 볼 수 있듯이 수렴한다.

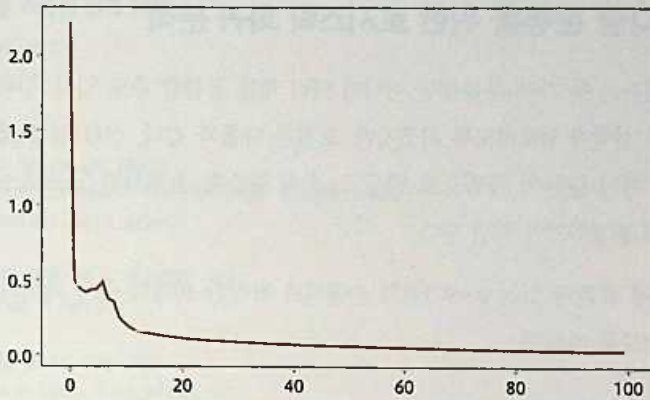


그림 2.4 로지스틱 회귀가 수렴하는 모습

예측은 **리스트 2.11** 처럼 기술하면 된다.

리스트 2.11 모델 작성

```
# 선형 결합의 결과
h = net(X)

# 시그모이드 함수를 적용한 결과는 y=1의 확률을 보여 준다
prob = nn.functional.sigmoid(h)

# 확률이 0.5이상인 것을 클래스1로 예측하고 그외는 0으로 한다
# PyTorch에는 Bool형이 없으므로 ByteTensor가 출력된다.
y_pred = prob > 0.5

# 예측 결과 확인 (y는 FloatTensor이므로 ByteTensor로 변환한 후에 비교)
(y.byte() == y_pred.view_as(y)).sum().item()
```

* 출력 시에 표시되는 경고 메시지는 생략



100

100개의 샘플 데이터 분류가 정상적으로 이루어졌다.

2.4.3 다중 분류를 위한 로지스틱 회귀 분석

로지스틱 회귀는 두 가지 분류뿐만 아니라 여러 개를 분류할 수도 있다. 수학적인 내용은 머신러닝 서적에 양보하도록 하겠지만, 요점은 다음과 같다. 선형 결합 계층의 출력을 1차원이 아닌 분류의 차원으로 만들고, 손실 함수를 소프트맥스 크로스 엔트로피라는 함수로 변경하기만 하면 된다.

scikit-learn에 포함돼 있는 0~9까지의 손글씨로 작성한 10개의 숫자 데이터를 사용해 테스트해 보도록 하겠다.

리스트 2.12 손으로 쓴 10개의 숫자 데이터를 분류하는 문제

📄 In

```
import torch
from torch import nn, optim
from sklearn.datasets import load_digits
digits = load_digits()

X = digits.data
y = digits.target

X = torch.tensor(X, dtype=torch.float32)
# CrossEntropyLoss 함수는 y로 int64형의 텐서를 받으니 주의하자
y = torch.tensor(y, dtype=torch.int64)

# 출력은 10(분류 수) 차원
net = nn.Linear(X.size()[1], 10)

# 소프트맥스 크로스 엔트로피
loss_fn = nn.CrossEntropyLoss()

# SGD
optimizer = optim.SGD(net.parameters(), lr=0.01)
```

이상으로 데이터와 모델 준비가 끝났다. 학습을 위한 반복 처리 부분은 기본적으로 두 개 클래스 경우와 거의 같지만, loss 함수에 인수를 전달하는 방법이 다르니 주의하자 (**리스트 2.13**).

리스트 2.13 학습용 반복 처리

```
# 손실 함수 로그
losses = []

# 100회 반복
for epoc in range(100):
    # 전회의 backward 메서드로 계산된 경사값을 초기화
    optimizer.zero_grad()

    # 선형 모델로 y 예측값을 계산
    y_pred = net(X)

    # MSE loss 미분 계산
    loss = loss_fn(y_pred, y)
    loss.backward()

    # 경사값 갱신한다
    optimizer.step()

    # 수렴 확인을 위한 losses를 기록해 둔다
    losses.append(loss.item())
```

선형 결합의 출력을 소프트맥스 함수에 전달하면 각 분류의 확률을 얻을 수 있으며, 이 중 가장 확률이 높은 분류를 예측값으로 한다. 소프트맥스 함수는 단순 증가 함수이므로 선형 결합 출력이 가장 큰 부분을 찾으면 된다(리스트 2.14).

리스트 2.14 정답률

```
# torch.max는 집계축을 지정하면 최대값뿐만 아니라 그 위치도 반환한다
_, y_pred = torch.max(net(X), 1)
# 정답률을 계산한다
(y_pred == y).sum().item() / len(y)
```

In

0.9415692821368948

Out

정답률 94% 이상을 달성했다.

2.5

정리

이 장에서 다룬 내용을 정리한다.

선형 모델의 대표적인 예인 선형 회귀와 로지스틱 회귀를 확률 모델의 관점에서 설명하고, 경사 하강법을 사용한 파라미터 최대 우도 추정에 대해 살펴보았다.

선형 회귀, 로지스틱 회귀 모두 선형 계층이 하나이고, 손실 함수로 평균제곱오차(MSE) 또는 크로스 엔트로피를 이용한다는 점에서 다음 장부터 본격적으로 다룰 신경망에도 적용할 수 있다.

또한, 경사 하강법에서 필요한 미분 계산은 로지스틱 회귀에서는 매우 번거롭지만, 파이토치의 자동 미분 기능을 사용하면 쉽게 최적화할 수 있다.



CHAPTER

3

다층 퍼셉트론

이 장에서는 드디어 신경망을 만든다. 선형 계층, 또는 전결합 계층으로 구성되는 신경망은 일반적으로 다층 퍼셉트론(MLP, Multi-layer Perceptron, **메모** 참고)이라고 불리며, 다양한 데이터에 적용할 수 있는 범용 신경망이다. 먼저, 2장의 마지막 부분에서 다룬 것처럼 손글씨 문자 데이터를 사용해서 간단한 MLP를 구축한 후, Dropout이나 BatchNorm 등의 정규화 및 학습 효율화 기법을 다룬다. 그리고 직접 계층을 만들어서 모델화하는 방법에 대해 설명한다.



MEMO

다층 퍼셉트론

엄밀하게는 퍼셉트론 알고리즘을 사용하고 있지 않지만, 무슨 이유에서인지 퍼셉트론이라 불리고 있다.

3.1

MLP 구축과 학습

MLP 구축과 학습 방법에 대해 다룬다.

먼저, 간단한 MLP를 파이토치를 사용해서 만들고, MLP를 훈련하는 방법을 설명하겠다. 모델 구축 방법은 약간 다르지만, 훈련은 2장에서 다룬 선형 모델과 똑같다.

MLP는 **그림 3.1** 처럼 선형 계층을 연결한 것이다. 첫 번째 층을 입력층, 마지막 층을 출력층, 그 이외의 층을 중간층 또는 숨김층이라고 부른다. 출력층은 회귀 문제의 경우는 선형 회귀, 분류 문제의 경우는 로지스틱 회귀와 구조가 같다.

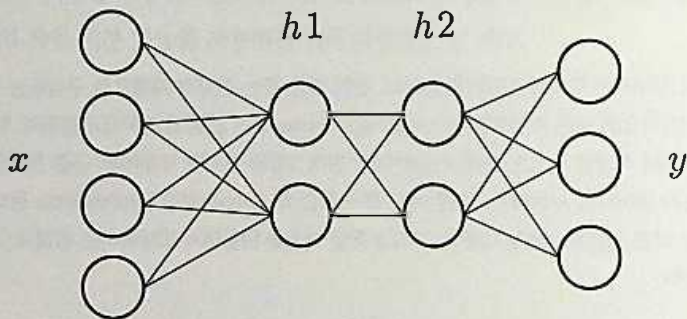


그림 3.1 중간층이 두 개인 MLP 구조

단순히 선형 계층을 연결하기만 하면 결과적으로 전체가 선형 함수가 되어 버리므로 각 층의 출력에는 **활성화 함수(Activation Function)**라는 비선형 함수를 적용해서 전체적

으로도 비선형 함수가 되도록 한다. 식을 보는 것보다 코드를 보면 바로 이해할 수 있다. **리스트 3.1**의 예에서는 2장에서도 다룬 scikit-learn의 손글씨 문자를 판별하는 MLP를 만든다.

리스트 3.1 손글씨 문자를 판별하는 MLP 작성



```
import torch
from torch import nn

net = nn.Sequential(
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 16),
    nn.ReLU(),
    nn.Linear(16, 10)
)
```

`nn.Sequential`은 `nn.Module` 층을 차례로 쌓아서 신경망을 구축할 때 사용한다. 이와 같이 층이 일직선으로 쌓인 형태의 신경망을 **피드포워드(Feedforward)형**이라고 한다.

`nn.ReLU`는 최근 신경망 학습용으로 자주 사용되는 ReLU(**메모** 참고)라는 활성화 함수다. 여기서 만든 `net`은 64차원 데이터를 받아 내부에서 여러 변환을 거친 후 10차원의 값을 반환하는 함수다. 이것은 **2장**에서 다룬 선형 회귀나 로지스틱 회귀와 처리 방식만 다르지 입출력 형태는 똑같다. 또한, 이 `net`은 미분이 가능해서 파이토치의 자동 미분과 SGD를 사용하면 학습도 똑같이 할 수 있다(**메모** 참고).



MEMO

ReLU

딥러닝 이전에는 신경망 활성화 함수에 시그모이드 함수나 $\tanh$ 등이 사용되고 있었지만, S자형의 함수에서는 원점으로부터 떨어지면 0에 가까워지는 성질이 있다. 즉, 신경망의 층이 늘어날수록 학습이 진행되지 않는 경사 소실이라는 문제가 발생한다. ReLU는 $f(x) = \max(0, x)$ 라는 형태를 하고 있어서 $x > 0$ 인 영역에서는 항상 미분이 유한한 값으로 이 소실 문제를 피할 수 있다. 딥러닝 시대로 들어서면서부터는 이 ReLU가 신경망 학습에 폭넓게 사용되고 있다.



MEMO

자동 미분

MLP의 미분은 활성화 함수를 포함하고 있거나 다층인 경우가 있어서 선형 모델과 비교하면 훨씬 복잡하다. 하지만 여기서도 자동 미분이 큰 도움을 준다.

특히 피드포워드형 신경망의 미분을 구할 때에는 **역전파(back propagation)**라고 하는 **동적 계획법** 알고리즘을 사용한다. 파이토치에서 미분을 구하는 메소드명이 backward인 것도 이런 이유 때문이다.

리스트 3.2에서는 손글씨 문자 데이터 학습 코드의 나머지 부분을 보여 주고 있다. 또한 **리스트 3.2**에서는 단순한 SGD가 아닌 수렴이 더욱 빠른 Adam(Adaptive Moment Estimation)이라는 개선된 SGD 알고리즘을 사용하고 있다.

리스트 3.2 손글씨 문자 데이터의 학습 코드의 나머지 부분



```
import torch
from torch import nn, optim
from sklearn.datasets import load_digits
digits = load_digits()

X = digits.data
Y = digits.target

# NumPy의 ndarray를 파이토치의 텐서로 변환
X = torch.tensor(X, dtype=torch.float32)
Y = torch.tensor(Y, dtype=torch.int64)

# 소프트맥스 크로스 엔트로피
loss_fn = nn.CrossEntropyLoss()

# Adam
optimizer = optim.Adam(net.parameters())

# 손실 함수의 로그
losses = []

# 100회 반복
for epoc in range(100):
```

```

# backward 메서드로 계산된
# 이전 값을 삭제
optimizer.zero_grad()

# 선형 모델로 y의 예측 값 계산
y_pred = net(X)

# MSE loss와 w를 사용한 미분 계산
loss = loss_fn(y_pred, Y)
loss.backward()

# 경사를 갱신
optimizer.step()

# 수렴 확인을 위해 loss를 기록해 둔다
losses.append(loss.item())

```

GPU로 학습하는 경우에는 변수와 함께 net도 텐서의 to 메서드를 통해 GPU로 전송한다(리스트 3.2)

리스트 3.2 to 메서드를 이용해 GPU로 전송



```

X = X.to("cuda:0")
Y = Y.to("cuda:0")
net.to("cuda:0")
# 이후 처리는 동일하게 optimizer를 설정해서 학습 루프를 돌린다

```



```

# 리스트 3.2의 결과와 같다

```

3.2

Datset과 DataLoader

지금까지는 모든 데이터를 모아서 학습용으로 사용했지만, 데이터가 늘어나거나 신경망 계층의 증가, 또는 파라미터가 늘어나면 전체 데이터를 메모리에서 처리하기 어려워진다. 여기서는 이 문제를 해결하기 위해 데이터의 일부 미니 배치(mini-batch)만 사용하는 SGD 학습법을 보도록 한다.

3.2.1 Dataset과 DataLoader

파이토치에는 Dataset과 DataLoader라는 기능이 있어서 미니 배치 학습이나 데이터 셔플(shuffle), 심지어 병렬 처리까지 간단히 수행할 수 있다.

TensorDataset은 Dataset을 상속한 클래스로 학습 데이터 X와 레이블 Y를 묶어 놓은 컨테이너다. 이 TensorDataset을 DataLoader에 전달하면 for 루프에서 데이터의 일부만 간단히 추출할 수 있게 된다(리스트 3.4). TensorDataset에는 텐서만 전달할 수 있으며, Variable은 전달할 수 없으니 주의하자.

리스트 3.4 TensorDataset을 DataLoader에 전달해서 데이터의 일부만 간단히 추출하는 예

```
import torch
from torch import nn, optim
from torch.utils.data import TensorDataset, DataLoader

# Dataset 작성
ds = TensorDataset(X, Y)
```



```

# 순서로 섞어서 64개씩 데이터를 반환하는 DataLoader 작성
loader = DataLoader(ds, batch_size=64, shuffle=True)

net = nn.Sequential(
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 16),
    nn.ReLU(),
    nn.Linear(16, 10)
)

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters())

# 최적화 실행
losses = []
for epoch in range(10):
    running_loss = 0.0
    for xx, yy in loader:
        # xx, yy는 64개만 받는다
        y_pred = net(xx)
        loss = loss_fn(y_pred, yy)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    losses.append(running_loss)

```

Dataset은 직접 작성할 수도 있어서 대량의 이미지 파일을 한 번에 메모리에 저장하지 않고, 필요할 때마다 읽어서 학습하는 등 다양하게 활용할 수 있다.¹

1 **참고** 오류가 발생한다면 **제2장 3.3**을 다시 실행한 후 이 코드를 실행하자. **제2장 3.3**에서 GPU로 설정돼 있으므로 3.3을 건너뛰어야 한다.

3.3

학습 효율화 팁

신경망은 표현력이 매우 높은 모델이지만, 한편으로는 '훈련된 데이터와만' 궁합이 좋아서 다른 데이터에 적용할 수 없거나 훈련이 불안정해서 시간이 오래 걸리는 문제가 있다. 여기서는 이런 문제를 해결하기 위한 두 가지 대표적인 기법인 Dropout과 Batch Normalization에 대해 다룬다.

3.3.1 Dropout을 사용한 정규화

신경망뿐만 아니라 머신러닝의 공통적인 문제로 **과학습**을 들 수 있다. 과학습이란, 훈련용 데이터에 파라미터가 과하게 최적화되어 다른 데이터에 대한 판별 능력이 떨어지게 되는 현상이다. 예를 들어, 사람이 시험 공부를 할 때 배경 이론은 이해하지 않고 암기만 해서 응용력이 떨어지는 것과 같다. 특히, 층이 깊은 신경망은 파라미터가 많아서 충분한 데이터가 없으면 과학습이 발생할 수 있다.

예를 들어, 3.1절에서 사용한 신경망을 **리스트 3.5**, **리스트 3.6**의 코드처럼 층을 증가시키면 **그림 3.2**의 왼쪽에 있는 것처럼 된다. 이렇게 필요 이상으로 SGD의 반복을 늘리면 검증용 데이터의 손실 함수가 반대로 올라가 버리는 것을 관측할 수 있다.

리스트 3.5 3.1절에서 사용한 신경망의 층을 늘린 코드①

```
import torch
from torch import nn, optim
# 데이터를 훈련용과 검증용으로 분할
from sklearn.model_selection import train_test_split
# 전체의 30%는 검증용
X = digits.data
```

↗ In


```

Y = digits.target
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3)

X_train = torch.tensor(X_train, dtype=torch.float32)
Y_train = torch.tensor(Y_train, dtype=torch.int64)
X_test = torch.tensor(X_test, dtype=torch.float32)
Y_test = torch.tensor(Y_test, dtype=torch.int64)

# 여러 층을 쌓아서 깊은 신경망을 구축한다
k = 100
net = nn.Sequential(
    nn.Linear(64, k),
    nn.ReLU(),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.Linear(k, 10)
)

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters())
# 훈련용 데이터로 DataLoader를 작성
ds = TensorDataset(X_train, Y_train)
loader = DataLoader(ds, batch_size=32, shuffle=True)

```

리스트 3.6 3.1절에서 사용한 신경망의 층을 늘린 코드②

↓ In

```

train_losses = []
test_losses = []
for epoch in range(100):
    running_loss = 0.0
    for i, (xx, yy) in enumerate(loader):
        y_pred = net(xx)
        loss = loss_fn(y_pred, yy)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    train_losses.append(running_loss / i)
    y_pred = net(X_test)
    test_loss = loss_fn(y_pred, Y_test)
    test_losses.append(test_loss.item())

```

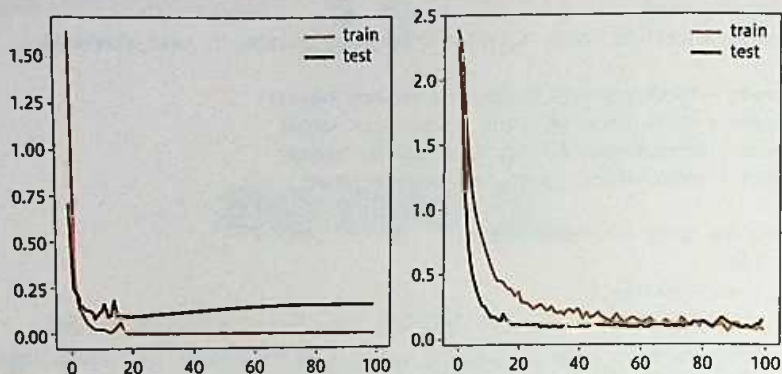


그림 3.2 (왼쪽) 과학을 일으켜서 test(검증용 데이터)의 손실 함수가 상승하고 있다
(오른쪽) Dropout을 추가해서 과학을 방지하고 있다

과학을 방지하는 것을 **정규화(regularization)**라고 한다. 정규화에는 다양한 방법이 있지만, 신경망에서는 Dropout이라는 것이 자주 사용된다. 몇 개의 노드(변수의 차원)를 랜덤으로 선택해서 의도적으로 사용하지 않는 방법이다.

Dropout은 신경망 훈련 시에만 사용하고, 예측 시에는 사용하지 않는 것이 일반적이다. 파이토치에서는 모델의 train과 eval 메서드로 Dropout을 적용 또는 미적용할 수 있다(**리스트 3.7**, **리스트 3.8**).

리스트 3.7 train과 eval 메서드로 Dropout 처리 적용/미적용①

인

```
# 확률 0.5로 랜덤으로 변수의 차원을
# 버리는 Dropout을 각 층에 추가
net = nn.Sequential(
    nn.Linear(64, k),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.Dropout(0.5),
```

```
nn.Linear(k, 10)
)
```

리스트 3.8 train과 eval 메서드로 Dropout 처리 적용/미적용②



```
optimizer = optim.Adam(net.parameters())

train_losses = []
test_losses = []
for epoch in range(100):
    running_loss = 0.0
    # 신경망을 훈련 모드로 설정
    net.train()
    for i, (xx, yy) in enumerate(loader):
        y_pred = net(xx)
        loss = loss_fn(y_pred, yy)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    train_losses.append(running_loss / i)
    # 신경망을 평가 모드로 설정하고
    # 검증 데이터의 손실 함수를 계산
    net.eval()
    y_pred = net(X_test)
    test_loss = loss_fn(y_pred, Y_test)
    test_losses.append(test_loss.item())
```

3.3.2 Batch Normalization을 사용한 학습 가속

SGD를 사용한 신경망 학습에서는 각 변수의 차원이 동일한 범위의 값을 가지는 것이 중요하다. 한 층으로 된 선형 모델 등에서는 사전에 데이터를 정규화해 두면 되지만, 깊은 신경망에서는 층이 늘어날수록 데이터 분포가 바뀐다. 따라서 입력 데이터의 정규화만으로는 부족하다. 또한, 애당초 앞에 있는 층의 학습에 의해 파라미터가 변화하므로 뒤에 있는 층의 학습이 불안정해지는 문제가 있다.

이런 문제를 해결하고 학습을 안정화 및 가속화하는 방법으로 **Batch Normalization**이 있다. Batch Normalization도 훈련 시에만 적용하며(리스트 3.9), 평가 시에는 사용하지 않는다. 따라서 Dropout과 동일하게 train과 eval 메서드로 모드를 적용하거나 미적용할 수 있다.

리스트 3.9 train과 eval 메서드로 Batch Normalization 모드를 적용/미적용할 수 있다



```
# Linear층에는 BatchNorm1d를 적용한다
net = nn.Sequential(
    nn.Linear(64, k),
    nn.ReLU(),
    nn.BatchNorm1d(k),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.BatchNorm1d(k),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.BatchNorm1d(k),
    nn.Linear(k, k),
    nn.ReLU(),
    nn.BatchNorm1d(k),
    nn.Linear(k, 10)
)
```

3.4

신경망의 모듈화

파이토치에서는 직접 신경망 계층을 정의할 수 있다. 객체 지향 프로그래밍에서 자체 클래스를 만들 수 있는 것처럼 자체 신경망 계층을 만들어서 재사용하거나 이것을 부품으로 이용해서 더 복잡한 신경망을 만들 수도 있다.

3.4.1 자체 신경망 계층(커스텀 계층) 만들기

파이토치에서 자체 신경망 계층(커스텀 계층)을 만들려면 `nn.Module`을 상속해서 클래스를 정의하면 된다. 사실은 `nn.Module`은 `nn.Linear`를 포함한 모든 계층의 기반 클래스(base class)다.

커스텀 계층을 만들 때에 `forward` 메서드를 구현하면 자동 미분까지 가능해진다. 이미 특정 `Variable`형의 `x`를 `net(x)` 형식으로 사용해서 신경망에 의한 예측을 몇 번이고 실행했었다. 사실은 `nn.Module`의 `__call__` 메서드는 내부에서 `forward` 메서드를 사용하고 있으므로 `net(x)` 형식이 가능했던 것이다.

리스트 3.10의 예에서는 활성화 함수 ReLU와 Dropout을 내장하고 있는 커스텀 선형 계층을 만들고, 그것을 이용해서 3.3절의 MLP을 작성하고 있다. 보면 알겠지만 훨씬 코드가 간략해졌다.

리스트 3.10 활성화 함수 ReLU와 Dropout을 내장하는 커스텀 선형 계층을 만들고, 그것을 이용해서 MLP 작성

```
class CustomLinear(nn.Module):
    def __init__(self, in_features,
```

 In


```

        out_features,
        bias=True, p=0.5):
    super().__init__()
    self.linear = nn.Linear(in_features,
                             out_features,
                             bias)

    self.relu = nn.ReLU()
    self.drop = nn.Dropout(p)

    def forward(self, x):
        x = self.linear(x)
        x = self.relu(x)
        x = self.drop(x)
        return x

mlp = nn.Sequential(
    CustomLinear(64, 200),
    CustomLinear(200, 200),
    CustomLinear(200, 200),
    nn.Linear(200, 10)
)

```

또한, **리스트 3.11**처럼 `nn.Sequential`을 사용하지 않고 `nn.Module`을 상속해서 클래스 내에서 모든 처리를 할 수도 있다.

리스트 3.11 `nn.Module`을 상속한 클래스 이용



```

class MyMLP(nn.Module):
    def __init__(self, in_features,
                  out_features):
        super().__init__()
        self.ln1 = CustomLinear(in_features, 200)
        self.ln2 = CustomLinear(200, 200)
        self.ln3 = CustomLinear(200, 200)
        self.ln4 = CustomLinear(200, out_features)

    def forward(self, x):
        x = self.ln1(x)
        x = self.ln2(x)
        x = self.ln3(x)
        x = self.ln4(x)
        return x

mlp = MyMLP(64, 10)

```

정리

이 장에서 설명한 내용을 정리했다.

대표적인 신경망인 MLP도 선형 모델과 마찬가지로 경사 하강법을 사용해서 학습할 수 있다. 이때 어려운 미분 계산을 직접 할 필요 없이 파이토치의 자동 미분 기능을 사용하면 쉽게 해결할 수 있다. 따라서 학습 처리 부분은 선형 모델에서 사용한 코드와 동일한 코드를 사용할 수 있다.

Dataset과 DataLoader를 사용하면 미니 배치 학습을 쉽게 수행할 수 있으며, 대규모 데이터를 사용한 신경망 훈련도 가능하다. MLP의 과학습에 대처하기 위한 방법인 Dropout에 대해 설명했다. 또한, 학습을 안정화, 가속화시키는 방법으로 Batch Normalization을 소개했다. nn.Module을 상속하면 커스텀 신경망 계층을 만들 수 있으며, 동일한 신경망을 반복해서 작성하고 있다면 커스텀(자체) 계층을 작성하는 것도 방법이다.

! ATTENTION

4장부터의 예제에 대해

4장부터는 아래의 자주 사용하는 import는 이미 모두 실행했다고 가정하고 설명한다. 코드 실행 전에 한 번은 실행해 두자.²

```
import torch
from torch import nn, optim
from torch.utils.data import (Dataset,
                               DataLoader,
                               TensorDataset)

import tqdm
```

² **참고** 컬래버레이터를 사용하는 경우 일정 시간이 지나면 설정이 초기화되므로 오류가 나는 경우는 위 import문을 다시 실행해 주면 좋다



CHAPTER

4

이미지 처리와 합성곱 신경망

이 장에서는 이미지 분류 등 컴퓨터 비전 분야에서 폭넓게 사용되고 있는 합성곱 신경망(CNN, Convolutional Neural Network)을 다룬다.

2012년에 ILSVRC라는 이미지 인식 대회에서 다층 CNN을 사용한 모델이 다른 모델들을 압도하면서 딥러닝이 주목받게 된 기폭제가 됐다. 현재도 딥러닝이 가장 활발히 사용되는 분야가 컴퓨터 비전 분야로 특히, CNN이 활발히 연구되고 있으며 VGG, ResNet, Inception 등 다양한 종류가 개발되고 있다.

이 장에서는 단순한 CNN을 사용한 이미지 분류와 전이 학습, 이미지 고해상도화, 그리고 GAN을 사용한 이미지 생성 등을 소개한다.

4.1

이미지와 합성곱 계산

CNN의 가장 기초인 합성곱 계산에 대해 설명한다.

이미지 분야에서 합성곱(Convolution)이란, **그림 4.1**처럼 이미지 위에 작은 커널(또는 필터라고도 함) 행렬을 이동시켜 가면서 각 요소(픽셀)의 곱(또는 합이나 평균 등)을 계산해 가는 방식이다. 예를 들어, **그림 4.1**에서는 3×3 의 커널을 사용하고 있으며, 모든 값이 $1/9$ 이다. 즉, 9개 픽셀의 평균값을 계산하는 것으로 이미지를 균등화(Equalization)하게 된다.³

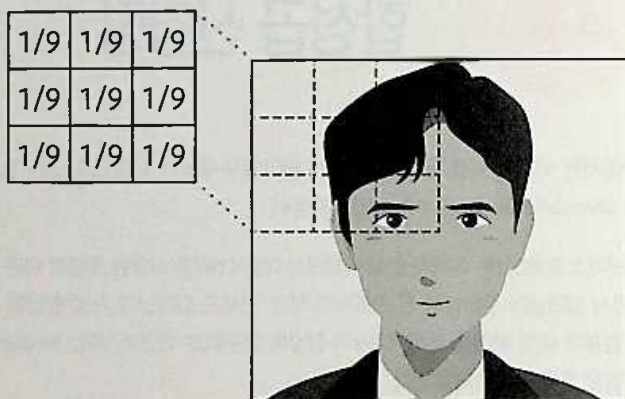


그림 4.1 이미지의 합성곱 계산

³ **참고** 커널은 다양한 크기의 창이라고 보면 된다. 어떤 창은 3×3 창틀로 이루어졌고, 어떤 창은 5×5 창틀로 이루어져 있는 것과 같다.

커널의 계수(커널 내의 숫자)를 변경하면 이미지를 날카롭게 만든다든가 경계선을 추출하는 등 다양한 처리가 가능하다. 커널과 이미지의 합성곱을 다른 관점에서 보면 커널을 사용해서 이미지로부터 특이량을 추출한다는 뜻도 된다. 미리 정의된 여러 종류의 커널을 사용해서 특이량을 추출하고, 이를 이미지 분류에 사용하는 방식도 생각할 수 있지만, 커널의 각 계수를 학습을 통해 자동으로 설정해 특이량을 추출할 수 있게 하는 것이 CNN의 기본적인 아이디어다(메모 참고). 사실 합성곱 계산은 선형 결합을 취하고 있는 것에 불과하다. 미분이 가능하므로 MLP과 마찬가지로 경사 하강법으로 학습할 수 있는 것이다.



MEMO

특이 학습

커널의 각 계수를 학습해서 자동 분류 등에 사용할 수 있는 중요한 특이량을 추출하는 것을 특이 학습(feature learning)이라고 한다.

CNN을 사용한 이미지 분류

CNN을 사용해서 실제로 이미지 분류를 해보도록 한다. 파이토치에는 합성곱 계층 클래스도 정의돼 있어서 3장의 MLP와 거의 동일하게 모델 구축 및 훈련을 할 수 있다.

CNN을 사용한 이미지 분류는 기본적으로 합성곱으로부터 ReLU 등의 활성화 함수를 적용하는 과정을 여러 번 실시하면 된다. 이미지 데이터는 (C, H, W) 형식으로 H, W는 이미지의 세로, 가로 크기이며, C는 색수 또는 채널(channel)이라고도 불린다. 원래 C=1 또는 C=3이지만, 최종적으로는 마지막 합성곱 계층의 채널 수가 된다. 이 처리를 통해 얻은 특이량을 MLP에 넣어서 최종적인 분류를 판별하게 된다. 합성곱 처리 후에는 위치 감도를 높이는 풀링(pooling)을 적용하거나 Dropout, Batch Normalization을 함께 사용하는 경우도 많다. 실제 파이토치를 사용한 구현 방법을 보도록 하자.

4.2.1 Fashion-MNIST

MNIST는 28×28 픽셀의 흑백 손글씨 숫자 데이터다. 이 데이터는 이미지 분류 시에 사용되는 대표적인 예제 데이터다. 최근에는 이 MNIST가 너무 간단하다는 지적도 있어서 손글씨 문자 대신에 10가지 분류의 옷 및 액세서리(신발, 구두 등)를 이미지 데이터로 이루어진 Fashion-MNIST(메모 참고)가 고안됐다. Fashion-MNIST도 28×28 픽셀 크기의 흑백 이미지로, 여기서도 이 데이터를 사용하도록 하겠다. 그림 4.2가 Fashion-MNIST 이미지 예다.

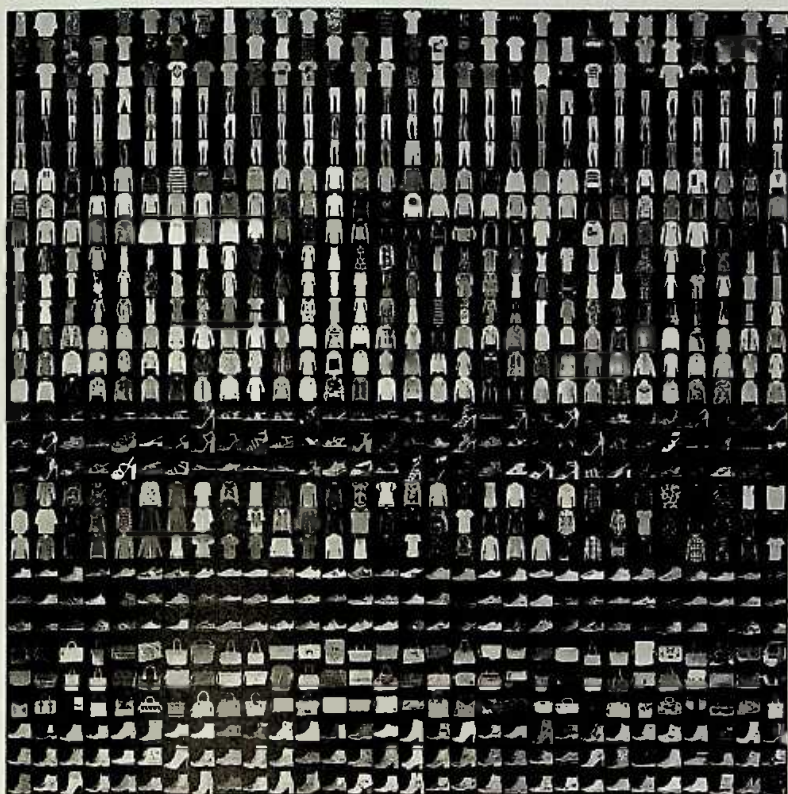


그림 4.2 Fashion-MNIST 이미지 예



MEMO

Fashion-MNIST

- zalando research/fashion-mnist

URL <https://github.com/zalando-research/fashion-mnist>

파이토치의 확장 기능인 torchvision이라는 라이브러리를 사용하면 [리스트 4.1](#) 과 같이 쉽게 Fashion-MNIST 데이터 다운로드부터, Dataset으로 변환, DataLoader 작성까지 할 수 있다. [리스트 4.1](#)의 코드에서 <your_path>에는 임의의 디렉터리를 지정하면 된

다. 그리고 이 책의 모든 데이터는 이 디렉터리에 저장하도록 하자. 예를 들어, 필자는 \$HOME/data를 사용한다.⁴

리스트 4.1 Fashion-MNIST 데이터로부터 DataLoader 작성⁵



```
import torch
from torch import nn, optim
from torch.utils.data import (Dataset, DataLoader, TensorDataset)
import tqdm

from torchvision.datasets import FashionMNIST
from torchvision import transforms

# 훈련용 데이터 가져오기
# 초기 상태에선 PIL (Python Imaging Library) 이미지 형식으로
# Dataset를 만들어 버린다.
# 따라서 transforms.ToTensor를 사용해 텐서로 변환한다
fashion_mnist_train = FashionMNIST("<your_path>/FashionMNIST",
    train=True, download=True,
    transform=transforms.ToTensor())
# 검증용 데이터 가져오기
fashion_mnist_test = FashionMNIST("<your_path>/FashionMNIST",
    train=False, download=True,
    transform=transforms.ToTensor())

# 배치 크기가 128인 DataLoader를 각각 작성
batch_size=128
train_loader = DataLoader(fashion_mnist_train,
    batch_size=batch_size, shuffle=True)
test_loader = DataLoader(fashion_mnist_test,
    batch_size=batch_size, shuffle=False)
```



```
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/ ➔
train-images-idx3-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/ ➔
```

- ⁴ 컬래버레이터를 사용하는 경우에는 데이터가 저장되는 기본 경로는 '/content'이다. 따라서 '<your_path>' 대신에 '/content'를 넣어 주면 된다.
- ⁵ 컬래버레이터의 경우 다운로드가 끝나면 왼쪽 'Files' 메뉴에서 'FashionMNIST' 폴더가 생성된 것을 확인할 수 있다. 참고로, 등록된 파일은 Runtime 세션이 종료되면 모두 사라지니 주의하도록 하자. 데이터가 지워졌다면 다시 다운로드해야 한다.

```
train-labels-idx1-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/ →
t10k-images-idx3-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/ →
t10k-labels-idx1-ubyte.gz
Processing...
Done!
```

4.2.2 CNN 구축과 학습

파이토치 이미지의 합성곱 처리를 위한 `nn.Conv2d`나 풀링을 위한 `nn.MaxPool2d` 등을 제공하므로 바로 CNN을 구축할 수 있다. **리스트 4.2**에서는 2계층 합성곱과 2계층 MLP를 연결한 CNN을 작성하고 있다.

파이토치에서는 `nn.Linear`에 입력 데이터의 차원을 반드시 입력해야 하지만, `nn.Conv2d`나 `nn.MaxPool2d`에서는 이미지 크기가 어떻게 바뀔지 예측하기가 힘들다. 따라서 `torch.ones` 함수로 작성한 더미(dummy) 데이터를 넣어서 계산하고 있다.

리스트 4.2 2층 합성곱과 2층 MLP를 연결한 CNN 작성

인

```
# (N, C, H, W) 형식의 텐서(N, C*H*W)로 불리는 계층
# 합성곱 출력을 MLP에 전달할 때 필요
class FlattenLayer(nn.Module):
    def forward(self, x):
        sizes = x.size()
        return x.view(sizes[0], -1)

# 5x5의 커널을 사용해서 처음에 32개, 다음에 64개의 채널 작성
# BatchNorm2d는 이미지용 Batch Normalization
# Dropout2d는 이미지용 Dropout
# 마지막으로 FlattenLayer 적용
conv_net = nn.Sequential(
    nn.Conv2d(1, 32, 5),
    nn.MaxPool2d(2),
    nn.ReLU(),
    nn.BatchNorm2d(32),
    nn.Dropout2d(0.25),
    nn.Conv2d(32, 64, 5),
    nn.MaxPool2d(2),
    nn.ReLU(),
```



```

nn.BatchNorm2d(64),
nn.Dropout2d(0.25),
FlattenLayer()
)

# 합성곱에 의해 최종적으로 이미지 크기가 어떤지름
# 더미 데이터를 넣어서 확인한다
test_input = torch.ones(1, 1, 28, 28)
conv_output_size = conv_net(test_input).size()[-1]

# 2층 MLP
mlp = nn.Sequential(
    nn.Linear(conv_output_size, 200),
    nn.ReLU(),
    nn.BatchNorm1d(200),
    nn.Dropout(0.25),
    nn.Linear(200, 10)
)

# 최종 CNN
net = nn.Sequential(
    conv_net,
    mlp
)

```

다음은 평가와 훈련을 위한 헬퍼 함수를 작성한다(**리스트 4.3**).

리스트 4.3 평가와 훈련용 헬퍼 함수 작성

↓ In

```

# 평가용 헬퍼 함수
def eval_net(net, data_loader, device="cpu"):
    # Dropout 및 BatchNorm을 무효화
    net.eval()
    ys = []
    ypreds = []
    for x, y in data_loader:
        # to 메서드로 계산을 실행할 디바이스로 전송
        x = x.to(device)
        y = y.to(device)
        # 확률이 가장 큰 분류를 예측(리스트 2.1 참고)
        # 여기서 forward (추론) 계산이 전부이므로 자동 미분에
        # 필요한 처리는 off로 설정해서 불필요한 계산을 제한한다
        with torch.no_grad():
            _, y_pred = net(x).max(1)
        ys.append(y)

```

```

ypreds.append(y_pred)

# 미니 배치 단위의 예측 결과 등을 하나로 묶는다
ys = torch.cat(ys)
ypreds = torch.cat(ypreds)
# 예측 정확도 계산
acc = (ys == ypreds).float().sum() / len(ys)
return acc.item()

# 훈련용 헬퍼 함수
def train_net(net, train_loader, test_loader,
              optimizer_cls=optim.Adam,
              loss_fn=nn.CrossEntropyLoss(),
              n_iter=10, device="cpu"):
    train_losses = []
    train_acc = []
    val_acc = []
    optimizer = optimizer_cls(net.parameters())
    for epoch in range(n_iter):
        running_loss = 0.0
        # 신경망을 훈련 모드로 설정
        net.train()
        n = 0
        n_acc = 0
        # 시간이 많이 걸리므로 tqdm을 사용해서 진행바를 표시
        for i, (xx, yy) in tqdm.tqdm(enumerate(train_loader),
                                     total=len(train_loader)):
            xx = xx.to(device)
            yy = yy.to(device)
            h = net(xx)
            loss = loss_fn(h, yy)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            n += len(xx)
            _, y_pred = h.max(1)
            n_acc += (yy == y_pred).float().sum().item()
        train_losses.append(running_loss / i)
        # 훈련 데이터의 예측 정확도
        train_acc.append(n_acc / n)

        # 검증 데이터의 예측 정확도
        val_acc.append(eval_net(net, test_loader, device))
        # epoch의 결과 표시
        print(epoch, train_losses[-1], train_acc[-1],
              val_acc[-1], flush=True)

```

이것으로 모든 준비가 됐다. 다음 **리스트 4.4**의 계산은 시간이 매우 오래 걸리므로 GPU 사용을 강하게 추천한다. 화면에 학습 진행 상황이 표시될 것이다.

리스트 4.4 모든 파라미터를 GPU로 전송해서 훈련 실행



```
# 신경망의 모든 파라미터를 GPU로 전송
net.to("cuda:0")

# 훈련 실행
train_net(net, train_loader, test_loader, n_iter=20,
device="cuda:0")
```



```
100%|*~| 469/469 [00:04<00:00, 99.15it/s]
0 0.3919695211717716 0.8644 0.8826999664306641
100%|>~| 469/469 [00:04<00:00, 99.95it/s]
1 0.2403168466228705 0.9113166666666667 0.896399974822998
100%|<<~| 469/469 [00:04<00:00, 104.50it/s]
2 0.19925596131982967 0.9264666666666667 0.9075999855995178
100%|*~| 469/469 [00:04<00:00, 101.92it/s]
3 0.1674202884045931 0.9389833333333333 0.9089999794960022
100%|<<~| 469/469 [00:04<00:00, 97.87it/s]
4 0.1434261213399024 0.9478833333333333 0.9157999753952026
( ...중략... )
16 0.15790939249862462 0.9412 0.9212999939918518
100%|<<~| 469/469 [00:10<00:00, 99.87it/s]
17 0.15936664253887203 0.94075 0.9160999655723572
100%|<<~| 469/469 [00:10<00:00, 99.87it/s]
18 0.15373828040006068 0.9216833333333333 0.9128999710083008
100%|<<~| 469/469 [00:11<00:00, 100.87it/s]
19 0.15288561017403746 0.9264 0.913999780654907
```

필자의 환경에선 20회 반복 중에 정확도가 가장 높았던 것이 0.926이었다. 참고로, 필자는 CPU는 core-i7 7700K, GPU는 GTX1060을 사용해서 검증했지만, GPU가 CPU보다 5배 정도 계산 속도가 빨랐다. 이것을 동일 계층 수의 MLP로 실행한 경우 가장 좋았던 결과는 0.88이었다. 합성곱이 더 효율적으로 학습된다는 것을 알 수 있다.

전이 학습

이 절에서는 전이 학습(transfer learning)이라는 신경망에서 볼 수 있는 매우 희귀하면서 유용한 학습법에 대해 다룬다. 전이 학습을 사용하면 특정 태스크를 위해 훈련된 모델을 다른 태스크에도 적용할 수 있어서 대량의 학습 데이터 없이도 복잡한 신경망을 훈련할 수 있다.

CNN에는 다양한 구조가 존재하며 VGG([메모](#) 참고), Inception([메모](#) 참고), ResNet([메모](#) 참고) 등은 정확도가 높기로 유명하다.

한편, 이 모델들은 층이 매우 깊은 신경망 구조로 대량의 파라미터를 가지고 있으며, 과합성 없이 파라미터를 최적화하려면 훈련용 이미지도 대량으로 필요하다. 이미지를 수집해야 하는 것뿐만 아니라 각각에 레이블을 붙이는 작업도 매우 힘든 작업이다.

CIFAR-10([메모](#) 참고)나 ImageNet([메모](#) 참고) 등 레이블이 할당된 범용 데이터를 사용할 때는 문제가 없지만, 실제로 자신의 서비스에서 사용할 이미지로 모델을 훈련하려고 하면 이런 작업은 많은 수고가 따른다. 또한, 이미지 데이터 자체를 수집하기 어려운 경우도 있다. 다행히 이 문제는 전이 학습이라는 기법을 이용하면 해결할 수 있다.



MEMO

VGG

옥스퍼드 대학의 VGG 그룹에 의해 고안된 모델로 3×3 의 작은 커널을 다중으로 겹쳐서 모델의 표현력을 높인 것이 특징이다.

- VERY DEEP CONVOLUTIONAL NETWORKS
FOR LARGE-SCALE IMAGE RECOGNITION

URL <https://arxiv.org/pdf/1409.1556.pdf>



MEMO

Inception

일명 GoogLeNet으로 불린다. Inception 모듈은 CNN과 비슷한 구조를 읽어서 전체 파라미터 수를 줄이면서 심층화에 성공한 모델이다.

- Going deeper with convolutions

URL <https://arxiv.org/pdf/1409.4842v1.pdf>



MEMO

ResNet

Residual 모듈이라는 단축 구조를 가지고 있으며, 이전 층의 입력도 그대로 다음 층에 전달하므로 경사 전달이 쉬우며, 깊은 신경망도 효율적으로 훈련할 수 있도록 개선한 구조다.

- Deep Residual Learning for Image Recognition

URL <https://arxiv.org/pdf/1512.03385.pdf>



MEMO

CIFAR-10

이미지 인식의 대표적인 예로 자주 언급되는 데이터다. 비행기, 자동차, 새, 고양이 등 총 10개의 카테고리를 가지고 있으며, 각 카테고리마다 훈련용 5,000장, 테스트용 1,000장의 이미지가 포함돼 있다(전체 6만 장의 이미지).

- The CIFAR-10 dataset

URL <http://www.cs.toronto.edu/~kriz/cifar.html>



MEMO

ImageNet

ImageNet은 이미지 인식 연구용으로 정비된 대규모 데이터로, ILSVRC라는 이미지 인식 대회에서도 사용되고 있다. 2만 개 이상의 카테고리로 구성되며, 이미지 수도 1,400만 장 이상이다.

- ImageNet

URL <http://www.image-net.org/>

전이 학습(transfer learning)이란, 특정 태스크(또는 도메인)에서 얻은 모델을 다른 태스크에 적용하는 기술을 말한다. 이미지 인식 분야의 신경망에서는 사전에 학습한 신경망의 마지막 출력 선형 계층만 새로 학습하고 다른 계층의 파라미터를 모두 고정시키면 좋은 결과를 얻을 수 있다고 알려져 있다. 특히, ImageNet이라는 대규모 이미지 인식 데이터로 사전 학습한 여러 신경망(ResNet 등)의 파라미터가 공개돼 있어서 전이 학습에 사용하기 좋다. 왜 이 방식이 좋은지는 사실 이론적으로 증명된 것은 없다(필자가 알고 있는 한). 단, CNN의 하위 계층에서 이미지를 인식할 때 필요한 일반적인 특징을 추출할 수 있다는 결과가 나와 있다.

4.3.1 데이터 준비

그러면 파이토치를 사용해 실제로 전이 학습을 구현해 보자. 여기서는 필자가 좋아하는 요리인 멕시코 음식 타코(taco)와 부리토(burrito)를 분류해 보도록 하겠다(그림 4.3). 참고로 타코와 부리토는 또띠아라는 인도의 난처럼 생긴 피에 고기나 야채를 넣어 싸 먹는 요리다. 타코는 옥수수로 만든 또띠아(피, **메모** 참고)를 사용하며, 부리토는 밀가루로 만든 것을 사용해서 타코보다 크기가 큰 것이 일반적이다.



MEMO

또띠아

또띠아(tortilla)는 동일한 스페인어라도 스페인과 멕시코에서 전혀 다른 음식을 가리킨다. 스페인에서는 오믈렛과 비슷한 음식을 의미한다.

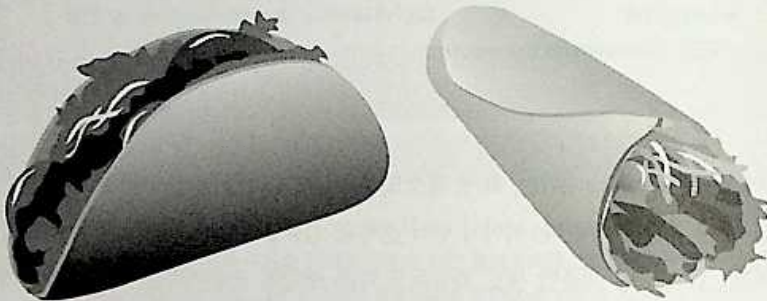


그림 4.3 왼쪽이 타코이고 오른쪽이 부리토

필자가 미리 웹에서 다운받아 크기를 조정한 이미지를 각각 400장 정도 준비해 두었다. 다음 URL을 통해 다운로드해서 임의의 디렉터리에 압축을 풀자.

- 멕시코 요리 타코와 부리토 데이터

URL https://github.com/lucidfrontier45/PyTorch-Book/raw/master/data/taco_and_burrito.tar.gz

이 압축 파일의 폴더 구성은 **그림 4.4**와 같다.

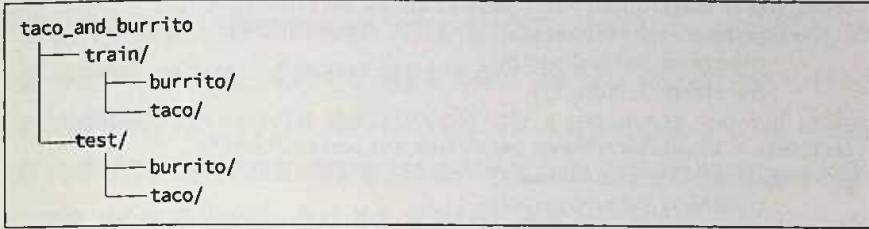


그림 4.4 분류용 데이터의 폴더 구조



MEMO

컬래버레이터에서 파일 다운로드 및 압축 파일 해제 방법

컬래버레이터의 경우 다음 명령을 실행하면 된다.

```
!wget https://github.com/lucidfrontier45/PyTorch-Book/raw/master/ =>
data/taco_and_burrito.tar.gz
!tar -zxvf taco_and_burrito.tar.gz
```

test에는 각각 30장씩 이미지가 저장돼 있으며 train에는 그외의 모든 이미지가 저장돼 있다. 이런 디렉터리 구조를 만들어 두면, torchvision의 ImageFolder로 읽어서 Dataset으로 쉽게 변환할 수 있다.

파이토치의 ImageNet을 사용한 학습 완료된 모델은 224×224 픽셀의 이미지를 입력 데이터로 받으므로 이 크기로 크롭(crop, 이미지를 자르는 처리)해야 한다. 학습용 데이터는 학습 결과를 향상시키기 위해 임의로 잘라 내고(RandomCrop) 검증용 데이터는 중심부를 잘라 낸다(CenterCrop)(**리스트 4.5**).

리스트 4.5 DataLoader 작성⁶

```
from torchvision.datasets import ImageFolder
from torchvision import transforms
```



⁶ **참고** 컬래버레이터의 경우 기본 디렉터리인 /content를 'your_path'에 지정하면 된다.

```
# ImageFolder 함수를 사용해서 Dataset 작성
train_imgs = ImageFolder("<your_path>/taco_and_burrito/train/",
    transform=transforms.Compose([
        transforms.RandomCrop(224),
        transforms.ToTensor()
    ])
test_imgs = ImageFolder("<your_path>/taco_and_burrito/test/",
    transform=transforms.Compose([
        transforms.CenterCrop(224),
        transforms.ToTensor()
    ])

# DataLoader 작성
train_loader = DataLoader(
    train_imgs, batch_size=32, shuffle=True)
test_loader = DataLoader(
    test_imgs, batch_size=32, shuffle=False)
```

ImageFolder를 사용하면 지정한 디렉터리의 하위 디렉터리명을 분류명으로 사용하며, 그 디렉터리에 있는 이미지와 분류 인덱스의 tuple을 반환하는 Dataset을 작성할 수 있다. 분류명과 분류 인덱스의 대응 관계는 **리스트 4.6**과 같이 확인할 수 있다.

리스트 4.6 분류명과 분류 인덱스 대응 관계 확인

```
print(train_imgs.classes)
```

↓ In

```
['burrito', 'taco']
```

↑ Out

```
print(train_imgs.class_to_idx)
```

↓ In

```
{'burrito': 0, 'taco': 1}
```

↑ Out

이것으로 데이터 준비가 완료됐다. 신경망 학습을 진행해 보자.

4.3.2 파이토치를 사용한 전이 학습

먼저, 사전 학습이 끝난(Pre-trained) 모델을 불러온다. 여기서는 ResNet18이라는 모델을 사용한다. 이 모델은 출력 선형 계층을 fc라는 이름으로 가져온다. 따라서 먼저 모든 파라미터를 미분 대상에서 제외시키고 fc에 새로운 선형 계층을 설정한다. 여기서는 두 가지 종류로 분류하므로 선형 계층의 출력 차원도 2로 설정한다(리스트 4.7). 새롭게 설정한 계층의 파라미터는 초기 설정 상태에서 미분 대상이 되지만, 이 방식으로 마지막 선형 계층만 미분 대상이 된다.

리스트 4.7 사전 학습이 끝난(Pre-trained) 모델 불러오기 및 정의⁷

```
from torchvision import models

# 사전 학습이 완료된 resnet18 불러오기
net = models.resnet18(pretrained=True)

# 모든 파라미터를 미분 대상에서 제외한다
for p in net.parameters():
    p.requires_grad=False

# 마지막 선형 계층을 변경한다
fc_input_dim = net.fc.in_features
net.fc = nn.Linear(fc_input_dim, 2)
```

In

```
Downloading: "https://download.pytorch.org/models/resnet18-5c106cde.pth" to /
content/.torch/models/resnet18-5c106cde.pth
100%|>| 46827520/46827520 [00:03<00:00, 11991926.36it/s]
```

Out

모델 정의는 이것이 전부다. 매우 간단하다는 것을 알 수 있을 것이다. 다음은 지금까지와 마찬가지로 모델의 훈련 함수를 작성하도록 한다. 유일한 차이는 fc의 파라미터만 최적화기에 전달하는 것이다(리스트 4.8). GPU로 실행하자(리스트 4.9).

⁷ 유분부 환경에서 두 번 이상 실행하는 경우에는 .torch/models/resnet18-5c106cde.pth를 삭제한 후 실행하도록 하자.


```

def eval_net(net, data_loader, device="cpu"):
    # Dropout 및 BatchNorm을 무효화
    net.eval()
    ys = []
    ypreds = []
    for x, y in data_loader:
        # to 메서드로 계산을 실행할 디바이스로 전송
        x = x.to(device)
        y = y.to(device)
        # 확률이 가장 큰 분류를 예측(리스트 2.11 참고)
        # 여기서 forward (추론) 계산이 전부이므로 자동 미분에
        # 필요한 처리는 off로 설정해서 불필요한 계산을 제한한다
        with torch.no_grad():
            _, y_pred = net(x).max(1)
        ys.append(y)
        ypreds.append(y_pred)

    # 미니 배치 단위의 예측 결과 등을 하나로 묶는다
    ys = torch.cat(ys)
    ypreds = torch.cat(ypreds)
    # 예측 정확도 계산
    acc = (ys == ypreds).float().sum() / len(ys)
    return acc.item()

def train_net(net, train_loader, test_loader, only_fc=True,
              optimizer_cls=optim.Adam,
              loss_fn=nn.CrossEntropyLoss(),
              n_iter=10, device="cpu"):
    train_losses = []
    train_acc = []
    val_acc = []
    if only_fc:
        # 마지막 섹션 계층의 파라미터만
        # optimizer에 전달
        optimizer = optimizer_cls(net.fc.parameters())
    else:
        optimizer = optimizer_cls(net.parameters())
    for epoch in range(n_iter):
        running_loss = 0.0
        # 신경망을 훈련 모드로 설정
        net.train()
        n = 0
        n_acc = 0
        # 시간이 많이 걸리므로 tqdm을 사용해서 진행 바를 표시
        for i, (xx, yy) in tqdm.tqdm(enumerate(train_loader),

```

```

total=len(train_loader)):
xx = xx.to(device)
yy = yy.to(device)
h = net(xx)
loss = loss_fn(h, yy)
optimizer.zero_grad()
loss.backward()
optimizer.step()
running_loss += loss.item()
n += len(xx)
_, y_pred = h.max(1)
n_acc += (yy == y_pred).float().sum().item()
train_losses.append(running_loss / i)
# 훈련 데이터의 예측 정확도
train_acc.append(n_acc / n)

# 검증 데이터의 예측 정확도
val_acc.append(eval_net(net, test_loader, device))
# epoch의 결과 표시
print(epoch, train_losses[-1], train_acc[-1],
      val_acc[-1], flush=True)

```

리스트 4.9 모든 파라미터를 GPU로 전송해서 훈련 실행



```

# 신경망의 모든 파라미터를 GPU로 전송
net.to("cuda:0")

# 훈련 실행
train_net(net, train_loader, test_loader, n_iter=20, device="cuda:0")

```



```

100%|* | 23/23 [00:02<00:00, 9.56it/s]
0 0.6974282251162962 0.601123595505618 0.8333333730697632
100%|* | 23/23 [00:01<00:00, 11.78it/s]
1 0.517780906774781 0.773876404494382 0.8500000238418579
(...중략...)
100%|* | 23/23 [00:01<00:00, 11.81it/s]
8 0.39653965492140164 0.848314606741573 0.8500000238418579
100%|* | 23/23 [00:02<00:00, 11.34it/s]
9 0.3141689578240568 0.8806179775280899 0.8833333849906921
(...중략...)

```

ResNet18은 꽤 크기가 큰 신경망으로 여기서도 GPU 이용을 추천한다. 필자의 환경에서는 10배 정도 속도 차이가 난다. 실행하면 10회 정도의 반복으로 검증용 데이터에 대해 약 88%의 정확도를 달성한다.

참고로, fc 계층 이외는 파라미터가 변하지 않아서 매회 똑같은 계산을 불필요하게 발생한다. 따라서 사전에 계산을 두고 그것을 입력하는 로지스틱 회귀 모델을 훈련하는 것도 좋은 방법이다. 마지막의 fc 계층을 제거하는 방법에는 여러 가지가 있지만,

리스트 4.10과 같이 입력한 것을 그대로 출력하는 더미 계층을 만들어 fc를 변경하는 것이 범용성이 높다.

리스트 4.10 입력을 그대로 출력하면 더미 계층을 만들어 fc를 변경



```
class FlattenLayer(nn.Module):
    def forward(self, x):
        sizes = x.size()
        return x.view(sizes[0], -1)

class IdentityLayer(nn.Module):
    def forward(self, x):
        return x

net = models.resnet18(pretrained=True)
for p in net.parameters():
    p.requires_grad=False
net.fc = IdentityLayer()
```

또한, 필자가 작성한 **리스트 4.11**의 CNN을 테스트한 결과, 학습 속도도 충분하고 정확도도 70% 정도로 전이 학습이 유용하다는 것을 알 수 있다.

리스트 4.11 필자가 작성한 CNN 모델 실행



```
conv_net = nn.Sequential(
    nn.Conv2d(3, 32, 5),
    nn.MaxPool2d(2),
    nn.ReLU(),
    nn.BatchNorm2d(32),
    nn.Conv2d(32, 64, 5),
    nn.MaxPool2d(2),
    nn.ReLU(),
    nn.BatchNorm2d(64),
```

```

nn.Conv2d(64, 128, 5),
nn.MaxPool2d(2),
nn.ReLU(),
nn.BatchNorm2d(128),
FlattenLayer()
)

# 합성곱에 의해 최종적으로 어떤 크기인지
# 실제로 데이터를 넣어서 확인
test_input = torch.ones(1, 3, 224, 224)
conv_output_size = conv_net(test_input).size()[~1]

# 최종 CNNN
net = nn.Sequential(
    conv_net,
    nn.Linear(conv_output_size, 2)
)

# 신경망의 모든 파라미터를 GPU로 전송
net.to("cuda:0")

# 훈련 실행
train_net(net, train_loader, test_loader, n_iter=10, only_fc=False, ➡
device="cuda:0")

```

 Out

```

100%| 23/23 [00:03<00:00, 7.41it/s]
0 2.2579465102065694 0.5898876404494382 0.5
100%| 23/23 [00:03<00:00, 7.53it/s]
1 2.665352626280351 0.6137640449438202 0.550000011920929
(...중략...)
100%| 23/23 [00:03<00:00, 7.37it/s]
9 2.2272912561893463 0.675561797752809 0.7000000476837158

```

4.4

CNN 회귀 모델을 사용한 이미지 해상도 향상

지금까지 CNN을 사용한 분류 방법을 보았지만, CNN은 회귀 문제에도 적용할 수 있다. 구체적으로는 CNN을 이용하면 원 이미지를 다른 이미지로 변환할 수 있다. 여기서는 CNN을 이용해서 얼굴 이미지의 해상도를 향상시켜 본다. 마치 과학수사 드라마에서 사진이나 영상을 확대해 범인의 얼굴을 확인하는 것과 같은 효과를 볼 수 있도록 CNN을 훈련시킨다.

4.4.1 데이터 준비

여기서는 대표적인 얼굴 이미지 데이터인 LFW(Labeled Faces in the Wild)에서 얼굴이 수직으로 나열돼 있는 **LFW deep funneled images**라는 데이터를 사용한다. 데이터는 다음 URL에서 All images aligned with deep funneling이라는 링크를 클릭하면 다운로드할 수 있다.

- Labeled Faces in the Wild Home

URL <http://vis-www.cs.umass.edu/lfw/>

다운로드해서 안을 보면 사람 이름으로 되어 있는 수많은 디렉터리를 볼 수 있을 것이다. 이것들을 훈련용 및 검증용으로 적당히 나누어서 **그림 4.5**에 있는 디렉터리 구성으로 만들어 보자. 필자는 XYZ로 시작하는 인물을 모두 test로 이동하고 나머지는 train으로 이동했다.

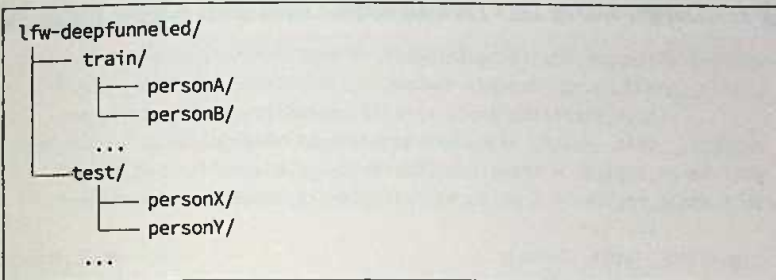


그림 4.5 LFW deep funneled images의 디렉터리 구성



MEMO

컬래버레이터에서 다운로드, 압축 파일 해제 및 디렉터리 생성, 이동

컬래버레이터에서는 다음 명령을 실행하면 된다.

```

!wget http://vis-www.cs.umass.edu/lfw/lfw-deepfunneled.tgz
!tar -zxvf lfw-deepfunneled.tgz
!mkdir lfw-deepfunneled/train
!mv lfw-deepfunneled/[A-W]* lfw-deepfunneled/train
!mkdir lfw-deepfunneled/test
!mv lfw-deepfunneled/[X-Z]* lfw-deepfunneled/test

```

● Dataset과 DataLoader 준비

다음은 Dataset과 DataLoader를 준비해야 한다. 여기서 32×32 픽셀의 이미지를 128×128 픽셀로 확대하는 것이 목적이므로 [리스트 4.12](#)와 같이 torchvision의 Resize를 사용해서 ImageFolder를 확장하고, (작은 이미지, 큰 이미지)의 튜플을 반환하는 Dataset을 설정한다. 참고로, Resize는 변환 후의 이미지 크기를 (h, w) 형의 튜플 또는 정수로 받는다. 튜플의 경우는 지정한 크기로 바로 조정이 되지만, 정수로 받는 경우는 작은 값을 기준으로 세로나 가로를 맞춘 후 나머지를 동일한 비율로 변환한다.

여기서 사용하는 LFW 데이터는 모두 250×250 픽셀의 정방형이지만, 일반적인 이미지 크기는 정해져 있지 않다. 만약 정방형의 이미지를 원한다면 Resize 후에 CenterCrop을 적용하면 된다(구체적인 예는 생략한다).

리스트 4.12 32×32픽셀의 이미지를 128×128픽셀로 확대⁸

```
class DownSizedPairImageFolder(ImageFolder):
    def __init__(self, root, transform=None,
                 large_size=128, small_size=32, **kwargs):
        super().__init__(root, transform=transform, **kwargs)
        self.large_resizer = transforms.Resize(large_size)
        self.small_resizer = transforms.Resize(small_size)

    def __getitem__(self, index):
        path, _ = self.imgs[index]
        img = self.loader(path)

        # 읽은 이미지를 128×128픽셀과 32×32픽셀로 리사이즈
        large_img = self.large_resizer(img)
        small_img = self.small_resizer(img)

        # 기타 변환 적용
        if self.transform is not None:
            large_img = self.transform(large_img)
            small_img = self.transform(small_img)

        # 32픽셀의 이미지와 128픽셀의 이미지 반환
        return small_img, large_img
```

● 훈련용과 검증용 DataLoader 작성

다음은 이것을 이용해서 훈련용과 검증용 DataLoader를 만들어야 한다(리스트 4.13). 여기서는 리사이즈(resize) 등 데이터 가공을 위한 CPU 처리가 많으므로 DataLoader의 num_workers를 늘려서 처리를 병렬화하고 있다. 필자의 환경에서는 CPU가 4코어이므로 num_workers=4를 지정하고 있다.

이것으로 데이터 준비가 끝났다.

8  컬래버레이터를 사용하는 경우 환경이 리셋됐을 수도 있다. 이 경우 다음 import를 추가하자. 필요하면 pytorch도 재설치해야 한다.

```
from torchvision.datasets import ImageFolder
from torchvision import transforms
```

리스트 4.13 훈련용과 검증용 DataLoader 작성⁹



```
train_data = DownSizedPairImageFolder(
    "<your_path>/lfw-deepfunneled/train",
    transform=transforms.ToTensor())
test_data = DownSizedPairImageFolder(
    "<your_path>/lfw-deepfunneled/test",
    transform=transforms.ToTensor())

batch_size = 32
train_loader = DataLoader(train_data, batch_size=batch_size,
                           shuffle=True, num_workers=4)
test_loader = DataLoader(test_data, batch_size=batch_size,
                          shuffle=False, num_workers=4)
```

입력된 디렉터리 지정

4.4.2 모델 작성

신경망은 Conv2d와 ConvTransposed2d에 stride=2를 설정해서 CNN에 가중치를 부여할 수 있다. stride=2는 합성곱 커널을 움직일 때 2픽셀씩 움직이라는 뜻이다. Conv2d에 이 설정을 하면 풀링 계층의 역할도 겸하므로 MaxPool2d를 넣지 않아도 이미지의 크기가 1/2이 된다.

한편, ConvTransposed2d는 Transposed Convolution이라는 합성곱 연산을 사용한다. 이는 합성곱 커널을 행렬로 전개했을 때 원래의 합성곱 행렬의 가로, 세로를 바꾸는 것이다(이를 전치(transpose)라고 한다). Transposed Convolution을 stride=2로 설정하면 이미지 크기가 약 2배가 된다. 즉, 입력과 출력 이미지 크기가 원래 합성곱의 반대가 되는 것이다. 이 성질을 이용해서 이미지를 확대할 수 있다.

리스트 4.14에서는 Conv2d를 두 개, ConTransposed2d를 네 개 연결한 CNN을 만들고 있다. 이것으로 이미지가 4배 확대된다. 활성화 함수에는 ReLU를 선택하고, Batch Normalization도 사용한다. 합성곱 계산 시에는 크기가 2배나 1/2 정도 미묘하게 다른 경우가 있으므로 padding을 넣어서 조정하고 있다. 이 부분은 실제로 더미 데이터를 넣어서 출력 크기를 확인하며 조절하는 것이 좋다.

⁹ 컬래버레이터의 경우 your_path를 /content로 지정하면 된다

리스트 4.14 모델 작성



```
net = nn.Sequential(
    nn.Conv2d(3, 256, 4,
              stride=2, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(256),
    nn.Conv2d(256, 512, 4,
              stride=2, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(512),
    nn.ConvTranspose2d(512, 256, 4,
                       stride=2, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(256),
    nn.ConvTranspose2d(256, 128, 4,
                       stride=2, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(128),
    nn.ConvTranspose2d(128, 64, 4,
                       stride=2, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(64),
    nn.ConvTranspose2d(64, 3, 4, stride=2, padding=1)
)
```

신경망 정의를 완료했으니 훈련 부분을 구현하도록 하자. 지금까지와 다른 점은 회귀 처리로 손실 함수에 MSE를 사용한다는 점이다. 원래 이미지와 확대한 이미지 사이의 MSE를 최소화하도록 CNN을 훈련한다. MSE 계산에는 nn.MSELoss 클래스의 인스턴스를 만드는 방법 외에도 nn.functional.mse_loss라는 함수를 사용하는 방법도 있다 (예제 참고). 이미지나 음성 등의 신호를 복원하는 처리에서는 MSE가 아닌 PSNR이라는 지표를 자주 사용하므로 여기에서도 마지막 결과를 표시할 때 이것을 이용한다. 단, PSNR은 MSE와 일대일 대응되므로 반드시 이 작업이 필요한 것은 아니다. PSNR은 다음 식으로 계산할 수 있다.

$$\text{PSNR} = 10 * \log_{10} (\text{MAX_}^2 / \text{MSE})$$



MEMO

nn.functional

nn.functional의 네임스페이스에는 nn으로 정의돼 있는 다양한 클래스를 바로 사용할 수 있는 함수들이 준비돼 있다.

MAX_I는 신호 강도의 최댓값으로 8비트의 정수인 경우 255가 되지만, 파이토치에서는 이미지를 [0, 1]의 실수로 표현하므로 MAX_I에 1을 대입한다([리스트 4.15](#)).

리스트 4.15 PSNR 계산



```
import math
def psnr(mse, max_v=1.0):
    return 10 * math.log10(max_v**2 / mse)

# 평가 헬퍼 함수
def eval_net(net, data_loader, device="cpu"):
    # Dropout 및 BatchNorm을 무효화
    net.eval()
    ys = []
    ypreds = []
    for x, y in data_loader:
        x = x.to(device)
        y = y.to(device)
        with torch.no_grad():
            y_pred = net(x)
        ys.append(y)
        ypreds.append(y_pred)
    # 미니 배치 단위로 예측 결과 등을 하나로 모은다
    ys = torch.cat(ys)
    ypreds = torch.cat(ypreds)
    # 예측 정확도(MSE) 계산
    score = nn.functional.mse_loss(ypreds, ys).item()
    return score

# 훈련 헬퍼 함수
def train_net(net, train_loader, test_loader,
              optimizer_cls=optim.Adam,
              loss_fn=nn.MSELoss(),
              n_iter=10, device="cpu"):
    train_losses = []
```



```

train_acc = []
val_acc = []
optimizer = optimizer_cls(net.parameters())
for epoch in range(n_iter):
    running_loss = 0.0
    # 신경망을 훈련 모드로 설정
    net.train()
    n = 0
    score = 0
    # 시간이 많이 걸리므로 tqdm를 이용해서
    # 진행 바 표시
    for i, (xx, yy) in tqdm.tqdm(enumerate(train_loader),
                                total=len(train_loader)):
        xx = xx.to(device)
        yy = yy.to(device)
        y_pred = net(xx)
        loss = loss_fn(y_pred, yy)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        n += len(xx)
    train_losses.append(running_loss / len(train_loader))
    # 검증 데이터의 훈련 정확도
    val_acc.append(eval_net(net, test_loader, device))
    # epoch의 결과 표시
    print(epoch, train_losses[-1],
          psnr(train_losses[-1]), psnr(val_acc[-1]), flush=True)

```

GPU로 10회 정도 반복한다([리스트 4.16](#)).

리스트 4.16 여러 번 반복 연산(10회)¹⁰

```

net.to("cuda:0")
train_net(net, train_loader, test_loader, device="cuda:0")

```

 In

```

100%|...| 409/409 [00:39<00:00, 10.48it/s]
0 0.017855500012253635 17.482279836385406 25.252477575679613
100%|...| 409/409 [00:39<00:00, 10.45it/s]

```

 Out

¹⁰  **29** 컬래버레이터 사용 중 PIL 관련 오류가 발생하는 경우 4장의 코드를 처음부터 다시 실행해야 한다. 본 예제는 메모리를 많이 사용하므로 메모리 관련 오류가 발생할 수도 있다.

```
1 0.0032189228471760207 24.922894325866054 24.594987626612376
(...중략...)
100%|**| 409/409 [00:39<00:00, 10.40it/s]
9 0.0021821165788681917 26.611220510864925 27.10737505859327
```

CNN 훈련이 완료됐다. **리스트 4.17** 처럼 실제로 몇 개의 이미지를 확대해서 원 이미지와 비교해 보자. 또한, 일반적으로 사용되는 Bilinear 보간법을 사용한 확대 예도 함께 출력한다. 이미지 출력에는 torchvision.utils.save_image(**메모** 참고)를 사용하면 편리하다. 이 함수는 텐서 리스트를 받아서 이미지 파일로 저장할 수 있다.



MEMO

torchvision.utils.save_image

URL <https://pytorch.org/docs/stable/torchvision/utils.html>

리스트 4.17 이미지를 확대해서 원본 이미지와 비교



```
from torchvision.utils import save_image

# 테스트 데이터로부터 랜덤으로 4개씩 추출하는 DataLoader
random_test_loader = DataLoader(test_data, batch_size=4, shuffle=True)
# DataLoader를 파이썬의 이터레이터로 변환해서 4개의 예로 추출
it = iter(random_test_loader)
x, y = next(it)

# Bilinear를 사용한 확대
bl_recon = torch.nn.functional.upsample(x, 128, mode="bilinear", align_corners=True)
# CNN으로 확대
yp = net(x.to("cuda:0")).to("cpu")

# torch.cat로 원본, Bilinear, CNN 이미지를 결합하고
# save_image로 결합한 이미지를 출력(저장)
save_image(torch.cat([y, bl_recon, yp], 0), "cnn_upscale.jpg", nrow=4)
```



그림 4.6 참고(컬래버레이터의 경우는 **메모** 참고)

그림 4.6 는 확대 결과를 보여 준다. 이처럼 CNN을 사용하면 Bilinear 보간법보다 훨씬 우수한 확대 이미지를 생성할 수 있다.



그림 4.6 얼굴 이미지 확대 결과: (상) 원본, (중) Bilinear 보간, (하) CNN



MEMO

컬래버레이터에서의 이미지 표시

컬래버레이터에서는 다음 명령을 실행하면 이미지가 표시된다.

```
from IPython.display import Image, display_jpeg
display_jpeg(Image('cnn_upscale.jpg'))
```

DCGAN을 사용한 이미지 생성

마지막으로, 적대적 생성 신경망(GAN, Generative Adversarial Network)을 CNN과 결합한 DCGAN(Deep Convolutional Generative Adversarial Networks)이라는 모델을 사용해서 이미지를 생성하는 방법을 소개한다. GAN은 딥러닝 영역에서 현재 가장 화제가 되고 있는 기법이다. DCGAN을 시작으로 다양한 모델이 고안되고 있으며, 실제로 이미지를 생성할 수 있다는 점에서도 많은 주목을 받고 있다.

4.5.1 GAN이란

GAN은 일반적인 신경망 학습과 달리 생성 모델 G (Generator)와 식별 모델 D (Discriminator)를 사용해 상호 간 학습을 진행한다. G 는 K 차원의 잠재적 특이 벡터를 입력값으로 받아서 대상(예를 들어, 64×64 픽셀의 이미지)과 동일 형식의 데이터를 생성하는 신경망이다. D 는 지금까지 본 것과 마찬가지로 대상 데이터를 입력값으로 받아 진위를 식별하는 신경망이다. GAN의 학습 순서를 수식을 사용하지 않고 설명하면 대략 다음과 같다.

1. 잠재적 특이 벡터 z 를 난수로 생성하고, $G(z)$ 를 사용해 가짜 데이터(fake_data)를 생성한다.

$$\text{fake_data} \leftarrow G(z)$$
2. fake_data를 D 로 식별한다. $\text{fake_out} \leftarrow D(\text{fake_data})$
3. 진짜 데이터의 샘플(real_data)을 D 로 식별한다. $\text{real_out} \leftarrow D(\text{real_data})$
4. fake_out이 진짜 데이터(1)라고 간주하고, 크로스 엔트로피 함수를 계산해서 G 의 파라미터를 갱신한다.
5. real_out이 진짜 데이터이고 fake_out이 가짜 데이터(0)라고 간주하고, 크로스 엔트로피 함수를 계산해서 D 의 파라미터를 갱신한다.

4에서 G가 생성한 데이터를 D를 잘 속였다고 해서 G를 갱신하며, 반대로 5에서는 D가 이 속임수를 잘 간파했다고 보고 D를 갱신한다. 이처럼 서로를 훈련시키는 것이 GAN의 핵심이다. G나 D에 깊은 CNN을 사용한 것이 DCGAN이다.

4.5.2 데이터 준비

실제로 DCGAN을 파이토치로 구현하기에 앞서 사용할 데이터를 준비하도록 한다. 여기서는 **Oxford 102**라는 꽃 데이터를 사용한다. 102종류의 꽃을 약 8,000장의 이미지 데이터로 제공한다. 데이터는 다음 URL에 있는 Dataset images에서 다운로드할 수 있다.

- 102 Category Flower Dataset

URL <http://www.robots.ox.ac.uk/~vgg/data/flowers/102/>

다운로드해서 압축 해제한 후 ImageFolder로 읽는다. 이를 위해서는 **그림 4.7**과 같은 디렉터리 구성으로 만들어야 한다.

```

oxford-102/
├── jpg/
│   ├── image_00000.jpg
│   ├── image_00001.jpg
│   ...

```

그림 4.7 ImageFolder의 디렉터리 구성



MEMO

컬래버레이터에서의 압축 파일 해제, 디렉터리 생성 및 이동

컬래버레이터에서는 다음 명령을 실행하면 된다.

```

!wget http://www.robots.ox.ac.uk/~vgg/data/flowers/102/ →
102flowers.tgz
!tar xf 102flowers.tgz
!mkdir oxford-102
!mkdir oxford-102/jpg
!mv jpg/*.jpg oxford-102/jpg

```


디렉터리와 파일 준비가 됐으면 늘 하던 것처럼 DataLoader를 만든다(리스트 4.18). 여기서는 64×64 픽셀의 이미지를 생성하므로 예제 데이터의 짧은 변을 80픽셀로 조절(resize)한 후, 가운데를 64×64 픽셀로 잘라내도록(crop) 한다. 이것으로 데이터 준비가 완료된다.

리스트 4.18 DataLoader 생성



```
img_data = ImageFolder("<your_path>/oxford-102/",
    transform=transforms.Compose([
        transforms.Resize(80),
        transforms.CenterCrop(64),
        transforms.ToTensor()
    ]))

batch_size = 64
img_loader = DataLoader(img_data, batch_size=batch_size,
    shuffle=True)
```

4.5.3 파이토치를 사용한 DCGAN

파이토치를 사용해서 DCGAN을 만들도록 한다. 여기서 소개하는 구현 방법은 DCGAN의 원 논문인 'dcgan-paper'(메모 참고)와 파이토치의 공식 예제에 있는 DCGAN인 'dcgan-pytorch-example'(메모 참고)을 기반으로 해서 가능한 한 알기 쉽게 필자가 재구성했다.



MEMO

DCGAN 원저자 논문

- Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks(dcgan-paper라고 일반적으로 불린다)

 <https://arxiv.org/pdf/1511.06434.pdf>



MEMO

dcgan-pytorch-example

- Deep Convolution Generative Adversarial Networks
(dcgan-pytorch-example라고 일반적으로 불린다)

URL <https://github.com/pytorch/examples/tree/master/dcgan>

● 잠재적 특이 벡터 z 를 100차원으로 만들기

먼저, 잠재적 특이 벡터 z 를 100차원으로 만든다. 이 z 로부터 $3 \times 64 \times 64$ (3은 3색을 나타냄)의 이미지를 만드는 생성 모델을 구축한다. 4.4절에서 다룬 Transposed Convolution (ConvTranspose2d)를 사용한다(**리스트 4.19**).

리스트 4.19 이미지 생성 모델 구축



```
nz = 100
ngf = 32

class GNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, ngf * 8,
                              4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(ngf * 8, ngf * 4,
                              4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(ngf * 4, ngf * 2,
                              4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(ngf * 2, ngf,
                              4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(ngf, 3,
                              4, 2, 1, bias=False),
```

```

        nn.Tanh()
    )

    def forward(self, x):
        out = self.main(x)
        return out

```

Transposed Convolution을 전부 5회 반복하고 있다. 이를 통해 처음에는 $100 \times 1 \times 1$ 의 z 가 $256 \times 4 \times 4$ 로 변환되며, 최종적으로는 $3 \times 64 \times 64$ 가 된다. 100이 아닌 $100 \times 1 \times 1$ 처럼 이미지와 동일하게 CHW 차원으로 z 를 만드는 것이 핵심이다.

Transposed Convolution에 의해 이미지 크기는 다음과 같이 변환한다. 참고로, 직접 변경하고 싶다면 이 식을 역으로 계산하면 된다.

```

out_size = (in_size - 1) * stride - 2 * padding \
            + kernel_size + output_padding

```

예를 들어, 첫 번째 Transposed Convolution에서는 다음과 같이 된다.

```

in_size = 1
stride = 1
padding = 0
kernel_size = 4
output_padding = 0

```

따라서 $(1-1) * 1 - 2 * 0 + 4 + 0 = 4$ 가 된다. 활성화 함수의 선택이나 Batch Normalization을 사용한다는 것은 원 논문에 언급돼 있는 것이다.

● 식별 모델 작성

다음은 식별 모델이다. 이것은 $3 \times 64 \times 64$ 이미지를 최종적으로는 1차원의 스칼라로 변환하는 신경망을 만들면 된다. 다양한 방법이 있지만 원 논문에서는 선형 계층을 사용하지 않는 방법을 추천하고 있으므로 여기에서도 이를 따르고 있다([리스트 4.20](#)).

```

ndf = 32

class DNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.main = nn.Sequential(
            nn.Conv2d(3, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
        )

    def forward(self, x):
        out = self.main(x)
        return out.squeeze()

```

5회 합성곱 연산으로 $3 \times 64 \times 64$ 이미지가 최종적으로 $1 \times 1 \times 1$ 이 된다. 그리고 forward 마지막에 있는 squeeze는 $A \times 1 \times B \times 1$ 처럼 불필요한 1이 들어 있는 shape를 $A \times B$ 처럼 조정하는 처리다.

Conv2d는 입력과 출력 모두 (batch_size, channel, height, width) 형식으로, 여기서는 최종적으로 (batch_size, 1, 1, 1)이 되므로 squeeze로 불필요한 차원을 삭제한다.

● 훈련 함수 작성

다음은 훈련 함수를 만든다(리스트 4.21). 먼저, 신경망 및 최적화기 등을 준비한다. DCGAN 도 시간이 많이 걸리는 계산이므로 GPU가 필수다. 여기서는 CPU 처리는 생략한다.

리스트 4.21 훈련 함수 작성



```
d = DNet().to("cuda:0")
g = GNet().to("cuda:0")

# Adam의 파라미터는 원 논문에서 제안하는 값
opt_d = optim.Adam(d.parameters(),
                    lr=0.0002, betas=(0.5, 0.999))
opt_g = optim.Adam(g.parameters(),
                    lr=0.0002, betas=(0.5, 0.999))

# 크로스 엔트로피를 계산하기 위한 보조 변수 등
ones = torch.ones(batch_size).to("cuda:0")
zeros = torch.zeros(batch_size).to("cuda:0")
loss_f = nn.BCEWithLogitsLoss()

# 모니터링용 z
fixed_z = torch.randn(batch_size, nz, 1, 1).to("cuda:0")
```

fixed_z는 훈련 모니터링용이고 훈련이 진행됨에 따라 어떤 이미지가 만들어지는지 확인한다. 실제 훈련 함수는 [리스트 4.22](#)와 같다.

리스트 4.22 훈련 함수



```
from statistics import mean

def train_dcgan(g, d, opt_g, opt_d, loader):
    # 생성 모델, 식별 모델의 목적 함수 추적용 배열
    log_loss_g = []
    log_loss_d = []
    for real_img, _ in tqdm.tqdm(loader):
        batch_len = len(real_img)

        # 실제 이미지를 GPU로 복사
        real_img = real_img.to("cuda:0")

        # 가짜 이미지를 난수와 생성 모델을 사용해 만든다
        z = torch.randn(batch_len, nz, 1, 1).to("cuda:0")
        fake_img = g(z)

        # 나중에 사용하기 위해서 가짜 이미지의 값만 별도로 저장해 둬
        fake_img_tensor = fake_img.detach()

        # 가짜 이미지에 대한 생성 모델의 평가 함수 계산
        out = d(fake_img)
```



```

loss_g = loss_f(out, ones[: batch_len])
log_loss_g.append(loss_g.item())

# 계산 그래프가 생성 모델과 식별 모델 양쪽에
# 의존하므로 양쪽 모두 경사하강을 끝낸 후에
# 미분 계산과 파라미터 갱신을 실시
d.zero_grad(), g.zero_grad()
loss_g.backward()
opt_g.step()

# 실제 이미지에 대한 식별 모델의 평가 함수 계산
real_out = d(real_img)
loss_d_real = loss_f(real_out, ones[: batch_len])

# 파이토치에서는 동일 텐서를 포함한 계산 그래프에
# 2회 backward를 할 수 없으므로 저장된 텐서를
# 사용해서 불필요한 계산은 생략
fake_img = fake_img_tensor

# 가짜 이미지에 대한 식별 모델의 평가 함수 계산
fake_out = d(fake_img_tensor)
loss_d_fake = loss_f(fake_out, zeros[: batch_len])

# 진위 평가 함수의 합계
loss_d = loss_d_real + loss_d_fake
log_loss_d.append(loss_d.item())

# 식별 모델의 미분 계산과 파라미터 갱신
d.zero_grad(), g.zero_grad()
loss_d.backward()
opt_d.step()

return mean(log_loss_g), mean(log_loss_d)

```

● DCGAN의 훈련 시작

이것으로 모든 준비가 끝났다. DCGAN의 훈련을 시작하자(리소스429). 300회 반복하지만, 100회를 전후로 해서 이미 좋은 품질의 이미지를 얻을 수 있으므로 거기서 중지해도 괜찮다.

리스트 4.23 DCGAN의 훈련



```
for epoch in range(300):
    train_dcgan(g, d, opt_g, opt_d, img_loader)
    # 10회 반복마다 학습 결과를 저장
    if epoch % 10 == 0:
        # 파라미터 저장
        torch.save(
            g.state_dict(),
            "<out_path>/g_{:03d}.prm".format(epoch),
            pickle_protocol=4)
        torch.save(
            d.state_dict(),
            "<out_path>/d_{:03d}.prm".format(epoch),
            pickle_protocol=4)
        # 모니터링용 z로부터 생성한 이미지 저장
        generated_img = g(fixed_z)
        save_image(generated_img,
                   "<out_path>/{:03d}.jpg".format(epoch))
```

* 출력에 시간이 걸린다



그림 4.8 참고

torch.save는 신경망의 파라미터를 디스크에 저장하기 위한 것이다. 자세한 내용은 7장에서 다룬다. 필자의 환경에서는 300회 반복에 40~50분 정도가 걸렸다. 최종적으로는 fixed_z로부터 그림 4.8과 같은 이미지를 얻을 수 있었다.



MEMO

컬래버레이터에서 이미지 표시

컬래버레이터에서는 다음 명령을 실행하면 된다.

```
from IPython.display import Image, display_jpeg
display_jpeg(Image('oxford-102/000.jpg'))
```

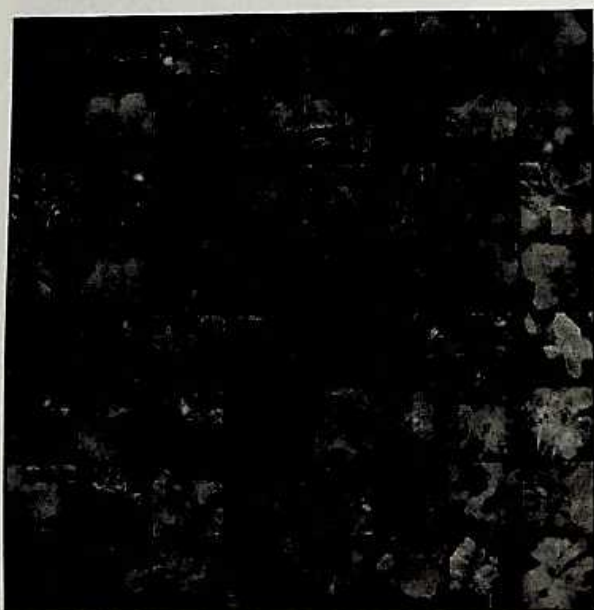
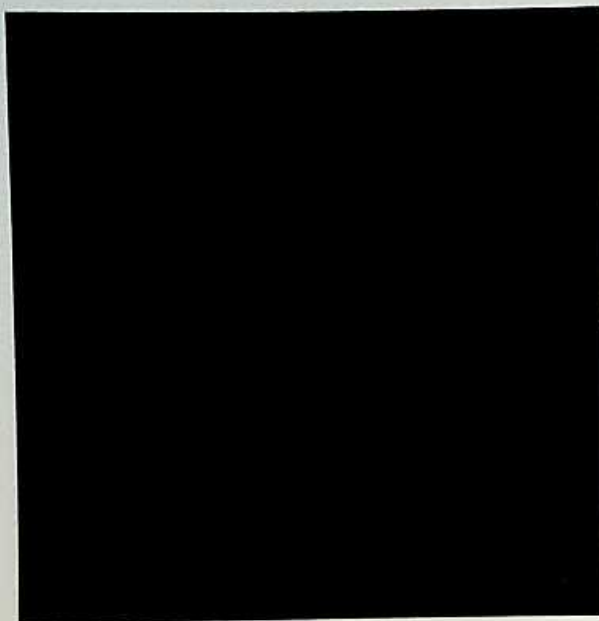


그림 4.8 (왼쪽 위) epoch=1, (왼쪽 아래) epoch=50, (오른쪽 위) epoch=150, (오른쪽 아래) epoch=300. 처음에는 아무것도 보이지 않는 이미지가 시간이 지날수록 꽃 모양을 갖추어 가는 것을 볼 수 있다. (계속)

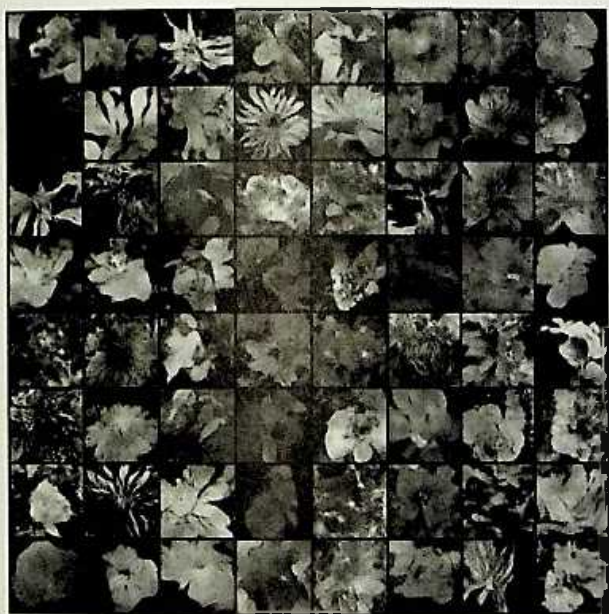


그림 4.8 (왼쪽 위) epoch=1, (왼쪽 아래) epoch=50, (오른쪽 위) epoch=150, (오른쪽 아래) epoch=300. 처음에는 아무것도 보이지 않는 이미지가 시간이 지날수록 꽃 모양을 갖추어 가는 것을 볼 수 있다.

GAN의 이미지 생성을 응용하는 방법에 대해서는 다양한 연구가 진행 중이며, BEGAN, WGAN, CycleGAN, DiscoGAN, StarGAN 등의 다양한 모델이 고안되고 있다. 그리고 주목해야 할 것은 이 모델들은 모두 파이토치로 구현돼서 깃허브(GitHub)에 공개돼 있다는 것이다. 이처럼 많은 학자들이 사용하고 있으며, 최신 연구가 파이토치로 구현되고 있다는 점은 파이토치가 지닌 강점이기도 하다. 관심 있는 독자는 꼭 소스 코드들을 깃허브를 통해 얻어서 코드를 읽거나 실행해 보자.

정리

이 장에서 다룬 내용을 정리했다.

파이토치를 사용한 CNN 구축 및 학습에 대해 살펴보았다. 파이토치는 이미 학습이 완료된 다양한 CNN 모델을 제공하고 있으므로 이를 이용해서 전이 학습을 하면 다량의 이미지 데이터 없이도 원하는 모델을 만들 수 있다. 또한, 이미지의 해상도 향상(확대)은 물론 DCGAN 등을 사용하면 새로운 이미지를 생성할 수도 있다.



CHAPTER

5

자연어 처리와 순환 신경망

이 장에서는 텍스트 데이터나 자연어 처리를 소재로 시계열 데이터를 다룰 때 자주 사용하는 순환 신경망(RNN, Recurrent Neural Netowrk)에 대해 설명한다.

RNN은 CNN 등과 달리 이전의 입력값을 기억해 두었다가 다음 출력에 반영할 수 있다. 따라서 문맥이나 이력, 흐름을 가미해서 향상된 시계열 분석을 할 수 있다. 시계열 데이터는 텍스트 데이터 이외에도 음성 데이터, 경제지표 데이터 등 다양한 분야에 등장하므로 **RNN**이 중요한 역할을 할 수 있다. 여기서는 RNN을 사용해 다음의 세 가지 문제를 해결하는 방법을 설명한다.

1. 문장 분류
2. 문장 생성
3. 기계 번역

RNN이란?

순환 구조 등 RNN의 기본 구조에 대해 설명한다.

RNN이 일반적인 신경망과 다른 점은 내부 상태를 저장하고 있다는 점이다. 특정 시점 t 의 입력 $x(t)$ 와 이전 시점의 내부 상태 $h(t-1)$ 을 입력하면 새로운 내부 상태 $h(t)$ 를 얻을 수 있으며, 이 $h(t)$ 를 다시 선형 계층 등에서 출력 $o(t)$ 로 변환한다. 이것이 전체적인 흐름이다.

$x(1)$ 에서 $x(t)$ 까지 이 과정을 반복하면 내부 상태도 $h(0)$ 부터 $h(1)$, $h(2)$, $h(3)$ 로 변경되며, 출력 $o(t)$ 도 현재 상태에 따라 변화해 간다. **그림 5.1**은 이 흐름을 시각화한 것이다.

가장 간단한 RNN인 Elman Network에선 $x(t)$ 와 $h(t-1)$ 에서 $h(t)$ 로 변환하는 부분이 선형 계층과 활성화 함수로 되어 있다. 그러면 **그림 5.1**처럼 RNN을 시간 방향으로 전개해 보면 신경망을 일직선으로 나열한 것으로, FeedForward형 신경망과 동일한 구조라는 것을 알 수 있다.

하지만 RNN은 일반 신경망보다 훈련이 어렵다. 오랜 시간 동안 쌓인 이력을 사용하려고 하면 그만큼 깊은 신경망이 되어야 하며, 경사 손실이나 경사 분실 등의 문제가 발생할 수 있다. 이것을 해결하기 위해 단순한 선형 계층이 아닌 더 정교한 처리를 모아 모듈 블록으로 바꾼 LSTM(Long Short Term Memory)나 GRU(Gated Recurrent Unit) 등의 RNN도 있다.

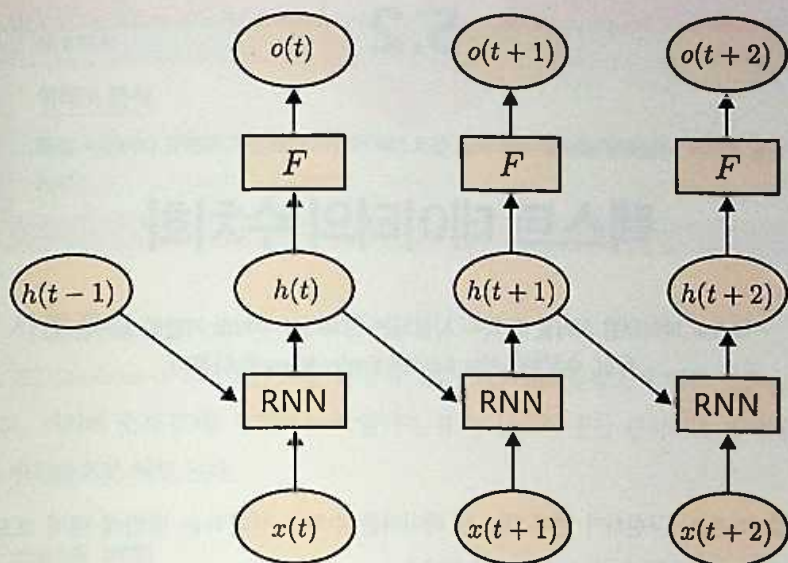


그림 5.1 RNN의 신경망 시각화. 새로운 내부 상태 $h(t)$ 는 이전 내부 상태 $h(t-1)$ 와 입력 $x(t)$ 에 의해 생성되며, $h(t)$ 는 다시 임의의 함수 F 에 의해 최종 출력 $o(t)$ 로 변환된다. 이 흐름이 계속 반복된다.

텍스트 데이터의 수치화

텍스트 데이터를 처리할 때 자주 사용되는 전처리나 수치화 기법을 소개한다.
특히, 수치화에서는 BoW와 Embedding을 다룬다.

실제로 RNN을 구현하기 전에 텍스트 데이터를 수치로 변환하는 방법에 대해 보도록 하겠다. 크게 다음 세 가지 단계로 구성된다.

1. 정규화와 토큰화
2. 사전(Dictionary) 구축
3. 수치로 변환

● 정규화와 토큰화

'1. 정규화(normalization)와 토큰화(tokenization)'에서는 문장을 특정 단위의 리스트로 분해한다. 예를 들어, 단어나 문자 등이 분할 단위가 될 수 있다. 단어 단위로 분할할 때는 영어 등의 유럽계 언어에서는 단순히 공백으로 나누면 되지만, 일본어나 중국어 등에서는 형태소 분석(예로 참고) 등의 처리가 필요한 경우도 있다.

토큰화와 동시에 표기 차이 등을 통일하는 정규화도 필요하다. 예를 들어, 대문자를 소문자로 통일시키거나, '한다', '하다'를 '하다'로 통일, isn't을 is not으로 통일시키는 처리가 필요하다.



MEMO

형태소 분석

특정 사전이나 문법에 기반해서 자연어 텍스트를 품사 등의 요소로 분해하는 처리를 일컫는다.

● 사전 구축

'2. 사전(Dictionary) 구축'이란, 모든 문장의 집합(Corpus)(**메모** 참고)에 대한 토큰을 수집하고, 거기에 숫자 ID를 부여하는 작업이다. ID는 단순히 등장 순서대로 부여해도 되고 빈도순으로 해도 된다.

● 수치로 변환

'3. 수치로 변환'은 토큰의 리스트로 분할된 문장을 2에서 구축한 사전을 사용해 ID 리스트로 변환하는 작업이다.



MEMO

코퍼스(corpus)

코퍼스란, 자연어 문장을 구조화해서 모은 것이다.

이 일련의 작업에서 하나의 긴 문자열이었던 문장이 최종적으로 수치 리스트로 변환된다. 이 수치 리스트를 다시 집계하고 각 ID의 등장 횟수를 벡터로 표현한 것이 **BoW** (Bags of Word)다. 예를 들면, 다음과 같이 구성된다.

(I, you, am, of, ...) = (1, 0, 1, 3, ...)

BoW는 계산이 간단하고 여러 문장을 모으면 희소 행렬(sparse matrix)로 표현할 수 있어서 매우 효율이 좋지만, 토큰 순서라는 중요한 정보를 잃는다는 문제점도 있다. 따라서

신경망에서는 **Embedding**이라는 기법으로 토큰을 벡터로 변환하고 벡터 데이터의 시계열로 문장을 처리하는 것이 주류를 이루고 있다.

파이토치에서는 `nn.Embedding`을 사용해서 Embedding 계층을 만들 수 있다. 예를 들어, 전체 10,000종류의 토큰이 있고, 이것을 20차원의 벡터로 표현하는 경우 **Embedding**처럼 기술한다.

리스트 5.1 전체 10,000종류의 토큰을 20차원 벡터로 표현하는 경우



```
emb = nn.Embedding(10000, 20, padding_idx=0)
# Embedding 계층의 입력은 int64 텐서
inp = torch.tensor([1, 2, 5, 2, 10], dtype=torch.int64)
# 출력은 float32 텐서
out = emb(inp)
```

`nn.Embedding`은 `padding_idx`를 지정하므로 ID가 모두 0인 벡터로 변환된다. 사전에 없는 토큰은 모든 ID를 0으로 하고, 실제 ID는 1부터 시작하도록 사용하면 좋다. 이때 토큰 종류는 0을 포함한 수를 `nn.Embedding`의 첫 번째 인수로 지정해야 한다.

참고로, `nn.Embedding`도 미분 가능하므로 이 내부의 가중치 파라미터도 신경망 전체 훈련 시에 최적화할 수 있으며, 사전에 학습된 `nn.Embedding`을 이용해서 다른 문제를 해결할 수도 있다(**메모** 참고).



MEMO

미리 학습 완료(Pre-trained Embedding)

이 책에선 다루고 있지 않지만, Word2Vec로 유명해진 Continuous-BOW나 Skip-Gram 등의 간단한 모델을 사용해서 미리 학습을 실시하는 경우가 많다.

RNN과 문장 분류

이 절에서는 문장 분류에 대해 설명한다. 예를 들어, 뉴스의 장르 분류나 리뷰가 좋은 리뷰인지 악성 리뷰인지를 분류할 때 응용할 수 있다. 참고로, 시계열 분류 문제는 일반적으로 **계열 레이블링**이라고도 불린다.

5.3.1 IMDb 리뷰 데이터

IMDb는 이 책 집필 시점(2018년 8월 현재)에 아마존(Amazon)에 의해 운영되고 있는 영화 및 TV 드라마 리뷰 사이트로, 각 리뷰는 0부터 10점까지의 점수가 부여된다. 미국 스탠퍼드 대학의 연구원들이 5만여 건의 리뷰를 추출해서 문장의 긍정/부정을 분석한 데이터를 온라인상에 공개하고 있다. 다음 URL을 통해 해당 데이터를 얻을 수 있다.

- Large Movie Review Dataset

URL http://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz

데이터를 다운로드해서 압축 해제하면 **그림 5.2**와 같은 디렉터리 구조를 볼 수 있다.

```

aclImdb/
├── imdb.vocab
├── train/
│   ├── pos/
│   ├── neg/
│   └── unsup/
└── test/
    ├── pos/
    └── neg/
  
```

그림 5.2 IMDb 리뷰 데이터의 디렉터리 구조(.feat나 .txt 확장자 파일은 생략)

imdb.vocab은 리뷰에 등장하는 모든 단어를 사전에 추출한 용어집이다. train/pos에는 훈련용 긍정적 리뷰 텍스트 파일이 다량으로 저장돼 있다.

이 데이터를 읽는 Dataset을 만들어 보자.



MEMO

컬래버레이터에서 파일 다운로드 및 해제

컬래버레이터에서는 다음 명령을 실행하면 된다.

```
!wget http://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
!tar xf aclImdb_v1.tar.gz
```

● 두 개의 함수 준비

먼저, **리스트 5.2**에 있는 두 가지 함수를 준비한다.

리스트 5.2 함수 작성



```
import glob
import pathlib
import re

remove_marks_regex = re.compile("[,\\.\\(\\)\\[\\]\\*:]|<.*?>")
shift_marks_regex = re.compile("([?!])")

def text2ids(text, vocab_dict):
    # !? 이외의 기호 삭제
    text = remove_marks_regex.sub("", text)
    # !?와 단어 사이에 공백 삽입
    text = shift_marks_regex.sub(r" \1 ", text)
    tokens = text.split()
    return [vocab_dict.get(token, 0) for token in tokens]

def list2tensor(token_idxes, max_len=100, padding=True):
    if len(token_idxes) > max_len:
        token_idxes = token_idxes[:max_len]
    n_tokens = len(token_idxes)
    if padding:
        token_idxes = token_idxes \
```



```

+ [0] * (max_len - len(token_idxes))
return torch.tensor(token_idxes, dtype=torch.int64), n_tokens

```

text2ids는 긴 문자열을 토큰 ID 리스트로 변환하는 함수다. 정규 표현을 사용해서 문장 부호나 괄호를 제거하고, !나 ?와 단어 사이에 공백을 넣어서 단어와 별도로 토큰으로 분할하고 있다. 이것은 imdb.vocab에 이 두 기호가 포함돼 있기 때문이다. 용어집에 포함되지 않는 토큰은 ID=0을 할당한다.

list2tensor는 ID 리스트를 int64의 텐서로 변환하는 함수다. 변환할 때는 각 문장을 분할한 후 토큰 수를 제한하고, 반대로 그 수에 미치지 못하는 경우에는 뒤를 0d로 채운다.

Dataset 클래스 작성

이 두 개의 함수를 사용해서 다음과 같이 Dataset 클래스를 만든다(**리스트 5.3**), 생성자 내에서 텍스트 파일의 경로와 레이블을 모은 튜플 리스트를 만들고, __getitem__ 내에서 이 파일을 읽어서 텐서로 변환하는 것이 핵심이다.

텐서는 max_len로 지정한 길이로 통일하고(padding하고) 있으므로 나중에 처리하기 용이하다. 또한, 0으로 채우기 전(padding하기 전)의 원래 길이와 n_tokens도 이후에 필요하므로 함께 반환하고 있다.

리스트 5.3 Dataset 클래스 작성

```

import torch
from torch import nn, optim
from torch.utils.data import (Dataset,
                               DataLoader,
                               TensorDataset)

import tqdm

```

 In

```

class IMDBDataset(Dataset):
    def __init__(self, dir_path, train=True,
                 max_len=100, padding=True):
        self.max_len = max_len

```

 In

```

self.padding = padding

path = pathlib.Path(dir_path)
vocab_path = path.joinpath("imdb.vocab")

# 용어집 파일을 읽어서 행 단위로 분할
self.vocab_array = vocab_path.open() \
    .read().strip().splitlines()
# 단어가 키이고 값이 ID인 dict 만들기
self.vocab_dict = dict((w, i+1) \
    for (i, w) in enumerate(self.vocab_array))

if train:
    target_path = path.joinpath("train")
else:
    target_path = path.joinpath("test")
pos_files = sorted(glob.glob(
    str(target_path.joinpath("pos/*.txt"))))
neg_files = sorted(glob.glob(
    str(target_path.joinpath("neg/*.txt"))))
# pos는 1, neg는 0인 label을 붙여서
# (file_path, label)의 튜플 리스트 작성
self.labeled_files = \
    list(zip([0]*len(neg_files), neg_files)) + \
    list(zip([1]*len(pos_files), pos_files))

@property
def vocab_size(self):
    return len(self.vocab_array)

def __len__(self):
    return len(self.labeled_files)

def __getitem__(self, idx):
    label, f = self.labeled_files[idx]
    # 파일의 텍스트 데이터를 읽어서 소문자로 변환
    data = open(f).read().lower()
    # 텍스트 데이터를 ID 리스트로 변환
    data = text2ids(data, self.vocab_dict)
    # ID 리스트를 텐서로 변환
    data, n_tokens = list2tensor(data, self.max_len, self.padding)
    return data, label, n_tokens

```

● 훈련용과 테스트용 DataLoader 작성

나머지는 이전 장에서처럼 훈련용과 테스트용 DataLoader를 만들면 된다(**리스트 5.4**).

리스트 5.4 훈련용과 테스트용 DataLoader 작성

```
train_data = IMDBDataset("<your_path>/aclImdb/")
test_data = IMDBDataset("<your_path>/aclImdb/", train=False)
train_loader = DataLoader(train_data, batch_size=32,
                           shuffle=True, num_workers=4)
test_loader = DataLoader(test_data, batch_size=32,
                          shuffle=False, num_workers=4)
```

 In

입력의 디렉터리 지정

입력의 디렉터리 지정

5.3.2 신경망 정의와 훈련

여기서 해결하고자 하는 문제는 '특정 정수의 시계열 X가 입력됐을 때에 0 또는 1이 출력되는 2진 분류 문제'다. 지금까지 설명한 것을 정리하면, 입력 X를 **Embedding**으로 벡터 시계열로 변환하고, 이것을 **RNN**에 넣어서 마지막 출력을 1차원 선형 계층과 연결하면 된다. **박스 5.5**가 신경망 정의다.

리스트 5.5 신경망 정의

```
class SequenceTaggingNet(nn.Module):
    def __init__(self, num_embeddings,
                  embedding_dim=50,
                  hidden_size=50,
                  num_layers=1,
                  dropout=0.2):
        super().__init__()
        self.emb = nn.Embedding(num_embeddings, embedding_dim,
                                padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim,
                              hidden_size, num_layers,
                              batch_first=True, dropout=dropout)
        self.linear = nn.Linear(hidden_size, 1)

    def forward(self, x, h0=None, l=None):
        # ID를 Embedding으로 다차원 벡터로 변환
        # x는 (batch_size, step_size)
        # -> (batch_size, step_size, embedding_dim)
```

 In

```

x = self.emb(x)
# 초기 상태 h0와 함께 RNN에 x를 전달
# x는 (batch_size, step_size, embedding_dim)
# -> (batch_size, step_size, hidden_dim)
x, h = self.lstm(x, h0)
# 마지막 단계만 추출
# x는 (batch_size, step_size, hidden_dim)
# -> (batch_size, 1)
if l is not None:
    # 입력의 원래 길이가 있으면 그것을 이용
    x = x[list(range(len(x))), l-1, :]
else:
    # 없으면 단순히 마지막 것을 이용
    x = x[:, -1, :]
# 추출한 마지막 단계를 선형 계층에 넣는다
x = self.linear(x)
# 불필요한 차원을 삭제
# (batch_size, 1) -> (batch_size, )
x = x.squeeze()
return x

```

파이토치에서는 Elman형 RNN은 nn.RNN, LSTM은 nn.LSTM, GRU는 nn.GRU로 사용할 수 있으며, 여기서는 nn.LSTM을 사용하고 있다. 이후부터는 이 세 가지 계층을 **파이토치의 RNN 계열**이라고 부르겠다. 파이토치의 RNN 계열 모두 여러 단계의 입력을 받아서 여러 단계의 출력과 마지막 내부 상태를 반환하도록 설계돼 있다.

5.1절에서 설명한 것처럼 입력을 한 단계씩 RNN 함수에 넣을 필요가 없다. 또한, 파이토치의 RNN 계열은 입력 차원, 중간층(내부 상태)의 차원 외에도 계층 수(num_layers), batch_first, dropout 등의 인수를 지정할 수 있다. 입력 차원은 Embedding의 출력 차원과 같다.

파이토치의 RNN 계열은 RNN을 여러 계층으로 연결할 수 있으며, 그 수를 num_layers로 지정한다. 이때 정규화로 마지막 RNN 계층 이외의 출력에는 dropout을 적용할 수 있어서 해당 확률의 파라미터로 dropout을 지정할 수 있다. batch_first는 입력 형식을 지정하는 옵션이다.

파이토치의 RNN 계열은 입출력의 차원이 (단계 수, 배치 수, 특이 수)순으로 기본 설정돼 있지만, batch_first=True를 지정하면 (배치 수, 단계 수, 특이 수)순으로 변경할 수

있다. 다른 신경망 층에서는 반드시 첫 번째 차원이 배치 수이므로 필자는 후자가 직감적으로 처리하기 쉽다고 본다.

forward 함수에는 입력 x 이외에 내부 상태의 초기값을 지정할 필요가 있지만, None을 지정하면 모든 값이 0인 벡터를 입력한 것과 같은 효과를 볼 수 있다. RNN의 출력 중 마지막 단계만 선형 계층에 전달해서 최종적으로 (배치 수, 1) 차원의 출력을 squeeze 메서드로 (배치 수,)처럼 해서 분류 대상이 두 개인 문제에서 사용하는 형태로 변환하고 있다.

마지막 단계를 추출하는 부분은 IMDBDataset이 반환하는 n_tokens 정보를 사용하며 Advanced-Indexing(☞참고)을 이용하고 있다.



MEMO

Advanced-Indexing

Advanced-Indexing에 대해서는 다음 포럼 등을 참고하자.

- PyTorch: Select tensor in a batch of sequences

URL <https://discuss.pytorch.org/t/select-tensor-in-a-batch-of-sequences/8613/2>

● 훈련 및 평가 작성

이후로는 앞 장과 마찬가지로 훈련 및 평가 코드를 작성하면 된다(☞리스트 5.6, ☞리스트 5.7). 분류 대상이 두 개이므로 손실 함수에는 ☞리스트 2.9와 똑같이 nn.BCEWithLogisticsLoss를 사용한다.

리스트 5.6 훈련 코드



```
def eval_net(net, data_loader, device="cpu"):
    net.eval()
    ys = []
    ypreds = []
    for x, y, l in data_loader:
        x = x.to(device)
        y = y.to(device)
```



```

l = l.to(device)
with torch.no_grad():
    y_pred = net(x, l=l)
    y_pred = (y_pred > 0).long()
    ys.append(y)
    ypreds.append(y_pred)
ys = torch.cat(ys)
ypreds = torch.cat(ypreds)
acc = (ys == ypreds).float().sum() / len(ys)
return acc.item()

```

리스트 5.7 평가 코드



```

from statistics import mean

# num_embeddings에는 0을 포함해서 train_data.vocab_size+1를 넣는다
net = SequenceTaggingNet(train_data.vocab_size+1, num_layers=2)
net.to("cuda:0")
opt = optim.Adam(net.parameters())
loss_f = nn.BCEWithLogitsLoss()

for epoch in range(10):
    losses = []
    net.train()
    for x, y, l in tqdm.tqdm(train_loader):
        x = x.to("cuda:0")
        y = y.to("cuda:0")
        l = l.to("cuda:0")
        y_pred = net(x, l=l)
        loss = loss_f(y_pred, y.float())
        net.zero_grad()
        loss.backward()
        opt.step()
        losses.append(loss.item())
    train_acc = eval_net(net, train_loader, "cuda:0")
    val_acc = eval_net(net, test_loader, "cuda:0")
    print(epoch, mean(losses), train_acc, val_acc)

```

```

100%|**| 782/782 [00:35<00:00, 22.07it/s]
0%| | 0/782 [00:00<?, ?it/s]0 0.680973151074651 ➔
0.5747199654579163 0.5752800107002258
100%|**| 782/782 [00:36<00:00, 21.31it/s]
0%| | 0/782 [00:00<?, ?it/s]1 0.6806320003841234 ➔
0.5991599559783936 0.5899199843406677
100%|**| 782/782 [00:36<00:00, 21.31it/s]
0%| | 0/782 [00:00<?, ?it/s]2 0.6770238574127407 ➔
0.6734799742698669 0.6436399817466736
100%|**| 782/782 [00:36<00:00, 21.17it/s]
0%| | 0/782 [00:00<?, ?it/s]3 0.5522478538210435 ➔
0.8149200081825256 0.7490800023078918
100%|**| 782/782 [00:36<00:00, 21.47it/s]
0%| | 0/782 [00:00<?, ?it/s]4 0.389272873530455 ➔
0.8837199807167053 0.7885199785232544
100%|**| 782/782 [00:36<00:00, 21.47it/s]
0%| | 0/782 [00:00<?, ?it/s]5 0.2973673498386617 ➔
0.9179999828338623 0.7927599549293518
100%|**| 782/782 [00:36<00:00, 21.41it/s]
0%| | 0/782 [00:00<?, ?it/s]6 0.2327607802932372 ➔
0.9450799822807312 0.794439971446991
100%|**| 782/782 [00:36<00:00, 21.33it/s]
0%| | 0/782 [00:00<?, ?it/s]7 0.18516844858551193 ➔
0.9618799686431885 0.7880399823188782
100%|**| 782/782 [00:36<00:00, 21.30it/s]
0%| | 0/782 [00:00<?, ?it/s]8 0.13803911597236915 ➔
0.9772799611091614 0.7853599786758423
100%|**| 782/782 [00:36<00:00, 21.32it/s]
9 0.09878007613022423 0.9833599925041199 0.7752799987792969

```

2계층 LSTM을 포함한 모델에서는 10회 정도 훈련하면 테스트 데이터에 대해 80% 정도 정확도를 보여 준다. 필자의 환경에서는 GPU를 사용하니 계산이 6배 정도 빨라졌다. 역시 GPU 사용을 권장한다.

● RNN을 사용하지 않은 모델 작성

여기서는 비교를 위해 **RNN을 사용하지 않은 모델**을 테스트해 보겠다. 이 예제에서는 **SVMLite 형식**이라는 ‘회소 행렬 + 레이블’ 형식의 BoW와 레이블 데이터를 포함하고 있다. 이것을 읽어서 로지스틱 회귀 모델로 학습해 보겠다. **scikit-learn**을 사용하면 **리스트** **5.8**처럼 쉽게 구현할 수 있다.

리스트 5.8 RNN을 사용하지 않는 모델 작성

 In

```
from sklearn.datasets import load_svmlight_file
from sklearn.linear_model import LogisticRegression

train_X, train_y = load_svmlight_file(
    "<your_path>/aclImdb/train/labeledBow.feats")
test_X, test_y = load_svmlight_file(
    "<your_path>/aclImdb/test/labeledBow.feats",
    n_features=train_X.shape[1])

model = LogisticRegression(C=0.1, max_iter=1000)
model.fit(train_X, train_y)
model.score(train_X, train_y), model.score(test_X, test_y)
```

* 출력에 시간이 걸린다

 Out

(0.8989200000000005, 0.39604)

훈련 데이터의 경우 어느 정도 정확도가 나오지만, 테스트 데이터의 결과는 터무니없을 정도로 낮다. 리뷰에는 같은 단어가 자주 나오므로 순서가 중요하다. 따라서 RNN을 이용해서 문맥을 고려한 모델을 구축해야 한다는 것을 알 수 있다.

5.3.3 가변 길이 계열 처리

지금까지는 서로 다른 길이의 계열을 패딩(고정)해서 길이를 맞추고, 마지막 출력으로부터 원래 길이 부분만 추출했지만, 파이토치에는 PackedSequence라는 가변 길이의 계열을 처리하기 위한 구조가 마련돼 있다.

파이토치의 RNN 계열은 PackedSequence를 입력으로 받으며, PackedSequence로 출력할 수도 있다. PackedSequence는 패딩된 텐서와 각 계열의 길이 리스트를 nn.utils.rnn.pack_padded_sequence 함수에 부여해서 작성할 수 있다. 또한, 이때 텐서와 길이 리스트는 길이순으로 정렬돼 있어야 한다.

PackedSequence를 사용한 모델은 자동으로 각 계열의 길이만큼 RNN을 계산한 후 중지한다. 출력은 PackedSequence와 함께 각 계열에서 멈춘 위치의 내부 상태를 반환한다. 이 성질을 이용한 모델이 [리스트 5.9](#)다.

리스트 5.9 PackedSequence 성질을 이용한 모델 작성



```
class SequenceTaggingNet2(SequenceTaggingNet):
```

```
    def forward(self, x, h0=None, l=None):
        # ID를 Embedding으로 다차원 벡터로 변환
        x = self.emb(x)

        # 길이가 주어진 경우 PckedSequence 만들기
        if l is not None:
            x = nn.utils.rnn.pack_padded_sequence(
                x, l, batch_first=True)

        # RNN에 입력
        x, h = self.lstm(x, h0)

        # 마지막 단계를 추출해서 선형 계층에 넣는다
        if l is not None:
            # 길이 정보가 있으면 마지막 계층의
            # 내부 상태 벡터를 직접 이용할 수 있다
            # LSTM는 보통 내부 상태 외에 불럭 셀 상태도
            # 가지고 있으므로 내부 상태만 사용한다
            hidden_state, cell_state = h
            x = hidden_state[-1]
        else:
            x = x[:, -1, :]

        # 선형 계층에 넣는다
        x = self.linear(x).squeeze()
        return x
```

훈련 부분은 **리스트 5.10** 과 같이 정렬 처리가 들어가지만, 다른 부분은 큰 변화가 없다.
평가 함수 eval_net도 마찬가지로 정렬 처리를 추가했다.

리스트 5.10 훈련 코드



```
for epoch in range(10):
    losses = []
    net.train()
    for x, y, l in tqdm.tqdm(train_loader):
        # 길이 배열을 길이 순으로 정렬
        l, sort_idx = torch.sort(l, descending=True)
        # 얻은 인덱스를 사용해서 x,y도 정렬

        x = x[sort_idx]
```



```

y = y[sort_idx]

x = x.to("cuda:0")
y = y.to("cuda:0")

y_pred = net(x, l=l)
loss = loss_f(y_pred, y.float())
net.zero_grad()
loss.backward()
opt.step()
losses.append(loss.item())

train_acc = eval_net(net, train_loader, "cuda:0")
val_acc = eval_net(net, test_loader, "cuda:0")
print(epoch, mean(losses), train_acc, val_acc)

```

 Out

```

100%|**| 782/782 [00:36<00:00, 21.65it/s]
0%| | 0/782 [00:00<?, ?it/s]0 0.07515874557802096 ➡
0.9892799854278564 0.7767199873924255
100%|**| 782/782 [00:37<00:00, 20.94it/s]
0%| | 0/782 [00:00<?, ?it/s]1 0.060927915427347885 ➡
0.991159975528717 0.7741599678993225
100%|**| 782/782 [00:37<00:00, 21.03it/s]
0%| | 0/782 [00:00<?, ?it/s]2 0.05283435225270003 ➡
0.9917999505996704 0.7780799865722656
100%|**| 782/782 [00:36<00:00, 21.39it/s]
0%| | 0/782 [00:00<?, ?it/s]3 0.0423438507409128 ➡
0.9941999912261963 0.7736799716949463
100%|**| 782/782 [00:37<00:00, 21.03it/s]
0%| | 0/782 [00:00<?, ?it/s]4 0.035124090445094534 ➡
0.994879961013794 0.7717199921607971
100%|**| 782/782 [00:37<00:00, 21.04it/s]
0%| | 0/782 [00:00<?, ?it/s]5 0.03184197584891816 ➡
0.9949599504470825 0.7683199644088745
100%|**| 782/782 [00:37<00:00, 21.01it/s]
0%| | 0/782 [00:00<?, ?it/s]6 0.030963344712351043 ➡
0.9894399642944336 0.7593599557876587
100%|**| 782/782 [00:37<00:00, 21.07it/s]
0%| | 0/782 [00:00<?, ?it/s]7 0.02651888659522247 ➡
0.9898799657821655 0.7690799832344055
100%|**| 782/782 [00:37<00:00, 20.93it/s]
0%| | 0/782 [00:00<?, ?it/s]8 0.02357390662000212 ➡
0.9969199895858765 0.7667199969291687
100%|**| 782/782 [00:37<00:00, 20.88it/s]
9 0.02062533009702654 0.9973199963569641 0.7696399688720703

```


RNN을 사용한 문장 생성

RNN을 응용하면 흥미로운 결과를 얻을 수 있다. 여기서는 단순한 회귀나 분류가 아니라 학습된 모델을 기반으로 새로운 문장을 생성하는 방법을 소개한다.

RNN을 사용하면 문장도 생성할 수 있다. 5.3절에서는 단어 단위의 모델을 통해 문장을 분류했지만, 여기서는 문장 단위의 모델로 셰익스피어의 문장을 학습해서 비슷한 문체를 생성하는 예를 다룬다. 이 내용은 Practical PyTorch(메모 참고)라는 튜토리얼을 기반으로 코드를 더 간략하게 정리한 것이다. 관심 있는 독자는 원본 튜토리얼의 내용도 참고하도록 하자.

여기서 설명하는 것은 5.5절에서 다루는 Encoder-Decoder 모델 기반 기계 번역에도 사용할 수 있는 것으로 RNN의 기본적인 사용 패턴 중 하나다.



MEMO

Practical PyTorch

단어 단위로 문장을 학습하는 모델을 소개한 자습서다.

- `spro/practical-pytorch`

URL <https://github.com/spro/practical-pytorch>

5.4.1 데이터 준비

여기서는 문장 단위로 모델을 작성하기 위해 **리스트 5.11**과 같이 어휘 사전과 두 개의 변환 함수를 만들도록 한다.

리스트 5.11 어휘 사전과 두 개의 변환 함수



```
# 모든 ascii 문자로 사전 만들기
import string
all_chars = string.printable
vocab_size = len(all_chars)
vocab_dict = dict((c, i) for (i, c) in enumerate(all_chars))

# 문자열을 수치 리스트로 변환하는 함수
def str2ints(s, vocab_dict):
    return [vocab_dict[c] for c in s]

# 수치 리스트를 문자열로 변환하는 함수
def ints2str(x, vocab_array):
    return "".join([vocab_array[i] for i in x])
```

● 텍스트 파일 다운로드

다음은 아래 URL에서 셰익스피어의 다양한 대본을 하나로 모아 놓은 텍스트 파일 (input.txt)(**메모** 참고)을 다운로드한다. 'tinyshakespeare.txt'라는 이름으로 저장하자.

- char-rnn/data/tinyshakespeare/input.txt

URL <https://github.com/karpathy/char-rnn/tree/master/data/tinyshakespeare>



MEMO

컬래버레이터에서 파일 업로드하기

위 예제 파일은 컬래버레이터로 바로 다운로드할 수 없다. 일단 본인 PC로 다운로드한 후에 다음 방법으로 컬래버레이터에 업로드하도록 하자.

```
from google.colab import files
# 창이 뜨면 파일을 선택해서 업로드한다
uploaded = files.upload()
```



char-rnn/data/tinysakespeare/input.txt

이 데이터를 만들어서 공개해 준 Andrej Karpathy 씨에게 감사를 드린다.

● Dataset 클래스 작성

이 거대한 텍스트 파일을 각 200문자로 구성된 문장으로 분할하도록 Dataset 클래스 (리스트 5.12)를 정의한다. 텐서의 split 메서드를 사용하면 지정한 길이로 분할된 텐서의 튜플(tuple)을 얻을 수 있다. 단, 마지막 하나는 길이가 모자랄 가능성이 있으므로 확인해서 버린다.

리스트 5.12 분할을 위한 Dataset 클래스 작성



```
import torch
from torch import nn, optim
from torch.utils.data import (Dataset,
                               DataLoader,
                               TensorDataset)

import tqdm
```



```
class ShakespeareDataset(Dataset):
    def __init__(self, path, chunk_size=200):
        # 파일을 읽어서 수치 리스트로 변환
        data = str2ints(open(path).read().strip(), vocab_dict)

        # 텐서로 변환해서 split 한다
        data = torch.tensor(data, dtype=torch.int64).split(chunk_size)

        # 마지막 덩어리(chunk)의 길이를 확인해서 부족한 경우 버린다
        if len(data[-1]) < chunk_size:
            data = data[:-1]

        self.data = data
        self.n_chunks = len(self.data)

    def __len__(self):
        return self.n_chunks
```

```
def __getitem__(self, idx):
    return self.data[idx]
```

리스트 5.12에서 작성한 Dataset 클래스를 사용해서 DataLoader를 만든다(**리스트 5.13**). 이것으로 데이터 준비가 완료된다.

리스트 5.13 Dataset 클래스를 사용해서 DataLoader 만들기



```
ds = ShakespeareDataset("<your_path>/tinyshakespeare.txt",
                        chunk_size=200)
loader = DataLoader(ds, batch_size=32, shuffle=True,
                   num_workers=4)
```

5.4.2 모델 정의 및 학습

● 문장 생성 모델 구축

문장 생성 모델 구축은 현재 문자를 x , 다음 문자를 y 라고 하면 x 로부터 y 를 예측하는 다중 식별 문제라고 볼 수 있다(**리스트 5.14**). 따라서 모델의 신경망 정의는 기본적으로 5.3절의 것과 똑같다. 단, 마지막 선형 계층의 출력 x 가 어휘 수가 된다.

리스트 5.14 문장 생성 모델 정의



```
class SequenceGenerationNet(nn.Module):
    def __init__(self, num_embeddings,
                  embedding_dim=50,
                  hidden_size=50,
                  num_layers=1, dropout=0.2):
        super().__init__()
        self.emb = nn.Embedding(num_embeddings, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim,
                             hidden_size,
                             num_layers,
                             batch_first=True,
                             dropout=dropout)
        # Linear의 output 크기는 첫 Embedding의
        # input 크기와 같은 num_embeddings
        self.linear = nn.Linear(hidden_size, num_embeddings)
```

```
def forward(self, x, h0=None):
    x = self.emb(x)
    x, h = self.lstm(x, h0)
    x = self.linear(x)
    return x, h
```

● 문장을 생성하는 함수 작성

리스트 5.14의 문장 생성 모델을 사용해서 실제로 문장을 생성하는 함수를 작성한다 (**리스트 5.15**).

리스트 5.15 문장을 생성하는 함수 작성

📄 In

```
def generate_seq(net, start_phrase="The King said ",
                 length=200, temperature=0.8, device="cpu"):
    # 모델을 평가 모드로 설정
    net.eval()
    # 출력 수치를 저장할 리스트
    result = []

    # 시작 문자열을 텐서로 변환
    start_tensor = torch.tensor(
        str2ints(start_phrase, vocab_dict),
        dtype=torch.int64
    ).to(device)
    # 선두에 batch 차원을 붙인다
    x0 = start_tensor.unsqueeze(0)
    # RNN을 통해서 출력과 새로운 내부 상태를 얻는다
    o, h = net(x0)
    # 출력을 정규화돼 있지 않은 확률로 변환
    out_dist = o[:, -1].view(-1).exp()
    # 확률로부터 실제 문자의 인덱스를 샘플링
    top_i = torch.multinomial(out_dist, 1)[0]
    # 결과 저장
    result.append(top_i)

    # 생성된 결과를 차례로 RNN에 넣는다
    for i in range(length):
        inp = torch.tensor([[top_i]], dtype=torch.int64)
        inp = inp.to(device)
        o, h = net(inp, h)
        out_dist = o.view(-1).exp()
        top_i = torch.multinomial(out_dist, 1)[0]
```



```
result.append(top_i)
```

```
# 시작 문자열과 생성된 문자열을 모아서 반환
```

```
return start_phrase + ints2str(result, all_chars)
```

이 함수는 첫 문자열을 받아서 다음 문장을 생성한다. 먼저, 시작 문자열을 텐서로 변환해서 RNN에 넣고, 예측한 출력과 새로운 내부 상태를 얻는다. 그리고 이 출력을 새로운 입력값으로 사용해서 새롭게 얻은 내부 상태와 함께 RNN에 넣는다. 이 과정을 반복한다.

또한, RNN의 출력은 선형 계층의 출력 그대로이므로 이것을 문자(의 인덱스)로 변환해야 한다. 단순히 max의 위치를 사용해 버리면 비슷한 문자만 생성되므로 확률로 변환해서 다항 분포를 이용해 샘플링한다(torch.multinomial).

확률로 변환할 때는 선형 계층의 출력을 softmax 함수에 넣어야 하지만, torch.multinomial은 입력의 확률 파라미터가 정규화(Normalized)([표 5.15](#) 참고)되지 않아도 괜찮으므로 지수 함수에 넣고 있다. 다음 식이 softmax 함수와 지수 함수의 관계를 보여준다. 지수 함수를 통과하면 정규화되지 않은 확률이 나온다는 것을 알 수 있다.

$$\text{softmax}(\mathbf{a})_i = \frac{\exp(a_i)}{\sum_j \exp(a_j)}$$



MEMO

정규화(Normalized)

여기서는 더해서 1이 되는 것을 뜻한다.

● 모델 훈련

모델 훈련은 [리스트 5.16](#)과 같이 한다. 핵심은 다음 두 가지다.

1. x는 각 chunk 문장의 첫 번째부터 마지막 문자의 하나 앞 문자까지를 이용하고, y는 두 번째부터 마지막 문자까지를 이용한다([그림 5.3](#)).

2. `nn.CrossEntropyLoss`는 $2D(\text{batch}, \text{dim})$ 의 예측값이고 $1D(\text{batch})$ 의 레이블 데이터만 사용할 수 있지만, 이 RNN 모델에서는 예측값이 $3D(\text{batch}, \text{step}, \text{dim})$ 이고 레이블이 $2D(\text{batch}, \text{step})$ 이므로 `view`를 이용해서 `batch`와 `step`을 통합해서 전달한다.

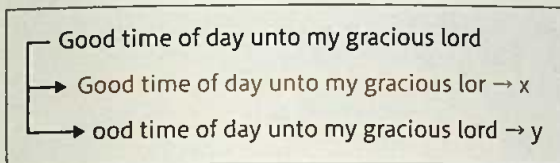


그림 5.3 문장을 x와 y로 분할

리스트 5.16 문장을 생성하는 함수 작성

```
from statistics import mean

net = SequenceGenerationNet(vocab_size, 20, 50,
                             num_layers=2, dropout=0.1)
net.to("cuda:0")
opt = optim.Adam(net.parameters())
# 다중 식별 문제이므로 SoftmaxCrossEntropyLoss가 손실 함수가 된다
loss_f = nn.CrossEntropyLoss()

for epoch in range(50):
    net.train()
    losses = []
    for data in tqdm.tqdm(loader):
        # x는 처음부터 마지막의 하나 앞 문자까지
        x = data[:, :-1]
        # y는 두 번째부터 마지막 문자까지
        y = data[:, 1:]

        x = x.to("cuda:0")
        y = y.to("cuda:0")

        y_pred, _ = net(x)
        # batch와 step 축을 통합해서 CrossEntropyLoss에 전달
        loss = loss_f(y_pred.view(-1, vocab_size), y.view(-1))
        net.zero_grad()
        loss.backward()
        opt.step()
        losses.append(loss.item())
    # 현재 손실 함수와 생성된 문장 예 표시
```

```
print(epoch, mean(losses))
with torch.no_grad():
    print(generate_seq(net, device="cuda:0"))
```

↑ Out

```
100%|**| 175/175 [00:17<00:00, 9.82it/s]
0 3.4523950699397496
0%|          | 0/175 [00:00<?, ?it/s]The King said
mytcumese,iNwcio;mwsd
'a dtr ewooap udr oy eadhAsoi gen
R Ri ne n h NOuhc,ul rn utila im
l
erts aeIEon
cntrdcpo linwSdei owatus r u,g ao n: bihe eranne tnt
obatschtyf n e deyyeetr aain roh f yedi s

(…중략…)

100%|**| 175/175 [00:10<00:00, 16.88it/s]
48 1.6563731929234096
0%|          | 0/175 [00:00<?, ?it/s]
The King said clemman theur beeting you both;
Sepfiness this know where some me turted bring; and but ere.

COLOUNEN EPIZE:
The lices, and my sive, you will behobe?

BENVOLIO:
And should firth him fall and returnts,
```

50회까지 학습하면 **리스트 5.16**의 Out처럼 문장이 생성된다. 이와 같이 문자 단위의 모델에서는 영어와 비슷한 단어를 생성할 수 있지만, 더 넓은 범위의 문맥을 고려해서 단어를 나열하는 것은 제대로 되지 않는 것을 알 수 있다.

인코더-디코더 모델을 사용한 기계 번역

마지막은 인코더-디코더(Encoder-Decoder)라는 두 개의 신경망으로 구성된 모델을 사용해서 기계 번역을 해보도록 한다.

인코더-디코더 모델은 임의의 두 대상이 있는 경우 한쪽을 사용해서 다른 한쪽을 생성할 수 있는 모델이다. 예를 들어, 영어와 프랑스어 문장이 있으면 영어를 프랑스어로 번역하는 모델을 만들 수 있다. 또는, 질문과 답이라는 두 대상이 있으면 자동 질의응답 시스템을 만들 수 있다.

뿐만 아니라 CNN과 조합하면 이미지로부터 설명문을 생성할 수 있어서 답러닝과 신경망에서 가장 주목받는 모델 중 하나라고 할 수 있다.

여기서는 영어를 스페인어로 번역하는 모델을 만들도록 한다. 스페인어는 영어처럼 유럽계 언어이며, 전처리가 용이하다. 또한, 데이터도 비교적 풍부하므로 스페인어(Spanish)([메모](#) 참고)를 선택했다.



MEMO

Spanish

필자가 스페인어를 공부하고 있다는 개인적인 이유도 있다.

5.5.1 인코더-디코더 모델이란

인코더-디코더 모델은 자유도가 높은 모델로 그 내용도 광범위하지만, 대략적인 동작을 정리해 보면 다음과 같다.

1. 번역 소스(번역할 데이터, src) 데이터를 인코더에 입력해서 특이량 벡터를 얻는다.
2. 특이량 벡터를 디코더에 입력해서 번역 타겟(번역 대상 데이터, trg) 데이터를 얻는다.

예를 들어, 5.3절에서 본 RNN을 이용한 분류 모델의 마지막 출력 계층을 임의 차원의 선형 계층으로 바꾼 것이 인코더에 해당한다. 그리고 5.4절에서 문장 생성 시에 사용한 RNN이나 4장에서 다룬 DCGAN의 Generator가 디코더에 해당된다.

● RNN을 사용한 인코더-디코더 모델

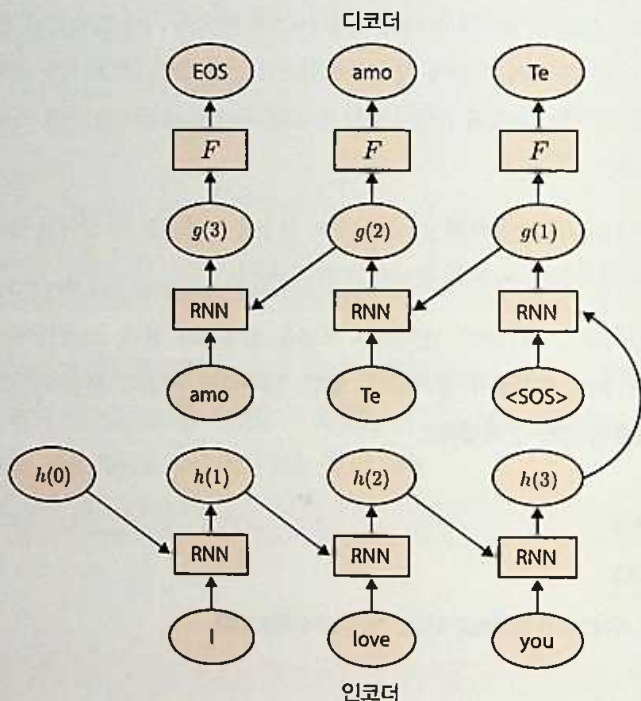


그림 5.4 인코더-디코더 모델. 하단이 인코더이고 상단이 디코더

여기서는 기계 번역을 하기 위해 가장 간단한 RNN 모델을 적용한 인코더-디코더 모델을 보도록 하겠다.

인코더는 소스 문장을 읽어서 마지막 단계의 내부 상태를 특이량 벡터 또는 컨텍스트(context)로 출력한다. 디코더는 이 컨텍스트를 초기 내부 상태로 하며, 시작 태그라는 특수한 입력과 함께 다음 단어와 새로운 내부 상태를 출력한다.

이 두 개의 출력을 다시 입력으로 사용해서 디코더에 넣는 작업을 반복한다.

그림 5.4처럼 인코더에 문장을 넣고 마지막 내부 상태와 시작 태그(<SOS>)를 디코더의 초깃값으로 넣어서 번역문을 생성한다.

5.5.2 데이터 준비

영어와 스페인어 번역 데이터는 Tatoeba.org라는 온라인 번역 데이터베이스 프로젝트가 공개하고 있는 데이터를 사용한다.

- Tatoeba.org

URL <https://tatoeba.org/eng/downloads>

추가로 이 데이터에 탭(tab) 구분 문자를 적용해 사용하기 쉽게 만든 것도 공개돼 있다. 이 책에서도 탭을 구분 문자로 사용한다.

‘Tab-delimited Bilingual Sentence Pairs’라는 사이트의 아래쪽에 있는 spa-eng.zip 파일을 다운로드한다. 압축 해제하면 spa.txt라는 파일이 있으므로 임의의 디렉터리에 복사한다.

- Tab-delimited Bilingual Sentence Pairs

URL <http://www.manythings.org/anki/>



MEMO

컬래버레이터에서 압축 파일 해제

컬래버레이터에서는 다음 명령을 실행하면 된다.

```
!wget http://www.manythings.org/anki/spa-eng.zip
!unzip spa-eng.zip
```

● 보조 함수 작성

다음은 Dataset을 만든다. 먼저, 몇 개의 보조 함수를 만들어야 한다(리스트 5.17).

리스트 5.17 보조 함수 작성



```
import torch
from torch import nn, optim
from torch.utils.data import (Dataset,
                               DataLoader,
                               TensorDataset)

import tqdm
```



```
import re
import collections
import itertools

remove_marks_regex = re.compile(
    "[\, \(\) \[ \] \* ; ; | < . * ? > ")
shift_marks_regex = re.compile("( [?! \. ] ")

unk = 0
sos = 1
eos = 2

def normalize(text):
    text = text.lower()
    # 불필요한 문자 제거
    text = remove_marks_regex.sub("", text)
    # ?!.와 단어 사이에 공백 삽입
    text = shift_marks_regex.sub(r" \1", text)
    return text
```

```

def parse_line(line):
    line = normalize(line.strip())
    # 번역 소스(src)와 번역 타겟(trg) 각각의 토큰을 리스트로 만든다
    src, trg = line.split("\t")
    src_tokens = src.strip().split()
    trg_tokens = trg.strip().split()
    return src_tokens, trg_tokens

def build_vocab(tokens):
    # 파일 안의 모든 문장에서 토큰의 등장 횟수를 확인
    counts = collections.Counter(tokens)
    # 토큰의 등장 횟수를 많은 순으로 나열
    sorted_counts = sorted(counts.items(),
                           key=lambda c: c[1], reverse=True)
    # 세 개의 태그를 추가해서 정방향 리스트와 역방향 용어집 만들기
    word_list = ["<UNK>", "<SOS>", "<EOS>"] \
        + [x[0] for x in sorted_counts]
    word_dict = dict((w, i) for i, w in enumerate(word_list))
    return word_list, word_dict

def words2tensor(words, word_dict, max_len, padding=0):
    # 끝에 종료 태그를 붙임
    words = words + ["<EOS>"]
    # 사전을 이용해서 수치 리스트로 변환
    words = [word_dict.get(w, 0) for w in words]
    seq_len = len(words)
    # 길이가 max_len 이하이면 패딩한다
    if seq_len < max_len + 1:
        words = words + [padding] * (max_len + 1 - seq_len)
    # 텐서로 변환해서 반환
    return torch.tensor(words, dtype=torch.int64), seq_len

```

normalize는 모두 소문자로 바꾼 후에 불필요한 문자를 제거하고, !?와 단어를 분할한다. 특히, 스페인어는 감탄문이나 의문문 앞에 ¡나 ¿처럼 거꾸로 된 기호를 붙이지만, 단순화를 하기 위해 이들 문자를 제거하고 영어처럼 문장 끝 기호만 남겨 두었다.

parse_line은 spa.txt의 첫 번째 줄을 영어와 스페인어 각각의 토큰 리스트로 변환하는 함수다. build_vocab는 어휘집을 만들기 위한 함수다. 영어와 스페인어 각각에 대해 이 파일에 포함돼 있는 단어 외에도 패딩, 시작, 종료 등 세 가지 태그를 추가해 둔다.

words2tensor는 단어 리스트를 텐서로 변환하는 함수다. 최대 길이를 지정하고 부족한 부분은 패딩한다.

○ TranslationPairDataset 클래스 작성

리스트 5.17에서 작성한 보조 함수를 사용해서 **리스트 5.18**과 같이 TranslationPair Database를 만든다. 이 클래스는 파일을 읽어서 번역 타겟(번역문)과 번역 소스(원문) 각각의 어휘 집과 텐서 리스트를 작성한다. 또한, 단어 수가 많은 문장은 학습이 어려우므로 사용하지 않게 설정했다.

참고로, 번역 타겟의 텐서는 -100으로 패딩하고 있다. 이것은 파이토치의 Cross EncryptionLoss가 기본으로 -100 레이블을 손실 함수에 포함하지 않게 되어 있어서 가변 길이의 제열 처리가 쉬워지기 때문이다.

리스트 5.18 TranslationPairDatabase 클래스 작성



```
class TranslationPairDataset(Dataset):
    def __init__(self, path, max_len=15):
        # 단어 수가 많은 문장을 걸러내는 함수
        def filter_pair(p):
            return not (len(p[0]) > max_len
                        or len(p[1]) > max_len)
        # 파일을 열어서, 파스 및 필터링
        with open(path) as fp:
            pairs = map(parse_line, fp)
            pairs = filter(filter_pair, pairs)
            pairs = list(pairs)
        # 문장의 소스와 타겟으로 나눔
        src = [p[0] for p in pairs]
        trg = [p[1] for p in pairs]
        # 각각의 어휘집 작성
        self.src_word_list, self.src_word_dict = \
            build_vocab(itertools.chain.from_iterable(src))
        self.trg_word_list, self.trg_word_dict = \
            build_vocab(itertools.chain.from_iterable(trg))
        # 어휘집을 사용해서 텐서로 변환
        self.src_data = [words2tensor(
            words, self.src_word_dict, max_len)
            for words in src]
        self.trg_data = [words2tensor(
            words, self.trg_word_dict, max_len, -100)
            for words in trg]

    def __len__(self):
        return len(self.src_data)
```

```
def __getitem__(self, idx):
    src, lsrc = self.src_data[idx]
    trg, ltrg = self.trg_data[idx]
    return src, lsrc, trg, ltrg
```

여기서는 최대 10개의 단어로 구성된 문장만 처리한다. **리스트 5.19**에서 Dataset과 DataLoader를 작성하면 데이터 준비가 끝난다.

리스트 5.19 Dataset과 DataLoader 작성



```
batch_size = 64
max_len = 10
path = "<your_path>/spa.txt"
ds = TranslationPairDataset(path, max_len=max_len)
loader = DataLoader(ds, batch_size=batch_size, shuffle=True,
                    num_workers=4)
```

원문의 디코딩 및 평가

5.5.3 파이토치를 사용한 인코더-디코더 모델

인코더와 디코더는 각각 5.3절과 5.4절에서 사용한 것과 많이 다르지 않다.

● 인코더 작성

먼저, 인코더는 **리스트 5.20**과 같다.

리스트 5.20 인코더 작성



```
class Encoder(nn.Module):
    def __init__(self, num_embeddings,
                  embedding_dim=50,
                  hidden_size=50,
                  num_layers=1,
                  dropout=0.2):
        super().__init__()
        self.emb = nn.Embedding(num_embeddings, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim,
                             hidden_size, num_layers,
                             batch_first=True, dropout=dropout)
```



```
def forward(self, x, h0=None, l=None):
    x = self.emb(x)
    if l is not None:
        x = nn.utils.rnn.pack_padded_sequence(
            x, l, batch_first=True)
    _, h = self.lstm(x, h0)
    return h
```

간단한 인코더-디코더 모델에서는 내부 상태만 디코더에 전달하므로 출력값은 무시한다(보통은 `o, h = self.lstm(x, h0)`이라고 하지만, 출력 `o`는 사용하지 않으므로 `_, h = ...` 형태를 하고 있다).

● 디코더 작성

다음은 디코더다(리스트 5.21).

리스트 5.21 디코더 작성



```
class Decoder(nn.Module):
    def __init__(self, num_embeddings,
                  embedding_dim=50,
                  hidden_size=50,
                  num_layers=1,
                  dropout=0.2):
        super().__init__()
        self.emb = nn.Embedding(num_embeddings, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_size,
                             num_layers, batch_first=True,
                             dropout=dropout)
        self.linear = nn.Linear(hidden_size, num_embeddings)

    def forward(self, x, h, l=None):
        x = self.emb(x)
        if l is not None:
            x = nn.utils.rnn.pack_padded_sequence(
                x, l, batch_first=True)
        x, h = self.lstm(x, h)
        if l is not None:
            x = nn.utils.rnn.pad_packed_sequence(x, batch_first=True,
                                                  padding_value=0)[0]
        x = self.linear(x)
        return x, h
```

디코더는 5.4절의 문장 생성 RNN과 거의 같다. 5.4절에서는 내부 상태의 초깃값에 모든 값이 0인 것을 사용했고, 입력 초깃값으로는 자신이 정한 문자열을 사용했다. 여기서는 내부 상태의 초깃값은 마지막 내부 상태를, 입력의 초깃값은 시작 토큰인 <SOS>를 사용한다는 것이 다르다.

● 번역 함수 작성

모델 정의를 마쳤으니 이것을 사용해서 실제로 번역하는 함수를 만들어 보자(**리스트 5.22**). 먼저, 번역할 문장(input_str)을 초기화해서 텐서로 변환하고, 시작 토큰 등 필요한 설정도 준비해 둔다. 초기화한 번역 소스의 텐서를 인코더에 넣어서 콘텍스트(내부 상태)를 생성하고, 이것을 시작 토큰과 함께 초깃값으로 디코더에 넣는다.

디코더의 출력은 다음 처리의 입력값이 되므로 이것을 반복한다. 출력은 기록해 두고, 마지막으로 수치를 번역 타깃(번역 결과물)의 문자열로 변환한다.

리스트 5.22 번역 함수 작성

↳ In

```
def translate(input_str, enc, dec, max_len=15, device="cpu"):
    # 입력 문자열을 수치화해서 텐서로 변환
    words = normalize(input_str).split()
    input_tensor, seq_len = words2tensor(words,
        ds.src_word_dict, max_len=max_len)
    input_tensor = input_tensor.unsqueeze(0)
    # 인코더에서 사용하므로 입력값의 길이도 리스트로 만들어 둔다
    seq_len = [seq_len]
    # 시작 토큰 준비
    sos_inputs = torch.tensor(sos, dtype=torch.int64)
    input_tensor = input_tensor.to(device)
    sos_inputs = sos_inputs.to(device)
    # 입력 문자열을 인코더에 넣어서 콘텍스트 얻기
    ctx = enc(input_tensor, l=seq_len)
    # 시작 토큰과 콘텍스트를 디코더의 초깃값으로 설정
    z = sos_inputs
    h = ctx
    results = []
    for i in range(max_len):
        # Decoder로 다음 단어 예측
        o, h = dec(z.view(1, 1), h)
        # 선형 계층의 출력이 가장 큰 위치가 다음 단어의 ID
        wi = o.detach().view(-1).max(0)[1]
```

```

if wi.item() == eos:
    break
results.append(wi.item())
# 다음 입력값으로 현재 출력 ID를 사용
z = wi
# 기록해 둔 출력 ID를 문자열로 변환
return " ".join(ds.trg_word_list[i] for i in results)

```

● 함수 동작 확인

작성한 함수를 사용해 보자(리스트 5.23). 아직 학습되지 않은 상태이므로 이상한 문장이 생성되지만, 실행은 문제없이 된다.

리스트 5.23 함수 동작 확인



```

enc = Encoder(len(ds.src_word_list), 100, 100, 2)
dec = Decoder(len(ds.trg_word_list), 100, 100, 2)
translate("I am a student.", enc, dec)

```



```

'utilizada calificaciones federal pillas miramos mecanográfico guiteau
toco incomoda idénticos avaricia comportamiento traducido garaje unánime'

```

● 모델 학습

드디어 학습에 들어간다. 이것도 매우 오랜 시간이 걸리는 계산이므로 GPU를 사용할 수 있는 환경이라면 꼭 사용하도록 하자. 먼저, 모델을 신규로 만들고, 최적화기와 실 손 함수를 준비한다. 최적화기의 파라미터는 필자가 여러 번 테스트해서 가장 좋은 것을 사용했다(리스트 5.24).

리스트 5.24 최적화기의 파라미터



```

enc = Encoder(len(ds.src_word_list), 100, 100, 2)
dec = Decoder(len(ds.trg_word_list), 100, 100, 2)
enc.to("cuda:0")
dec.to("cuda:0")
opt_enc = optim.Adam(enc.parameters(), 0.002)
opt_dec = optim.Adam(dec.parameters(), 0.002)
loss_f = nn.CrossEntropyLoss()

```

학습 부분은 **리스트 5.25**와 같다. 손실 함수는 enc의 출력을 초기 상태로 하고, 스페인어 문장의 다음 단어를 예측하고 있다. 번역 결과 문장의 교사 데이터는 각각 길이가 다르지만, -100으로 패딩하고 있어서 쉽게 미니 배치 처리를 할 수 있다.

리스트 5.25 모델의 학습 부분(손실 함수 등)



```
from statistics import mean

def to2D(x):
    shapes = x.shape
    return x.reshape(shapes[0] * shapes[1], -1)

for epoc in range(30):
    # 신경망을 훈련 모드로 설정
    enc.train(), dec.train()
    losses = []
    for x, lx, y, ly in tqdm.tqdm(loader):
        # x의 PackedSequence를 만들기 위해 번역 소스의 길이로 내림차순 정렬한다
        lx, sort_idx = lx.sort(descending=True)
        x, y, ly = x[sort_idx], y[sort_idx], ly[sort_idx]
        x, y = x.to("cuda:0"), y.to("cuda:0")
        # 번역 소스를 인코더에 넣어서 콘텍스트를 얻는다
        ctx = enc(x, l=lx)

        # y의 PackedSequence를 만들기 위해 번역 소스의 길이로 내림차순 정렬
        ly, sort_idx = ly.sort(descending=True)
        y = y[sort_idx]
        # Decoder의 초기값 설정
        h0 = (ctx[0][:, sort_idx, :], ctx[1][:, sort_idx, :])
        z = y[:, :-1].detach()
        # -100인 상태에서는 Embedding 계산에서 오류가 발생하므로 0으로 변경
        z[z==-100] = 0
        # 디코더에 넣어서 손실 함수 계산
        o, _ = dec(z, h0, l=ly-1)
        loss = loss_f(to2D(o[:]), to2D(y[:, 1:max(ly)]).squeeze())
        # Backpropagation(오차 역전파 실행)
        enc.zero_grad(), dec.zero_grad()
        loss.backward()
        opt_enc.step(), opt_dec.step()
        losses.append(loss.item())
    # 전체 데이터의 계산이 끝나면 현재의
    # 손실 함수 값이나 번역 결과를 표시
    enc.eval(), dec.eval()
    print(epoc, mean(losses))
```



```
with torch.no_grad():
    print(translate("I am a student.",
                    enc, dec, max_len=max_len, device="cuda:0"))
    print(translate("He likes to eat pizza.",
                    enc, dec, max_len=max_len, device="cuda:0"))
    print(translate("She is my mother.",
                    enc, dec, max_len=max_len, device="cuda:0"))
```

Out

```
100%|**| 1600/1600 [01:31<00:00, 17.45it/s]
0%|          | 0/1600 [00:00<?, ?it/s]0
5.409873641133308
un buen .
a él le gusta comer pizza .
ella es mi madre .

(…중략…)

100%|**| 1600/1600 [01:39<00:00, 16.12it/s]
0%|          | 0/1600 [00:00<?, ?it/s]28 0.4390222669020295
soy un estudiante .
a él le gusta comer pizza .
ella es mi madre .
100%|**| 1600/1600 [01:43<00:00, 15.52it/s]29 0.43130578387528656
soy un estudiante .
a él le gusta comer pizza .
ella es mi madre .
```

- I am a student. -> soy un estudiante .
- He likes to eat pizza. -> a él le gusta comer pizza .
- She is my mother. -> ella es mi madre .

그림 5.5 번역 결과

30회 정도 계산하면 **그림 5.5**와 같은 결과를 얻을 수 있다. 여기서 소개한 예문은 비교적 간단한 것으로 모두 제대로 번역된다. 예를 들어, 첫 번째 예를 번역하면 *yo soy un estudiante*로 번역해야 하지만, 스페인어에서는 명확한 경우에는 주어를 생략할 수 있으므로 *yo*가 생략된다. 따라서 영어보다 한 단어 적은 번역문이 만들어진다.¹¹

¹¹ 이 책은 스페인어 책이 아니므로 스페인어 문법에 대해서는 다루지 않는다. 스페인어 전문서 등을 참고하도록 하자.

두 번째 문장은 스페인어에서 자주 나오는 형식으로, 목적대명사를 앞에 두는 도치 구문이다. 이것도 정확하게 번역돼 있으므로 단순히 단어만 바꾼 것이 아님을 알 수 있다. 참고로, 여기서 다른 모델은 모두 정보를 내부 상태라는 변수에 넣어 두는 제일 간단한 모델이므로 문장이 길어지면 제대로 번역되지 않는다. 하지만 최신 연구 결과에서는 Attention이라는 구조(메모 참고)를 사용해서 위치 단위로 다른 정보를 사용할 수 있게 됐다. 이를 통해 긴 문장도 높은 정확도로 번역할 수 있다.



MEMO

Attention 구조

- Attention and Augmented Recurrent Neural Networks

URL <https://distill.pub/2016/augmented-rnns/>

5.6

정리

이 장에서 다룬 내용을 정리했다.

과거의 흐름을 기록해서 시계열 분석에 적합한 신경망을 만드는 것이 RNN이다. Elman형의 RNN에서는 긴 시계열 학습이 경사 손실 등의 문제 때문에 어려웠지만, LSTM이나 GRU 등의 모델로 이를 해결할 수 있다.

RNN은 문장 판별이나 문장 생성에 사용할 수 있으며, 이 두 개를 조합한 인코더-디코더 모델을 사용해서 기계 번역 등 폭넓게 활용할 수 있다.



CHAPTER

6

추천 시스템과 행렬 분해

이 장에선 행렬 인수분해(MF, Matrix Factorization)라는 추천(recommendation) 시스템에서 사용되는 대표적인 알고리즘을 신경망으로 표현해 보도록 한다. 또한, 이것을 다계층화해서 비선형으로 확장하는 방법을 소개한다.

신경망을 사용한 추천 시스템은 비교적 새로운 연구 분야로 행렬 인수분해나 Factorization Machines 등을 비선형으로 확장하는 등 활발하게 연구가 이루어지고 있다. 이들은 이미지 처리나 자연어 처리 등에서 사용되는 CNN이나 RNN과는 다른 모델로, 신경망의 범용성이 높다고 볼 수 있다.

! ATTENTION

6장의 코드에 대해

이 장의 코드는 환경에 따라서는 다음 오류가 발생할 수 있다.

- DataLoader causing 'RuntimeError: received 0 items of ancdata'#973

URL <https://github.com/pytorch/pytorch/issues/973>

이 책의 번역 시점(2018년 12월 현재)에는 스크립트나 주피터 노트북을 실행하기 전에 터미널에서 `ulimit -n 4096`을 입력해서 OS의 최대 파일 수를 늘려야 한다.

6.1

행렬 인수분해

대표적인 추천 기법인 행렬 인수분해에 대해 설명하고,
파이토치의 신경망 모듈을 사용한 구현 방법을 보도록 한다.

6.1.1 이론적 배경

행렬 인수분해(MF, Matrix Factorization)에서는 **그림 6.1**처럼 열이 상품이고 행이 사용자, 그리고 해당 사용자가 해당 상품을 구입한 횟수나 평가지 등의 구매 이력이 값이 된다. 상품 수를 M , 사용자 수를 N 이라고 하면 $N \times M$ 인 행렬 A 가 되며, 이것도 희소 행렬이다. N 이나 M 보다 훨씬 작은 K 를 정하고, 이 행렬을 $N \times K$ 인 사용자 인수 행렬 U 와 $M \times K$ 인 상품 인수 행렬 V 의 곱으로 근사하는 것이 MF의 기본 개념이다.

U 와 V 의 곱을 계산한 A' 는 A 와 유사성을 유지한다. 또한, 구매하지 않은 사용자와 상품 조합이 0이 아닌 경우는 사용자가 관심을 가지고 있지만,

아직 구매하지 않은 상품이므로 추천 대상이 된다.

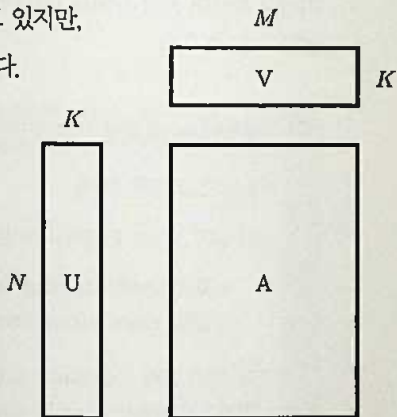


그림 6.1 행렬 인수분해. $A \approx UV^T$ (U 는 전치)로 A 를 근사한다. U 는 사용자의 특이값 벡터를 묶은 행렬이고, V 는 상품의 특이값 벡터를 묶은 행렬이라고 보면 된다.

실제로는 사용자 i 의 특이량 벡터와 특정 상품 j 의 특이량 벡터를 내적하면 사용자 i 의 상품 j 에 대한 평가를 계산할 수 있다. 이처럼 분해를 찾는 알고리즘에는 다양한 것이 있으며, **특이값 분해(SVD, Singular Value Decomposition)**나 **특이값 분해를 개선한 비음수 행렬 인수분해(NMF, Non-negative Matrix Factorization)** 등이 유명하다.

참고로, 자연어 처리에서는 이런 분석을 **잠재 의미 분석(LSA, Latent Semantic Analysis)**이라고도 한다.

6.1.2 MovieLens 데이터

MovieLens는 추천 시스템의 대표적인 샘플 데이터다. 이것은 2만 7,000여 편의 영화에 대한 10만 개 이상의 리뷰 데이터(5단계의 숫자)로 구성돼 있다. 평가 외에도 각 영화의 장르 데이터도 포함하고 있다. 다음 URL의 웹사이트에서 ml-20m.zip이라는 파일을 다운로드하자.

- MovieLens

URL <http://files.grouplens.org/datasets/movielens/ml-20m.zip>

이 파일 안에서 ratings.csv라는 파일을 사용한다. 이 CSV 파일은 userId, movieId, rating, timestamp 등 네 개의 열을 가지고 있으며, 여기서는 이 중 **앞의 세 개**만 사용한다. userId와 movieId는 명칭에서도 알 수 있듯이 사용자와 영화 아이디이고 rating은 평가 점수다. 점수는 0부터 5까지 있으며 0.5씩 증가하지만, 필자가 확인한 결과 0점인 영화는 하나도 없었다.



MEMO

컬래버러터리에서 압축 파일 해제

다음 명령을 실행하면 압축 파일을 다운로드해서 해제한다.

```
!wget http://files.grouplens.org/datasets/movielens/ml-20m.zip
!unzip ml-20m.zip
```


● Dataset과 DataLoader 작성

리스트 6.1의 코드에서는 판다(Pandas)를 사용해서 파일을 읽는다. 그리고 훈련 데이터와 테스트 데이터로 나눈 후에 각각의 Dataset과 DataLoader를 작성하고 있다. (userId, movieId) 쌍이 X이고 rating이 Y가 되도록 Dataset을 만든다.

리스트 6.1 Dataset과 DataLoader 작성

```
import torch
from torch import nn, optim
from torch.utils.data import (Dataset,
                               DataLoader,
                               TensorDataset)

import tqdm
```



```
import pandas as pd
# 훈련 데이터와 테스트 데이터를 나누기 위해 사용한다
from sklearn import model_selection
```

```
df = pd.read_csv("raiting.csv 파일의 경로")
# X는 (userId, movieId) 쌍
X = df[["userId", "movieId"]].values
Y = df[["rating"]].values
```

입력 데이터인 torch.tensor(X)의 경우: 평점 데이터의 경우 '/content/ml-20m/raiting.csv'가 된다.

```
# 훈련 데이터와 테스트 데이터를 9대 1로 분할
train_X, test_X, train_Y, test_Y\
    = model_selection.train_test_split(X, Y, test_size=0.1)
```

```
# X는 ID이고 정수이므로 int64, Y는 실수이므로 float32의 텐서로 변환
train_dataset = TensorDataset(
    torch.tensor(train_X, dtype=torch.int64), torch.tensor(train_Y, dtype=
torch.float32))
test_dataset = TensorDataset(
    torch.tensor(test_X, dtype=torch.int64), torch.tensor(test_Y, dtype=
torch.float32))
```

```
train_loader = DataLoader(
    train_dataset, batch_size=1024, num_workers=4, shuffle=True)
test_loader = DataLoader(
    test_dataset, batch_size=1024, num_workers=4)
```



6.1.3 파이토치에서 행렬 인수분해하기

파이토치를 사용해 행렬 인수분해 모델을 만들고 훈련하는 방법을 소개한다.

● 사용자 및 상품 ID를 K 차원의 벡터로 변환

먼저 사용자 및 상품 ID를 K 차원의 벡터로 변환하는 방법을 생각해 보자. 흥미로운 것은 5장에서 다른 `nn.Embedding`을 사용하면 딱 맞는다. 특히 벡터가 만들어지면 내적만 계산하면 되므로 [리스트 6.2](#)와 같이 작성할 수 있다. 단순히 내적을 그대로 평가 예측값으로 사용하면 $[0, 5]$ 범위를 벗어날 가능성이 있으므로 마지막에 시그모이드 함수를 사용해서 $[0, 5]$ 범위로 들어오게 조정한다.

리스트 6.2 행렬 인수분해



```
class MatrixFactorization(nn.Module):
    def __init__(self, max_user, max_item, k=20):
        super().__init__()
        self.max_user = max_user
        self.max_item = max_item
        self.user_emb = nn.Embedding(max_user, k, 0)
        self.item_emb = nn.Embedding(max_item, k, 0)

    def forward(self, x):
        user_idx = x[:, 0]
        item_idx = x[:, 1]
        user_feature = self.user_emb(user_idx)
        item_feature = self.item_emb(item_idx)

        # user_feature*item_feature는 (batch_size,k)차원이므로
        # k의 sum을 구하면 각 샘플의 내적이 된다
        out = torch.sum(user_feature * item_feature, 1)

        # [0, 5] 범위 내로 조정
        out = nn.functional.sigmoid(out) * 5
        return out
```

● 사용자 및 상품 개수

실제로 이 클래스의 인스턴스를 만들려면 사용자 및 상품 개수를 알아야 한다. MovieLens 데이터에서는 ID가 1부터 시작하므로 사용자 및 상품 ID의 최댓값 + 1을 개수로 정하면 된다(**리스트 6.3**).¹²

리스트 6.3 사용자 및 상품 개수



```
max_user, max_item = X.max(0)
# np.int64형을 파이썬의 표준 int로 캐스트
max_user = int(max_user)
max_item = int(max_item)
net = MatrixFactorization(max_user+1, max_item+1)
```

● 평가 함수 작성

다음은 늘 했던 것처럼 테스트 데이터를 사용한 평가 함수와 실제 훈련 부분을 만들면 된다. 먼저, 테스트 데이터의 평가 함수를 보자. 여기서는 **MAE(Mean Absolute Error)**라는 **평균 절대 오차** 지표를 사용한다. 파이토치에도 `nn.L1Loss` 또는 `nn.functional.l1_loss`라는 MAE 계산 클래스 및 함수가 있으므로 이것을 이용하도록 한다(**리스트 6.4**).

리스트 6.4 평가 함수 작성



```
def eval_net(net, loader, score_fn=nn.functional.l1_loss, device="cpu"):
    ys = []
    ypreds = []
    for x, y in loader:
        x = x.to(device)
        ys.append(y)
        with torch.no_grad():
            ypred = net(x).to("cpu").view(-1)
        ypreds.append(ypred)
    score = score_fn(torch.cat(ys).squeeze(), torch.cat(ypreds))
    return score.item()
```

¹² **리스트 6.3** 개수는 상품 ID의 최댓값과 같다. 하지만 5장에서 설명한 `nn.Embedding()`에서 `padding_idx=0`을 사용하므로 +1로 처리한다.(베타리더의 보충 설명)

● 훈련 부분 작성

준비됐으면 실제 훈련을 해보도록 한다(**리스트 6.5**). MovieLens는 데이터가 많아서 SGD 학습률을 높게 설정해도 괜찮다. 따라서 기본값의 10배인 0.01을 설정하고 있다.

리스트 6.5 훈련 부분 작성



```
from statistics import mean

net.to("cuda:0")
opt = optim.Adam(net.parameters(), lr=0.01)
loss_f = nn.MSELoss()

for epoch in range(5):
    loss_log = []
    for x, y in tqdm.tqdm(train_loader):
        x = x.to("cuda:0")
        y = y.to("cuda:0")
        o = net(x)
        loss = loss_f(o, y.view(-1))
        net.zero_grad()
        loss.backward()
        opt.step()
        loss_log.append(loss.item())
    test_score = eval_net(net, test_loader, device="cuda:0")
    print(epoch, mean(loss_log), test_score, flush=True)
```



```
0 1.6175087428118726 0.7348315119743347
1 0.884978003734221 0.711135983467102
2 0.8419708782708769 0.7031168341636658
3 0.8203578021781451 0.6991654634475708
4 0.8070978848076604 0.69717937707901
```

5회 정도 계산하면 MAE가 0.7 미만까지 개선된다. 예를 들어, 사용자1의 영화10에 대한 평가를 실제로 예측하려면 **리스트 6.6**과 같이 하면 된다.

리스트 6.6 사용자1의 영화10에 대한 평가를 예측



```
# 훈련한 모델을 CPU로 이동
net.to("cpu")
```

```
# 사용자1의 영화10에 대한 평가 계산
query = (1, 10)

# int64 텐서로 변환하고 batch 차원을 추가
query = torch.tensor(query, dtype=torch.int64).view(1, -1)

# net에 전달
net(query)
```

Out

```
tensor([ 3.9975])
```

또한, 특정 사용자의 전체 영화에 대한 평가를 계산해서 상위 5개 영화를 추출하는 것도 쉽게 해낼 수 있다. 영화 수만큼 (userId, movieId) 쌍을 만들어서 신경망에 전달하고 평가값을 계산한다. 그리고 torch.topk 함수를 사용해서 상위 5개만 추출하면 된다. **리스트 6.7**에선 사용자1의 상위 5개 영화를 선택하고 있다. torch.topk는 상위 k개의 값뿐만 아니라 위치도 알려 준다.

리스트 6.7 사용자1의 상위 5개 영화 선택

In

```
query = torch.stack([
    torch.zeros(max_item).fill_(1),
    torch.arange(1, max_item+1)
], 1).long()

# scores는 상위 k개의 점수
# indices는 상위 k개의 위치, 즉 movieId
scores, indices = torch.topk(net(query), 5)
```


신경망 행렬 인수분해

행렬 인수분해를 비선형으로 확장한 신경망 행렬 인수분해를 소개한다.
파이토치를 사용하면 매우 유연하게 모델링할 수 있다.

6.2.1 행렬 인수분해를 비선형화

6.1절에서는 MF를 파이토치의 자동 미분 기능을 사용해서 구현했지만, 실제로 실행해 보면 다른 MF 라이브러리와 비교해 GPU를 사용한다고 해도 속도가 빠르지 않다. 이것은 MF라는 간단한 모델을 억지로 신경망에 맞추려고 해서 불필요한 계산이 발생하기 때문이다.

하지만 신경망에는 **일반 MF보다 유연한 모델링이 가능하다는** 장점이 있다. 일반 MF에서는 사용자와 상품의 특이량 벡터 내적이라는 아주 단순한 선형 모델을 사용했지만, 신경망을 사용하면 비선형 함수를 이용해서 모델링할 수 있다. **리스트 6.8**의 예를 보면 이해가 갈 것이다.

리스트 6.8 비선형 함수를 사용한 모델링

↗ In

```
class NeuralMatrixFactorization(nn.Module):
    def __init__(self, max_user, max_item,
                  user_k=10, item_k=10,
                  hidden_dim=50):
        super().__init__()
        self.user_emb = nn.Embedding(max_user, user_k, 0)
        self.item_emb = nn.Embedding(max_item, item_k, 0)
        self.mlp = nn.Sequential(
            nn.Linear(user_k + item_k, hidden_dim),
```

```

        nn.ReLU(),
        nn.BatchNorm1d(hidden_dim),
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.BatchNorm1d(hidden_dim),
        nn.Linear(hidden_dim, 1)
    )

    def forward(self, x):
        user_idx = x[:, 0]
        item_idx = x[:, 1]
        user_feature = self.user_emb(user_idx)
        item_feature = self.item_emb(item_idx)
        # 사용자 특이량과 상품 특이량을 모아서 하나의 벡터로 만들
        out = torch.cat([user_feature, item_feature], 1)
        # 모든 벡터를 MLP에 넣는다
        out = self.mlp(out)
        out = nn.functional.sigmoid(out) * 5
        return out.squeeze()

```

여기서는 NeuralMF(NeuralMatrixFactorization)라는 형식으로 모델에 이름을 붙였다. 이 모델의 가장 큰 특징은 사용자와 상품의 특이량 벡터 내적을 구하는 대신에 양쪽을 결합해서 새로운 벡터를 만들고, 이것을 MLP에 넣어서 비선형 함수를 모델링한다는 점이다. 이 모델을 MF와 동일하게 훈련하면 5회 반복에서 MAE = 0.62까지 개선돼서 MF보다 높은 정확도를 얻을 수 있다.

NeuralMF는 내적을 사용하지 않으므로 사용자와 상품의 특이량 차원에 서로 다른 값을 사용할 수도 있다. 사용자와 상품의 개수가 다른 경우 등에는 서로 다른 차원을 사용해서 정확도를 크게 떨어뜨리지 않고 파라미터 수를 줄일 수 있다. 이를 통해 훈련 속도도 빨라지게 된다.

또한, NeuralMF는 Batch Normalization 등 신경망을 훈련할 때에 사용하는 기술을 그대로 적용할 수 있다는 점도 주목할 부분이다.

6.2.2 부속 정보 이용

NeuralMF의 장점은 이것뿐만이 아니다. 일반 MF는 사용자와 상품만 고려할 수 있지만, NeuralMF에서는 이외에도 부속 정보를 사용해 모델을 쉽게 확장할 수도 있다.

MovieLens에는 각 영화의 장르 정보도 포함되어 있으므로 이것도 이용해 보도록 하자. ml-20m.zip을 압축 해제한 디렉터리에 movies라는 파일이 있을 것이다. 이 파일에 각 영화의 장르가 기록돼 있다. 구체적으로는 다음과 같은 형식이다.

```
movieId,title,genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
2,Jumanji (1995),Adventure|Children|Fantasy
3,Grumpier Old Men (1995),Comedy|Romance
```

영화 ID, 제목, 장르순으로 나열돼 있으며, 장르가 여러 개인 경우 파이프 기호(|)로 연결하고 있다. 이와 같이 복수의 항목을 가변 길이로 지니고 있는 데이터를 수치화할 때는 여러 가지 방법이 있다. 여기서는 장르가 24가지밖에 없으므로 간단하게 BoW를 사용하도록 한다.

● 장르를 BoW로 변환

리스트 6.9에서는 scikit-learn의 CountVectorizer를 사용해서 A|B|C 형식으로 연결된 장르를 BoW로 변환하고 있다.

리스트 6.9 연결된 장르를 BoW로 변환



```
import csv
from sklearn.feature_extraction.text import CountVectorizer

# csv.DictReader를 사용해서 CSV 파일 읽기
# 필요한 부분만 추출
with open("<your_path>/ml-20m/movies.csv") as fp:
    reader = csv.DictReader(fp)
    def parse(d):
        movieId = int(d["movieId"])
        genres = d["genres"]
        return movieId, genres
    data = [parse(d) for d in reader]
```

```

movieIds = [x[0] for x in data]
genres = [x[1] for x in data]

# 데이터에 맞추어 CountVectorizer를 훈련
cv = CountVectorizer(dtype="f4").fit(genres)
num_genres = len(cv.get_feature_names())

# key가 movieId이고 value가 BoW인 텐서의 dict 만들기
it = cv.transform(genres).toarray()
it = (torch.tensor(g, dtype=torch.float32) for g in it)
genre_dict = dict(zip(movieIds, it))

```

● 커스텀 Dataset 작성

리스트 6.9에서 장르 사전을 만들었다. 이 사전을 이용해서 영화 ID를 입력해 장르 정보를 가져올 수 있다. 다음은 사용자 ID나 영화 ID를 함께 장르의 BoW를 반환하는 Dataset을 만든다(**리스트 6.10**). 장르 사전을 생성자에서 전달하고 요소를 가져올 때에 영화 ID를 사용해 장르의 BoW를 함께 반환하고 있다.

리스트 6.10 커스텀 Dataset 작성



```

def first(xs):
    it = iter(xs)
    return next(it)

class MovieLensDataset(Dataset):
    def __init__(self, x, y, genres):
        assert len(x) == len(y)
        self.x = x
        self.y = y
        self.genres = genres

        # 장르 사전에 없는 movieId를 위한 더미 데이터
        self.null_genre = torch.zeros_like(
            first(genres.values()))

    def __len__(self):
        return len(self.x)

    def __getitem__(self, idx):
        x = self.x[idx]
        y = self.y[idx]

```



```
# x = (userId, movieId)
movieId = x[1]
g = self.genres.get(movieId, self.null_genre)
return x, y, g
```

● DataLoader 작성

MovieLensDataset이 준비됐으니 DataLoader까지 만들어 보자(리스트 6.11).

리스트 6.11 DataLoader 작성



```
train_dataset = MovieLensDataset(
    torch.tensor(train_X, dtype=torch.int64),
    torch.tensor(train_Y, dtype=torch.float32),
    genre_dict)
test_dataset = MovieLensDataset(
    torch.tensor(test_X, dtype=torch.int64),
    torch.tensor(test_Y, dtype=torch.float32),
    genre_dict)
train_loader = DataLoader(
    train_dataset, batch_size=1024, shuffle=True, num_workers=4)
test_loader = DataLoader(
    test_dataset, batch_size=1024, num_workers=4)
```

● 신경망 모델 작성

다음은 장르 정보를 이용한 신경망 모델인 NeuralMatrixFactorization2를 만든다(리스트 6.12). 6.2.1절의 NeuralMatrixFactorization과 다른 점은, MLP 계층의 첫 번째 입력인 Linear 차원에 장르 BoW의 차원이 추가된 점과, forward 인수에 장르 BoW를 전달해서 사용자 및 상품 특이량과 결합하고 있는 점이다.

리스트 6.12 신경망 모델 작성



```
class NeuralMatrixFactorization2(nn.Module):
    def __init__(self, max_user, max_item, num_genres,
                 user_k=10, item_k=10, hidden_dim=50):
        super().__init__()
        self.user_emb = nn.Embedding(max_user, user_k, 0)
        self.item_emb = nn.Embedding(max_item, item_k, 0)
        self.mlp = nn.Sequential(
            # num_genres만큼 차원이 늘어난다
```



```

        nn.Linear(user_k + item_k + num_genres, hidden_dim),
        nn.ReLU(),
        nn.BatchNorm1d(hidden_dim),
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.BatchNorm1d(hidden_dim),
        nn.Linear(hidden_dim, 1)
    )

    def forward(self, x, g):
        user_idx = x[:, 0]
        item_idx = x[:, 1]
        user_feature = self.user_emb(user_idx)
        item_feature = self.item_emb(item_idx)
        # 장르 BoW를 cat로 특이 벡터에 결합한다
        out = torch.cat([user_feature, item_feature, g], 1)
        out = self.mlp(out)
        out = nn.functional.sigmoid(out) * 5
        return out.squeeze()

```

● 헬퍼 함수 수정

DataLoader가 반환하는 변수의 형이 다르므로 평가 헬퍼 함수도 약간 수정해야 한다
([리스트 6.13](#)).

리스트 6.13 헬퍼 함수 수정



```

def eval_net(net, loader, score_fn=nn.functional.l1_loss, device="cpu"):
    ys = []
    ypreds = []
    # loader는 장르 BoW도 반환
    for x, y, g in loader:
        x = x.to(device)
        g = g.to(device)
        ys.append(y)
        # userId, movieId 외에 장르 BoW
        # BoW도 신경망 함수에 전달
        with torch.no_grad():
            ypred = net(x, g).to("cpu")
            ypreds.append(ypred)
    score = score_fn(torch.cat(ys).squeeze(), torch.cat(ypreds))
    return score

```

● 훈련 부분 작성

모든 준비가 완료됐으니 훈련해 보도록 한다(리스트 6.14). train_loader가 장르 BoW도 반환하는 것에 주의하자.

리스트 6.14 훈련 부분 작성

```
net = NeuralMatrixFactorization2(
    max_user+1, max_item+1, num_genres)
opt = optim.Adam(net.parameters(), lr=0.01)
loss_f = nn.MSELoss()

net.to("cuda:0")
for epoch in range(5):
    loss_log = []
    net.train()
    for x, y, g in tqdm.tqdm(train_loader):
        x = x.to("cuda:0")
        y = y.to("cuda:0")
        g = g.to("cuda:0")
        o = net(x, g)
        loss = loss_f(o, y.view(-1))
        net.zero_grad()
        loss.backward()
        opt.step()
        loss_log.append(loss.item())
    net.eval()
    test_score = eval_net(net, test_loader, device="cuda:0")
    print(epoch, mean(loss_log), test_score.item(), flush=True)
```

```
0 0.7065280483494481 0.6462
1 0.6938810969105664 0.6383
2 0.6711728837724852 0.6307
3 0.6565303146869658 0.6268
3 0.6565303146869658 0.6268
4 0.6377939984445665 0.6172
```

5회 반복으로 MAE가 0.62를 하회하며, NeuralMF보다 정확도가 많이 향상된 것을 알 수 있다. 장르 정보를 사용하므로 정확도 개선 외에도 사용자 ID와 장르 정보만으로 추천(recommend)이 가능해진다. 즉, 이 데이터에 없는 상품(영화)에 대해서도 점수를 제

산할 수 있게 되는 것이다. 물론 이 경우는 정확도가 떨어지긴 하지만, 아직 판매되지 않고 있는 상품의 장르(종류)만 알고 있으면 사용자 단위로 어떤 상품을 좋아하는지 파악할 수 있게 된다.

예를 들어, 사용자 100명을 대상으로 각 장르를 하나씩 포함하고 있는(24개 장르) 영화의 점수를 계산해 보자(리스트 6.15). 데이터에 없는 영화이므로 ID에 0을 지정한다. 이렇게 하면 리스트 6.12에서 nn.Embedding 생성자의 세 번째 인수 padding_idx에 0을 지정했으므로 영화의 특이량 벡터가 모두 0이 된다.

리스트 6.15 사용자 100명을 대상으로 각 장르를 하나씩 포함하는 영화 점수 계산하기



```
# 지정한 위치만 1이고 나머지가 0인 텐서 반환
def make_genre_vector(i, max_len):
    g = torch.zeros(max_len)
    g[i] = 1
    return g

query_genres = [make_genre_vector(i, num_genres)
                 for i in range(num_genres)]
query_genres = torch.stack(query_genres, 1)

# num_genres분만큼 userId=100과 movieId=0의 텐서를 만들어 결합
query = torch.stack([
    torch.empty(num_genres, dtype=torch.int64).fill_(100),
    torch.empty(num_genres, dtype=torch.int64).fill_(0)
], 1)

# GPU로 전송
query_genres = query_genres.to("cuda:0")
query = query.to("cuda:0")

# 점수 계산
net(query, query_genres)
```



```
tensor([ 3.1709,  3.1712,  3.1704,  3.1711,  3.1683,  3.1723,  3.1722,
         3.1700,  3.1716,  3.1706,  3.1687,  3.1705,  3.1723,  3.1697,
         3.1707,  3.1703,  3.1701,  3.1695,  3.1703,  3.1714,  3.1682,
         3.1691,  3.1707,  3.1708], device='cuda:0')
```

이것으로 미지의 영화에 대해서도 대략적인 예측값을 계산할 수 있다. MovieLens에는 포함돼 있지 않지만, 부족 데이터도 동일한 방법으로 모델에 포함시킬 수 있다. 만약 그런 데이터를 가지고 있다면 꼭 테스트해 보도록 하자.

6.3

정리

이 장에서 배운 내용을 정리했다.

파이토치를 사용해서 신경망을 기반으로 한 행렬 인수분해 방법을 소개했다. 단순한 방법(선형 모델)보다는 속도는 떨어지지만, 사용자와 상품의 비선형 관계를 모델링하거나 사용자 및 상품의 부속 정보를 포함하는 등 모델을 유연하게 확장해서 높은 예측 정확도를 가진 모델을 구축할 수 있다. 파이토치는 이미지 인식 처리나 자연어 처리 이외에도 다양한 분야에 적용할 수 있다.



CHAPTER

7

애플리케이션 적용

이 장에서는 학습 모델을 실제 애플리케이션에 적용하는 방법을 소개한다. 주로 웹 API를 만들기 위해 모델의 저장 및 불러오기를 구현하고, 도커(Docker)를 사용한 애플리케이션 배포 방법도 다룬다.

! ATTENTION

7장의 실행 환경

7.1절부터 7.3절까지 내용은 우분투 장비에서 코드를 커맨드 라인에서 실행하고 있다. 또한, 우분투의 미니콘다(Miniconda) 사용을 전제로 하고 있다. 컬래버테리 환경에서는 실행할 수 없으니 양해 바란다. 단, 7.4절 이후는 컬래버테리에서 실행할 수 있다.

모델 저장과 불러오기

모델의 저장과 불러오기에 대해 설명한다. GPU 등으로 학습한 모델을 웹 API로 만들거나 CPU만 있는 서버에 배포할 때 자주 사용하는 방식이다.

신경망이나 딥러닝뿐만 아니라 머신러닝의 학습 모델을 저장해 두었다가 나중에 재사용하는 것은 매우 중요하다. 이때 **모델 저장**이란, 모델 구조 자체와 학습한 파라미터를 함께 저장하는 것을 뜻한다. 모델 구조는 코드를 그대로 사용하면 되며, 학습한 파라미터는 대량의 수치 데이터이므로 파일로 저장할 필요가 있다.

● 파이토치에서의 모델 저장과 불러오기

파이토치에서는 `state_dict` 메서드를 사용해서 파라미터의 텐서를 사전 형식으로 추출할 수 있으며, `torch.save`라는 pickle의 래퍼(wrapper) 함수를 사용해서 파일로 저장할 수 있다 (리스트 7.1).

참고로, `pickle_protocol=4`는 파이썬 3.4 이후에 추가된 pickle 형식으로 큰 객체를 효율적으로 저장할 수 있다.

리스트 7.1 모델 저장의 예

 In

```
# 학습이 끝난 신경망 모델
net_params = net.state_dict()
# net.prm라는 파일로 저장
torch.save(params, "net.prm", pickle_protocol=4)
```

한편 저장한 파라미터 파일을 불러올 때는 `torch.load` 함수를 사용하고, `nn.Module`의 `load_state_dict` 메서드에 전달하면 신경망 모델에 파라미터를 설정할 수 있다(리스트 7.2).

리스트 7.2 모델 불러오기 예

 In

```
# net.prm 불러오기
params = torch.load("net.prm", map_location="cpu")
net.load_state_dict(params)
```

`map_location`은 읽은 파라미터를 어디에 저장할지 정하는 것이다. 예를 들어, GPU로 학습한 모델의 파라미터를 그대로 저장하면 `torch.load`로 읽는 경우 우선 CPU로 불러온 후에 GPU로 전송한다. GPU가 없는 서버(장비)에서 사용할 때는 오류가 발생해 버린다.

이런 오류를 방지하기 위해 `map_location` 인수를 사용해 읽은 후의 동작을 지정한다. `map_location`에는 데이터를 어디에 어떻게 배치할지 제어하는 함수를 지정할 수도 있지만, 파이토치 버전 0.4부터는 단순히 장비명만 지정해도 되도록 변경됐다. 예를 들어, 리스트 7.2에서처럼 `map_location="cpu"`라고 하면 CPU의 메모리상에 그대로 저장된다.

참고로, 리스트 7.3처럼 파라미터 저장 시에 일단 모델을 CPU로 전송해 두는 것도 좋다. 이때는 `map_location`을 지정하지 않고 바로 `torch.load`로 읽을 수 있다.

리스트 7.3 파라미터를 CPU로 전송한 후 저장하는 예

 In

```
# 일단 CPU로 모델 이동
net.cpu()
# 파라미터 저장
params = net.state_dict()
torch.save(params, "net.prm", pickle_protocol=4)
```

플라스크를 사용한 웹 API화

파이토치로 학습한 모델을 사용할 수 있게 해주는 웹 API 작성법에 대해 설명한다.

웹 API 등의 웹 애플리케이션을 개발해서 실행할 때는 애플리케이션 프레임워크와 애플리케이션 서버 두 가지가 필요하다(**메모** 참고). 애플리케이션 프레임워크는 실제로 웹 애플리케이션 내에서 이루어지는 다양한 처리를 작성하기 쉬운 형태로 제공하는 라이브러리다. 반면 애플리케이션 서버는 HTTP 등의 방식으로 클라이언트로부터 요청(request)을 받아서, 웹 애플리케이션 내부에서 요청에 맞는 함수를 호출하고, 그 결과(response)를 HTTP 등을 통해 반환하는 역할을 한다.

파이썬에는 WSGI(Web Server Gateway Interface)라는 인터페이스가 정해져 있어서, 애플리케이션 프레임워크와 애플리케이션 서버가 모두 이 인터페이스를 따르고 있다면, 어떤 형태로든 조합해서 사용할 수 있게 되어 있다. 여기서는 **플라스크(Flask)**라는 대표적인 프레임워크(**그림 7-1**)와 **지유니콘(Gunicorn)**(**그림 7-2**)이라는 서버를 사용한다.

기본적으로는 파이토치의 모델을 내장하고 있는 플라스크의 WSGI 애플리케이션을 작성하고, 이것을 지유니콘상에서 실행하는 방식이다. **4장**에서 작성한 타코와 부리토 식별 모델을 예를 들어 설명하도록 한다.



[overview](#) // [docs](#) // [community](#) // [extensions](#) // [donate](#)

Flask is a microframework for Python based on Werkzeug, Jinja 2 and good intentions. And before you ask: It's BSD licensed!

Flask is Fun

Latest Version: 1.0.2

그림 7.1 플라스크

- Flask web development, one drop at a time

URL <http://flask.pocoo.org/>



그림 7.2 자유니콘

- Gunicorn

URL <http://gunicorn.org/>



MEMO

애플리케이션 프레임워크와 애플리케이션 서버

최근에는 Node.js나 Golang처럼 언어 자체에 서버가 내장돼 있는 경우도 많다.

그림 7.3과 같은 디렉터리 구성을 하고 있다.¹³

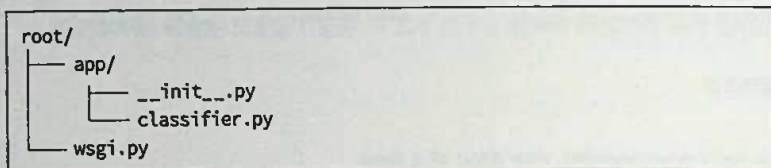


그림 7.3 디렉터리 구성

● 래퍼 클래스 작성

먼저, 파이토치 모델을 사용하기 쉽게 래퍼(wrapper) 클래스를 만들도록 한다. **리스트 7.4**의 신경망 생성 함수와 **리스트 7.5** 클래스 정의를 작성하자.

리스트 7.4 classifier.py(신경망 생성 함수)

```

# classifier.py
from torch import nn
from torchvision import transforms, models

def create_network():
    # resnet18 읽기
    # 파라미터는 뒤에서 설정하므로 pretrained=True는 필요 없다
    net = models.resnet18()

    # 마지막 계층을 2출력의 선형 계층으로 변경
    fc_input_dim = net.fc.in_features
  
```

13 **참고** 설치하는 pip install을 사용해도 된다. 콘솔 화면에서 다음 명령을 실행하자.

```

pip install flask
pip install gunicorn
  
```

```
net.fc = nn.Linear(fc_input_dim, 2)
return net
```

리스트 7.5 classifier.py(클래스 정의)

```
class Classifier(object):
    def __init__(self, params):
        # 식별 네트워크 작성
        self.net = create_network()
        # 학습 완료 파라미터 설정
        self.net.load_state_dict(params)
        # 평가 모드로 설정
        self.net.eval()
        # 이미지를 텐서로 변환하는 함수
        self.transformer = transforms.Compose([
            transforms.CenterCrop(224),
            transforms.ToTensor()
        ])
        # 분류 ID와 분류명 연결
        self.classes = ["burrito", "taco"]

    def predict(self, img):
        # 이미지를 텐서로 변환
        x = self.transformer(img)
        # 파이토치는 항상 배치로 처리하므로
        # batch의 차원을 선두에 추가
        x = x.unsqueeze(0)
        # 신경망의 출력 계산
        out = self.net(x)
        out = out.max(1)[1].item()
        # 예측한 분류명 반환
        return self.classes[out]
```

create_network이라는 함수가 ResNet18을 기반으로 하고 있는 마지막 계층을 2출력의 선형 계층으로 바꾼 신경망을 만든다. Classifier의 생성자 내에서 이 신경망을 만들고 7.1절에서 설명한 방법으로 이미 학습돼 있는 파라미터를 설정한다.

ResNet18은 Batch Normalization을 포함하고 있으므로 eval 메서드를 사용해서 신경망을 평가 모드로 설정하는 것을 잊으면 안 된다. 또한, PIL 패키지(메모 참고)로 읽은 이미지를 파이토치의 텐서로 변환하는 함수를 torchvision.transforms를 사용해서 만들어 둔다.

predict 메서드는 PIL의 이미지를 받아서 처리 가능한 형태로 변경한 후 텐서로 변환한다. 그리고 이 텐서를 신경망으로 보내서 예측한 분류 ID를 구하고, 실제 분류명 (burrito 또는 taco)을 반환한다.



MEMO

PIL 패키지

pip 등을 사용해서 설치할 때의 패키지명은 pillow다. 원래는 PIL이라는 라이브러리가 있었지만, 개발이 지체되면서 fork됐고 pillow가 표준 패키지로 자리 잡았다.

● 플라스크 애플리케이션 작성

다음은 이 식별 모델을 사용하는 플라스크 애플리케이션을 만들어 본다. **리스트 7.6**의 코드를 `__init__.py` 파일에 작성하자.

리스트 7.6 `__init__.py`

```
# __init__.py
from flask import Flask, request, jsonify
from PIL import Image

def create_app(classifier):
    # 플라스크 애플리케이션 생성
    app = Flask(__name__)

    # POST / 요청을 처리하는 함수 정의
    @app.route("/", methods=["POST"])
    def predict():
        # 받은 파일의 핸들러 취득
        img_file = request.files["img"]

        # 파일이 비어 있는지 확인
        if img_file.filename == "":
            return "Bad Request", 400

        # PIL을 사용해서 이미지 파일 읽기
        img = Image.open(img_file)
```

```
# 식별 모델을 사용해서 타코인지 부리토인지 예측
result = classifier.predict(img)

# 결과를 JSON 형식으로 반환
return jsonify({
    "result": result
})

return app
```

플라스크에서는 처음에 플라스크의 인스턴스를 만들고, `@app.route` 같은 데코레이터(decorator)로 처리 핸들러를 추가한다. 클라이언트가 `multipart/form-data` 형식으로 전송한 파일은 `request.files`라는 dict에 저장된다. 여기서는 `img`라는 정해진 키를 dict으로 사용하고 있다. 파일이 제대로 전송됐는지 확인하며, 만약 문제가 있으면 400 Bad Request를 반환한다.

참고로, 예외 처리는 최소한만 구현했다. 필요에 따라 추가하도록 하자. 이미지 파일은 PIL 패키지를 사용해서 읽는다. scikit-image나 OpenCV 등을 이용해서 읽어도 되지만, torchvision은 PIL 형식의 데이터 편집을 위한 다양한 함수를 제공하므로 여기서는 PIL을 사용하고 있다.

마지막으로, 이미지 데이터를 `classifier.predict`에 입력해서 얻은 분류명을 `jsonify` 함수를 사용해 JSON 형식으로 클라이언트에 반환한다. 이때 반환하는 응답(response)은 **리스트 7.7**과 같다.

리스트 7.7 json

```
{
    "result": "burrito"
}
```

● 엔트리 포인트 'wsgi.py' 작성

마지막으로, 지유니콘을 실행할 때에 사용할 엔트리 포인트(entrypoint)인 `wsgi.py` 파일을 만든다(**리스트 7.8**). 저장해 둔 학습이 끝난 파라미터를 읽어서 지금까지 설명한 Classifier와 플라스크 애플리케이션을 생성하는 것이 전부다. 파라미터 파일의 경로는

인수가 아닌 환경 변수로 설정하면 지유니콘으로 쉽게 실행할 수 있으며, 마지막에서 소개하는 도커(Docker)로 배포할 때도 편리하다. smart_getenv라는 패키지를 사용하면 기본값 등을 지정해서 환경 변수를 쉽게 가져올 수 있다.

리스트 7.8 wsgi.py

```
# wsgi.py
import torch
from smart_getenv import getenv

from app import create_app
from app.classifier import Classifier

# 파라미터 파일의 경로를 환경 변수에서 읽기
prm_file = getenv("PRM_FILE", default="/data/taco_burrito.prm")
# 파라미터 파일 읽기
params = torch.load(prm_file,
                    map_location=lambda storage, loc: storage)
# Classifier와 Flask 애플리케이션 생성
classifier = Classifier(params)
app = create_app(classifier)
```

● 지유니콘 실행

준비가 끝났으니 지유니콘을 실행해 보자. 다음 명령을 wsgi.py가 있는 디렉터리에서 실행하면 된다. 명령 실행 전에 학습이 끝난 파라미터 파일도 동일 디렉터리에 taco_burrito.prm이라는 파일명으로 저장해 두어야 한다. 이 파일은 4.3절에서 작성한 모델을 7.1절에서 소개한 방법으로 출력할 수 있으며, 이 책이 제공하는 예제 코드 파일을 통해서도 얻을 수 있다.

우분투 환경

```
$ PRM_FILE=taco_burrito.prm gunicorn \
  --access-logfile - -b 0.0.0.0:8080 \
  -w 4 --preload wsgi:app
```

지유니콘에는 수많은 인수와 설정이 있어서 모두 설명할 수는 없다. 여기서 사용하고 있는 5개 설정을 보도록 하자.

1. --access-logfile -: 표준 출력으로 접속 로그를 작성
2. -b 0.0.0.0:8080: 서버의 접속 주소와 포트 지정
3. -w 4: 4개 작업자(프로세스) 사용
4. --preload: 작업자를 실행하기 전에 WSGI 애플리케이션 코드를 읽는다
5. wsgi:app: wsgi.py 안의 app이라는 객체가 WSGI 엔트리 포인트라고 지정

콘솔에 다음과 같이 표시되면 실행이 성공한 것이다. 종료는 (Ctrl) + (C) 키를 누르면 된다.

● 우분투 환경

```
[2017-12-28 22:55:07] [20288] [INFO] Starting gunicorn 19.7.1
[2017-12-28 22:55:07] [20288] [INFO] Listening at:
http://0.0.0.0:5000
[2017-12-28 22:55:07] [20288] [INFO] Using worker: sync
[2017-12-28 22:55:07] [20299] [INFO] Booting worker with pid: 20299
[2017-12-28 22:55:07] [20300] [INFO] Booting worker with pid: 20300
[2017-12-28 22:55:07] [20301] [INFO] Booting worker with pid: 20301
[2017-12-28 22:55:07] [20302] [INFO] Booting worker with pid: 20302
```

● 이미지 전송 확인

마지막으로, API 서버에 실제로 이미지를 전송해서 분류가 이루어지는지 확인해 보자. 여기서는 HTTP 클라이언트로 파이썬의 HTTPie(메모 참고)를 사용한다. 다음과 같이 multipart/form-data 형식으로 이미지 파일을 POST할 수 있다. 원래는 URL을 http://127.0.0.1:8080이라고 지정해야 하지만, HTTPie는 생략해서 :8080이라고만 지정해도 돼서 편리하다. <your_path>에는 4장과 마찬가지로 타코와 부리토 이미지 파일이 있는 디렉터리를 지정한다. 먼저, 타코 이미지를 API에 입력해 보자.



MEMO

HTTPie

wget이나 curl 등을 사용해도 되지만, HTTPie는 출력이 다양한 색으로 강조돼서 보기 쉽고, 인수도 비교적 간단히 설정할 수 있어서 편리하다. pip이나 apt-get 등을 사용해 쉽게 설치할 수 있다.

우분투 환경

```
$ sudo apt install httpie  
$ http --form post :8080 img@<your_path>/test/taco/360.jpg
```

출력의 디렉터리 지정

```
HTTP/1.1 200 OK  
Connection: close  
Content-Length: 23  
Content-Type: application/json  
Date: Thu, 28 Dec 2017 14:37:56 GMT
```

Server: gunicorn/19.7.1

```
{  
  "result": "taco"  
}
```

제대로 식별되는 것을 확인할 수 있다. 부리토도 테스트해 보자.

우분투 환경

```
$ sudo apt install httpie  
$ http --form post :8080 img@<your_path>/test/burrito/360.jpg
```

출력의 디렉터리 지정

```
HTTP/1.1 200 OK  
Connection: close  
Content-Length: 23  
Content-Type: application/json  
Date: Thu, 28 Dec 2017 14:40:47 GMT  
Server: gunicorn/19.7.1
```

```
{  
  "result": "burrito"  
}
```

역시 식별이 제대로 이루어진다.

7.3

도커를 이용한 배포

이 절은 도커에 대한 기본 지식이 있고 도커를 사용해 본 경험이 있는 독자를 대상으로 하고 있다. 아직 도커를 다뤄 본 적이 없는 독자라면 공식 튜토리얼(Docker Documentation, **메모** 참고) 등을 학습한 후에 보면 도움이 될 것이다.



MEMO

Docker Documentation

도커의 공식 튜토리얼(**그림 7.4**)



그림 7.4 Docker Documentation

- Get Started, Part 1: Orientation and setup

URL <https://docs.docker.com/get-started/>

7.3.1 nvidia-docker 설치

파이토치를 도커에서 사용할 때 최대 걸림돌은 GPU 지원이다. 훈련이 끝난 모델을 돌리려면 CPU만으로도 충분할 수 있지만, 모델을 훈련해야 하는 경우에는 GPU가 필수다. 도커는 가상화 기술로 GPU를 사용하려면 특수한 과정을 거쳐야 하지만, 다행히 엔비디아(NVIDIA)사가 nvidia-docker라는 패키지를 제공하고 있어서 이것을 사용하면 아무런 설정 없이 바로 GPU를 사용할 수 있다.

우분투 16.04에서는 nvidia-docker는 다음과 같이 설치할 수 있다. 먼저, apt의 리포지토리와 키를 추가한다.

우분투 환경

```
$ curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | \
sudo apt-key add -
$ curl -s -L https://nvidia.github.io/nvidia-docker/ubuntu16.04/amd64/ \
nvidia-docker.list | \
sudo tee /etc/apt/sources.list.d/nvidia-docker.list
```

apt 패키지를 최신 버전으로 업데이트한다.

우분투 환경

```
$ sudo apt-get update
```

nvidia-docker를 설치한다.

우분투 환경

```
$ sudo apt-get install -y nvidia-docker2
```

docker daemon을 재시작한다.

우분투 환경

```
$ sudo kill -SIGHUP dockerd
```


7.3.2 파이토치의 도커 이미지 작성

다음은 파이토치를 내장하고 있는 기본 이미지를 만든다. **리스트 7.9**와 같은 Dockerfile을 작성하고 빌드한다. 이 Dockerfile은 nvidia/cuda:9.0-base를 기반으로 하고 있으며, CUDA 프로그램을 실행하기 위한 최소한의 패키지로 구성돼 있다. 그리고 미니콘다(Miniconda)를 설치해서 conda 명령을 실행해 파이토치를 설치하고 있다.

리스트 7.9 Dockerfile

```
FROM nvidia/cuda:9.0-base

# 미니콘다를 설치하기 위한 최소 패키지 설치
RUN set -ex \
    && deps=' \
        bzip2 \
        ca-certificates \
        curl \
        libgomp1 \
        libgfortran3 \
    ' \
    && apt-get update \
    && apt-get install -y --no-install-recommends $deps \
    && rm -rf /var/lib/apt/lists/*

ENV PKG_URL https://repo.continuum.io/miniconda/Miniconda3-latest-Linux-x86_64.sh
ENV INSTALLER miniconda.sh

# 미니콘다 설치
RUN set -ex \
    && curl -kfsL $PKG_URL -o $INSTALLER \
    && chmod 755 $INSTALLER \
    && ./$INSTALLER -b -p /opt/conda3 \
    && rm $INSTALLER

# 미니콘다를 PATH에 추가
ENV PATH /opt/conda3/bin:$PATH

# PyTorch v1.0 설치
ENV PYTORCH_VERSION 1.0

RUN set -ex \
    && pkgs=" \
        pytorch=${PYTORCH_VERSION} \
```



```
torchvision \
" \
&& conda install -y ${pkgs} -c pytorch \
&& conda clean -i -l -t -y
```

마지막으로, conda clean을 실행해서 위 설치 과정에서 다운로드한 패키지 파일을 모두 삭제한다. 특히 파이토치의 패키지 파일은 500MB 정도로, 이 명령으로 도커의 이미지 파일 크기를 크게 줄일 수 있다. 참고로, 도커 이미지는 도커 허브(Docker Hub)에 lucidfrontier45/pytorch라는 이름으로 등록돼 있으니 풀(pull)해서 사용하거나 기반 이미지로 사용해도 좋다.

- 도커 허브

 <https://hub.docker.com/>

nvidia-docker를 사용해서 실제로 파이토치와 CUDA를 사용할 수 있는지 확인하려면 다음 명령을 실행하면 된다.

우분투 환경

```
$ sudo nvidia-docker run -it --rm lucidfrontier45/pytorch \
python -c "import torch; print(torch.randn(3).to('cuda:0'))"
(_종락...)
```

tensor([0.2023, -1.0424, -1.2000], device='cuda:0')

docker 명령 대신에 nvidia-docker를 사용하고 있다. 그리고 파이토치를 임포트(import)해서 텐서를 GPU로 전송할 수 있는지 확인하고 있다. 위 예에서처럼 tensor([0.2023, -1.0424, -1.2000], device='cuda:0')라고 표시되면 성공한 것이다.

7.3.3 웹 API 배포

7.2절에서 작성한 웹 API를 실제로 도커상에 배포해 보자. **그림 7.5**와 같이 requirements.txt라는 Dockerfile을 작성한다.

```

root/
├── app/
│   ├── __init__.py
│   └── classifier.py
├── wsgi.py
├── requirements.txt
└── Dockerfile

```

그림 7.5 디렉터리 구성

requirements.txt에는 **리스트 7.10**에 있는 것처럼 파이토치 외에 필요한 패키지를 기술한다.

리스트 7.10 requirements.txt

```

smart-getenv
flask
gunicorn

```

Dockerfile에서는 **리스트 7.11**처럼 먼저 필요한 파일을 모두 복사하고, pip 명령으로 라이브러리들을 설치한다. 마지막 부분에서는 지유니콘을 실행한다.

리스트 7.11 Dockerfile

```

FROM lucidfrontier45/pytorch

RUN mkdir /webapp
WORKDIR /webapp

COPY requirements.txt /webapp
RUN pip install --no-cache-dir -r requirements.txt

COPY app /webapp/app

COPY taco_burrito.prm /webapp/
COPY wsgi.py /webapp/

ENV PRM_FILE /webapp/taco_burrito.prm

CMD gunicorn --access-logfile - \
    -b 0.0.0.0:8080 -w 4 \
    --preload wsgi:app

```

이것을 빌드해서 실행해 보자. 참고로, 여기서는 학습이 끝난 모델만 배포하고 있으므로 `nvidia-docker`가 아닌 일반 `docker` 명령을 사용해도 된다.

🍌 우분투 환경

```
$ sudo docker build -t taco-burrito-api .
Successfully built f8e37b726772
Successfully tagged taco-burrito-api:latest
$ sudo docker run -it --rm -p 8080:8080 taco-burrito-api

[2017-12-29 15:49:15] [7] [INFO] Starting gunicorn 19.7.1
[2017-12-29 15:49:15] [7] [INFO] Listening at: http://0.0.0.0:8080
[2017-12-29 15:49:15] [7] [INFO] Using worker: sync
[2017-12-29 15:49:15] [12] [INFO] Booting worker with pid: 12
[2017-12-29 15:49:15] [13] [INFO] Booting worker with pid: 13
[2017-12-29 15:49:15] [14] [INFO] Booting worker with pid: 14
[2017-12-29 15:49:15] [15] [INFO] Booting worker with pid: 15
```

API 서버가 실행된다. 이전과 마찬가지로 HTTPie 등으로 사용해서 동작을 확인해 보자. 별도 터미널을 열어서 다음 명령을 실행하면 된다.

🍌 별도 우분투 터미널

```
$ http --form post :8080 img@<your_path>/test/taco/360.jpg

HTTP/1.1 200 OK
Connection: close
Content-Length: 23
Content-Type: application/json
Date: Thu, 29 Dec 2017 20:03:13 GMT
Server: gunicorn/19.7.1

{
  "result": "taco"
}
```

식별 API가 제대로 동작하고 있다. 종료는 `(Ctrl) + (C)` 키를 누르면 된다.

7.4

ONNX를 사용한 다른 프레임워크와의 연계

ONNX라는 신경망 모델의 표준 포맷에 대해 설명하고, 파이토치로 학습한 모델을 ONNX를 경유해서 Caffe2라는 프레임워크에서 실행해 본다.

7.4.1 ONNX란



Facebook
Open Source



Microsoft

그림 7.6 ONNX

ONNX(Open Neural Network Exchange)는 페이스북과 마이크로소프트가 추측으로 개발한 신경망 모델의 최신 호환 포맷이다. 즉, 특정 프레임워크에서 작성한 모델을 다른 프레임워크에서도 사용할 수 있게 하는 구조다(그림 7.6). 파이토치는 물론 양사가 추측이 되어 개발한 Caffe2나 CNTK, 그리고 아마존이 후원하고 있는 MXNet 등을 지원한다.¹⁴

- ONNX

URL <https://onnx.ai/>

특히 파이토치는 파이썬의 라이브러이므로 모바일 기기 등에서는 사용이 어렵다. 하지만 ONNX를 경유해서 Caffe2나 MXNet 등의 C++ 기반의 프레임워크를 사용해 모바일 기기에 배포할 수 있어서 응용 범위가 넓어진다(그림 7.7).

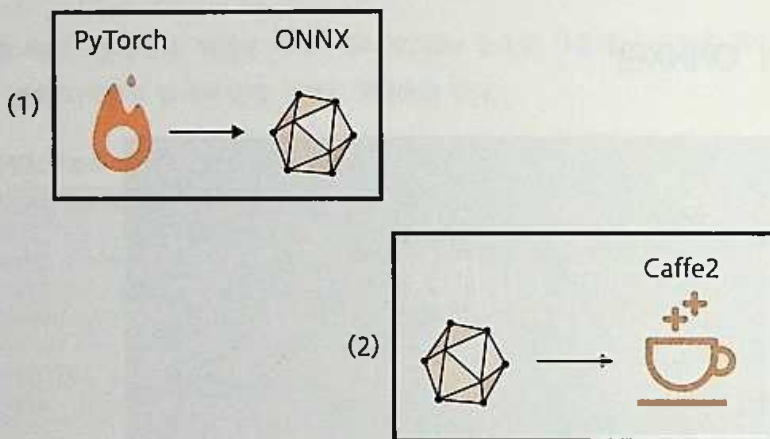


그림 7.7 ONNX의 용도. (1) GPU를 탑재한 리눅스 서버나 PC에서 파이토치 등의 다루기 쉬운 프레임워크를 사용해 모델을 만들고 학습한 후 그 결과를 ONNX 형식으로 익스포트한다. (2) Caffe2 처럼 C/C++로 작성되어 다양한 환경에서 실행되는 프레임워크를 이용해서 ONNX 포맷 모델을 임포트하고 모바일 애플리케이션으로 실행한다.

14 **참고** 구글에서 만든 플랫폼인 TensorFlow는 기본적으로 protobuf라는 구글의 serialize 방법으로 데이터를 저장하므로, ONNX 포맷과는 호환되지 않지만, 변환기를 통해서 상호 변환하는 방법을 사용할 수 있다.(베타리더의 보충 설명)

여기서는 파이토치에서 생성한 모델을 ONNX를 경유해서 Caffe2에서 실행해 보도록 한다. ONNX는 매우 새로운 포맷으로 각 프레임워크 간 대응이 아직 완벽한 것은 아니다. 하지만 파이토치와 Caffe2는 모두 페이스북이 개발한 것으로 비교적 안정적으로 동작한다.

7.4.2 파이토치 모델 익스포트

파이토치 모델을 ONNX를 변환해 보자. 여기서 다시 타코와 부리토 식별 모델을 사용한다.

● 학습 완료된 모델 불러오기

여기서부터는 주피터 노트북상에서 실행하고 있다. 먼저, 학습이 완료된 모델을 불러온다. **리스트 7.12**. 신경망 모델은 잊지 말고 평가 모드로 설정하자.

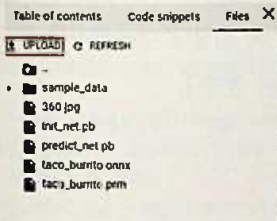
리스트 7.12 학습이 완료된 모델 불러오기¹⁵



```
from torchvision import models

def create_network():
    # resnet18 기반의 이중 분류 식별 모델
    net = models.resnet18()
    fc_input_dim = net.fc.in_features
```

15 **참고** 이후 예제를 콜라보레이터에서 실행하려면 taco_burrito.prm과 360.jpg 파일을 콜라보레이터상에 업로드해야 한다. 이 파일들은 함께 제공되는 예제 파일의 pytorch_chapter7/webapi/ 폴더 안에 있다. 그리고 실행이 안 되는 경우에는 onnx 패키지를 별도로 설치해야 할 수도 있다.



```
!pip install onnx
```

```

net.fc = nn.Linear(fc_input_dim, 2)
return net

# 모델 생성
net = create_network()

# 파라미터 읽기 및 모델에 설정
prm = torch.load("/content/taco_burrito.prm", map_location="cpu")
net.load_state_dict(prm)

# 평가 모드로 설정
net.eval()

```

● ONNX로 익스포트

다음은 ONNX로 익스포트해야 한다. 파이토치는 동적인 계산 그래프를 사용하므로 익스포트할 때에 신경망 계산을 실제로 한 번 실행해야 한다. 이를 위해 실제 이미지 데이터 대신에 차원이 동일한 더미 데이터를 사용해도 된다.

여기서는 224×224 픽셀의 3색 이미지이므로 배치 차원도 포함해서 (1, 3, 224, 224)의 더미 데이터를 사용한다. 신경망 모델, 더미 데이터, 그리고 저장할 파일명을 torch.onnx.export에 지정한다(**리스트 7.13**).

리스트 7.13 taco_burrito.onnx 출력

```

import torch.onnx

dummy_data = torch.empty(1, 3, 224, 224, dtype=torch.float32)
torch.onnx.export(net, dummy_data, "taco_burrito.onnx")

```



이상으로 익스포트가 완료된다. ONNX의 제약 중 하나는 파이토치 등의 동적 계산 프레임워크로부터 익스포트할 경우 1회 계산을 해야 하므로 신경망 내에서 if문 등으로 분기가 이루어지는 경우에 제대로 익스포트되지 않는다는 점이다.

7.4.3 Caffe2에서 ONNX 모델 사용하기

ONNX 모델을 불러오려면 onnx 패키지가 필요하며, Caffe2에서 ONNX 모델을 사용하려면 Caffe2에 포함돼 있는 caffe2.python.onnx라는 패키지를 사용해 변환해야 한다 (리스트 7.14).

리스트 7.14 ONNX 임포트



```
import onnx
from caffe2.python.onnx import backend as caffe2_backend

# ONNX 모델 불러오기
onnx_model = onnx.load("taco_burrito.onnx")

# ONNX 모델을 Caffe2 모델로 변환
backend = caffe2_backend.prepare(onnx_model)
```

다음은 backend.run에 이미지 데이터를 NumPy의 ndarray 형식으로 넣으면 계산할 수 있다. 리스트 7.15의 예에서는 원래 파이토치 모델과 ONNX 경유로 Caffe2에서 사용한 모델이 동일한 결과를 보여 주는 것을 확인할 수 있다.

리스트 7.15 파이토치 모델과 ONNX 경유 Caffe2 모델 비교



```
from PIL import Image
from torchvision import transforms

# 이미지를 잘라서 텐서로 변환하는 함수
transform = transforms.Compose([
    transforms.CenterCrop(224),
    transforms.ToTensor()
])

# 이미지 불러오기
img = Image.open("/content/360.jpg")

# 텐서로 변환해서 배치 차원을 더함
img_tensor = transform(img).unsqueeze(0)
# ndarray로 변환
img_ndarray = img_tensor.numpy()

# 파이토치로 실행
net(img_tensor)
```

 Out

```
tensor([[ 1.1262, -1.8448]])
```

 In

```
# ONNX/Caffe2로 실행
output = backend.run(img_ndarray)
output[0]
```

 Out

```
array([[ 1.126245 , -1.8447802]], dtype=float32)
```

내부 처리가 다르므로 세세한 오차가 있지만, 동일 계산 결과가 반환된다. 파이토치로 작성한 신경망이 ONNX 경유로 Caffe2에서 제대로 실행되는 것을 확인할 수 있다.

7.4.4 ONNX 모델을 Caffe2 모델로 저장

다음 예에선 ONNX 모델을 실행하기 위해 Caffe2를 백엔드로 사용하고, 신경망 계산 자체는 Caffe2의 API를 사용하고 있다. ONNX에 의존하지 않고 단순한 Caffe2 모델로 변환하려면 [리스트 7.16](#)과 같이 작성하면 된다.

리스트 7.16 ONNX에 의존하지 않고 Caffe2 모델로 변환

 In

```
from caffe2.python.onnx.backend import Caffe2Backend
init_net, predict_net = \
    Caffe2Backend.onnx_graph_to_caffe2_net(onnx_model)
```

`onnx_graph_to_caffe2_net`은 Caffe2의 신경망 정의 (`predict_net`)와 파라미터 (`init_net`)를 생성한다. 이 두 가지는 [리스트 7.17](#)처럼 파일로 저장할 수 있다.

리스트 7.17 생성한 Caffe2 신경망 정의와 파라미터 저장

 In

```
with open('init_net.pb', "wb") as fopen:
    fopen.write(init_net.SerializeToString())
with open('predict_net.pb', "wb") as fopen:
    fopen.write(predict_net.SerializeToString())
```


이처럼 ONNX에 의존하지 않고, Caffe2의 API만 사용해서 모델을 추론할 수 있다. 이렇게 하면 파이썬뿐만 아니라 C++인 Caffe2의 API를 사용할 수 있어서 파이썬을 사용할 수 없는 모바일 환경에도 배포할 수 있다.

이 책의 범위를 넘어서기 때문에 다루진 않지만, Caffe2를 사용한 안드로이드(Android) 데모 애플리케이션(cafe2-android)([메모](#) 참고)도 공개돼 있으니 관심 있는 독자라면 시도해 보도록 하자. 또한, Caffe2 외에 MXNet도 ONNX 및 모바일을 지원하고 있으므로 동일한 방식으로 사용할 수 있다.

최근에는 퀄컴(Qualcomm)을 필두로 모바일 SoC([메모](#) 참고) 수준에서 ONNX를 지원하고 있으므로 장래에는 이렇게 프레임워크에 의존하지 않고 OS 수준에서 ONNX를 이용한 추론 API가 제공될 가능성도 있다. 이후 ONNX 사용이 점점 늘어날 것으로 예상된다.



MEMO

cafe2-android

- Integrating Caffe2 on iOS/Android

[URL https://caffe2.ai/docs/mobile-integration.html](https://caffe2.ai/docs/mobile-integration.html)

- AI Camera Demo and Tutorial

[URL https://caffe2.ai/docs/AI-Camera-demo-android.html](https://caffe2.ai/docs/AI-Camera-demo-android.html)



MEMO

SoC(System on Chip)

CPU를 포함해서 다양한 칩을 하나로 모아서 만든 비메모리 반도체다.

정리

이 장에서 배운 내용을 정리했다.

파이토치의 학습이 완료된 모델을 저장하고 불러오는 방법을 설명하고, 플라스크를 이용해서 웹 API를 작성했다. 웹 API는 도커를 이용하면 배포 환경을 구축하지 않고 손쉽게 배포할 수 있다. 도커를 GPU와 함께 이용하고 싶은 경우에는 `nvidia-docker`를 사용하면 된다.

ONNX는 다양한 프레임워크 간에 모델을 표준화하는 구조다. 이것을 사용하면 파이토치 등 시행착오에 기반한 프레임워크에서 모델을 구축, 훈련하고 ONNX를 경유해서 Caffe2나 MNet 등의 프레임워크에서 사용할 수 있다. 이를 통해 모바일 기기 등 파이썬을 사용할 수 없는 환경에서도 학습이 완료된 모델을 배포할 수 있다.



APPENDIX

A

훈련 상태 가시화

각 장에서 실시한 훈련을 가시화하는 방법에 대해 설명한다.

A1.1

텐서보드를 사용한 가시화

이 책에서는 훈련 상태를 print를 사용해서 표시하긴 했지만, 실시간으로 가시화하지는 못했다. 여기서 소개하는 **텐서보드(TensorBoard)**는 텐서플로(TensorFlow)에서 제공하는 훌륭한 가시화 툴이다. 파이토치도 텐서플로와 동일한 형식의 로그를 출력할 수 있으므로 텐서보드를 통해 가시화할 수 있다.

텐서보드X(tensorboardX)라는 외부 라이브러리를 이용해서 훈련 로그를 출력하고, 이 로그를 텐서보드를 통해 가시화한다.

● 텐서보드와 텐서보드X 설치

다음과 같이 텐서보드와 텐서보드X를 설치한다.

● 우분투 환경

```
$ pip install tensorflow tensorboard tensorboardX
```

● 텐서보드X 사용

사용법은 간단하다. SummaryWriter라는 클래스의 인스턴스를 만들고, add_scalar 등의 메서드로 기록하고 싶은 값을 작성하면 된다. SummaryWriter를 만들 때에 지정한 출력 디렉터리에 로그가 저장되므로 이 디렉터리를 텐서보드에서 지정하면 로그를 가시화할 수 있다.

● 우분투

```
$ tensorboard --logdir <log_dir>
```

디렉터리 지정

● 웹 브라우저를 통해 확인

초기 설정에서는 포트 번호 6006을 사용해 웹서버를 실행하므로 브라우저에서 `http://localhost:6006/`을 입력해서 확인할 수 있다. 4장의 Fashion-MNIST(4.2절) 예를 사용해 테스트해 보자. 먼저, **리스트 4.3**의 `train_net`을 **리스트 A1.1**과 같이 변경한다.

리스트 A1.1 `train_net` 함수 작성



```
# 평가 헬퍼 함수
(중략)
# 훈련 헬퍼 함수
def train_net(net, train_loader, test_loader,
              optimizer_cls=optim.Adam,
              loss_fn=nn.CrossEntropyLoss(),
              n_iter=10, device="cpu", writer=None):
    train_losses = []
    train_acc = []
    val_acc = []
    optimizer = optimizer_cls(net.parameters())
    for epoch in range(n_iter):
        running_loss = 0.0
        # 신경망을 훈련 모드로 설정
        net.train()
        n = 0
        n_acc = 0
        # 시간이 매우 오래걸리므로 tqdm을 사용해서 진행 바 표시
        for i, (xx, yy) in tqdm.tqdm(enumerate(train_loader),
                                     total=len(train_loader)):
            xx = xx.to(device)
            yy = yy.to(device)
            h = net(xx)
            loss = loss_fn(h, yy)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            n += len(xx)
            _, y_pred = h.max(1)
            n_acc += (yy == y_pred).float().sum().item()
        train_losses.append(running_loss / i)
        # 훈련 데이터 예측 정확도
        train_acc.append(n_acc / n)
        # 검증 데이터 예측 정확도
        val_acc.append(eval_net(net, test_loader, device))
        # 이 epoch의 결과 표시
```



```
print(epoch, train_losses[-1], train_acc[-1], val_acc[-1], flush=True)
if writer is not None:
    writer.add_scalar('train_loss', train_losses[-1], epoch)
    writer.add_scalars('accuracy', {
        "train": train_acc[-1],
        "validation": val_acc[-1]
    }, epoch)
```

함수의 마지막 인수가 SummaryWriter의 객체다. 각 epoch의 루프 마지막에서 add_scalar나 add_scalars 메서드를 사용해서 로그를 출력한다. add_scalars란, 여러 값을 동일 그래프에 출력하는 메서드다. 그리고 [리스트 A1.2](#)에서 실제로 훈련할 때는 먼저 SummaryWriter의 인스턴스를 만들어 train_net에 전달한다.

[리스트 A1.2](#)의 예에서는 /tmp/cnn이라는 디렉터리에 로그를 출력하고 있다.

리스트 A1.2 로그 출력



```
from tensorboardX import SummaryWriter

# SummaryWriter 작성
writer = SummaryWriter("/tmp/cnn")

# 훈련 실행
net.to("cuda:0")
train_net(net, train_loader, test_loader, n_iter=20, device="cuda:0",
writer=writer)
```

[리스트 4.1](#) → [리스트 4.2](#) → [리스트 A1.1](#) 순으로 실행한 후 [리스트 A1.2](#)를 실행하면 학습이 시작된다. /tmp/cnn에 로그 파일이 생성되는 것을 확인했으면, 텐서보드를 실행해서 브라우저로 <http://127.0.0.1:6006/>에 접속해 보자([그림 A1.1](#)). 참고로, 실행할 때는 바로 전에 실행한 텐서보드를 **(Ctrl) + C** 키를 눌러 종료해 주어야 한다.¹⁶

우분투 환경

```
$ tensorboard --logdir /tmp/cnn
```

¹⁶ [그림 4.9](#) 안타깝게도 클래버레터에서는 텐서보드 화면을 확인할 수 없다. [리스트 A1.1](#)과 [A1.2](#)가 실행은 되니 로그 파일은 확인할 수 있다. 단, /tmp/cnn을 /content/cnn으로 변경해야 한다.

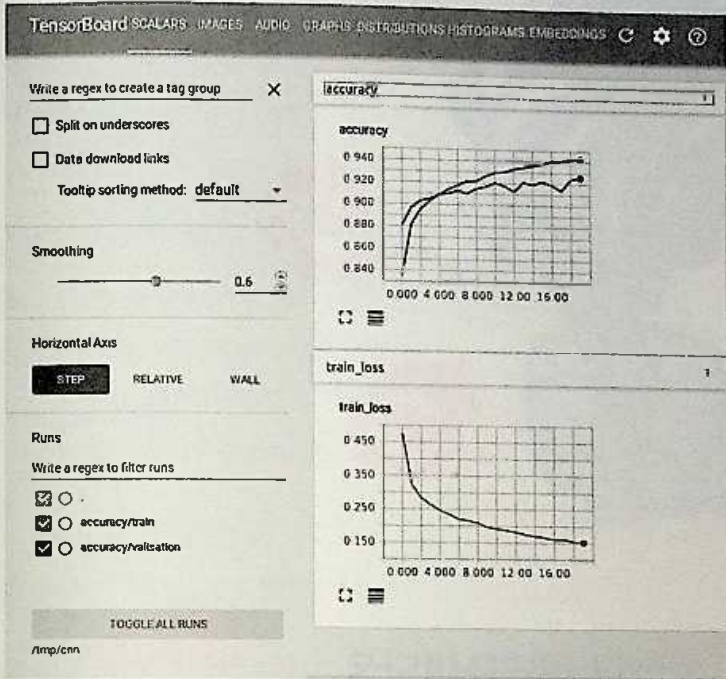


그림 A1.1 훈련 가시화

텐서보드X의 API에 대한 상세한 내용은 다음 공식 Doc을 참고하자.

- 텐서보드X

 <http://tensorboard-pytorch.readthedocs.io/en/latest/tensorboard.html>



APPENDIX

B

컬래버러터리로 파이토치 개발 환경 구축

구글이 제공하는 무료 개발 환경인 컬래버러터리(Colaboratory)를 소개한다.

B1.1

컬래버레이터를 사용한 파이토치 개발 환경 구축 방법

컬래버레이터로 파이토치 개발 환경을 구축하는 방법을 소개한다.

B1.1.1 컬래버레이터란

컬래버레이터는 구글이 공개한 웹 서비스로 주피터 노트북과 비슷한 환경을 제공한다. ipynb 파일은 구글 드라이브(Google Drive)상에 저장되며, 구글 독스(Google Docs)처럼 공유할 수도 있다. 컬래버레이터는 엔비디아의 Tesla K80이라는 GPU도 무료로 제공하므로 GPU 장비 없이도 손쉽게 딥러닝을 검증할 수 있다.

B1.1.2 장비 사양

이 책 집필 시점(2018년 8월)의 사양은 다음과 같다.

- CPU: Intel Xeon 2.5GHz 2코어
- 메모리: 13GB
- 디스크: 40GB
- GPU: NVIDIA Tesla K80
- OS: Ubuntu 17.10

이 사양의 가상 머신을 최대 12시간 연속으로 사용할 수 있다. 12시간을 초과하거나 80분 이상 대기 상태가 지속되면 가상 머신이 삭제되며, 데이터도 전부 파기되므로 주의하도록 하자.

B1.1.3 파이토치 환경 구축

다음 사이트에 접속하면 콜라버레이터를 열 수 있다. 구글 계정에 로그인해야 하므로 계정이 없으면 미리 만들어 두도록 하자.

- 콜라버레이터

<https://colab.research.google.com/>

● GPU 환경 설정하기

콜라버레이터에 접속하면 **그림 B1.1** 화면이 나온다. 'CANCEL'을 클릭하자.

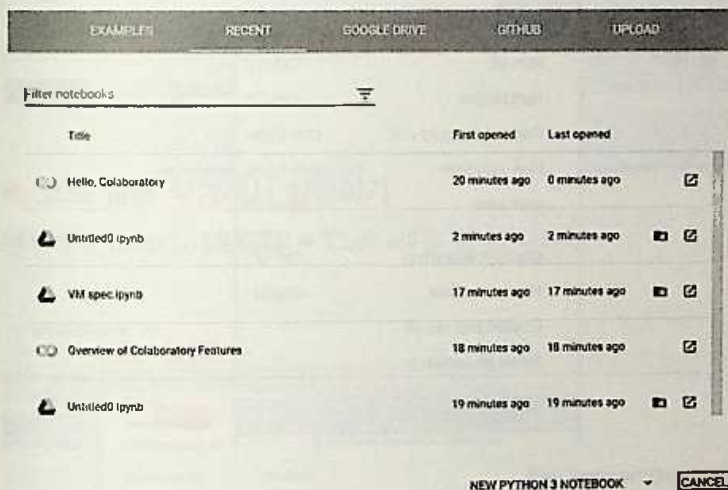


그림 B1.1 'CANCEL' 클릭

메뉴에서 'File'① **그림 B1.2** → 'New Python 3 notebook'을 선택한다②.



그림 B1.2 'New Python 3 notebook' 선택

메뉴에서 'Runtime'① **그림 B1.3** → 'Change runtime type'을 선택한다②.

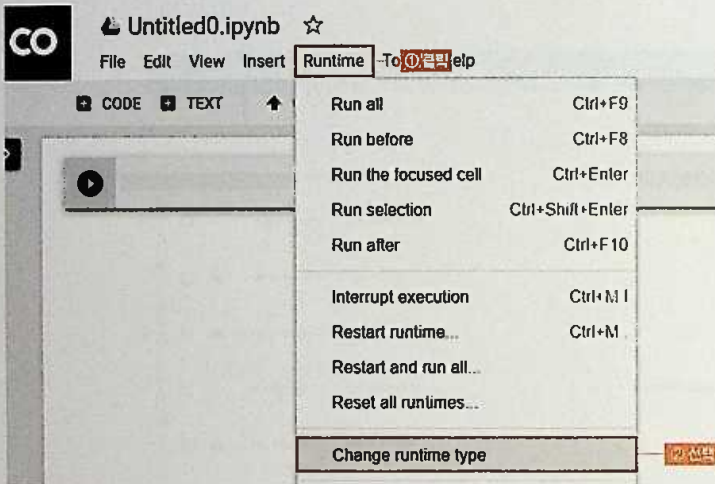


그림 B1.3 'Change runtime type' 선택

'Notebook settings' 화면이 열린다. 'Hardware accelerator'의 ▼를 클릭해서① **그림 B1.4**, 'GPU'를 선택한다②.

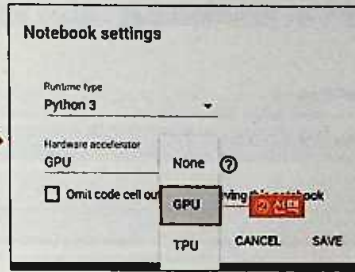
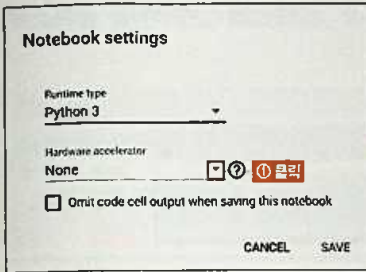


그림 B1.4 'GPU' 선택

선택한 후에 'SAVE' 버튼을 눌러 설정을 저장한다(그림 B1.5).

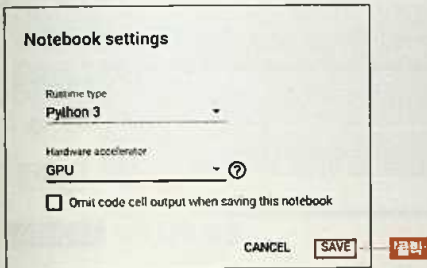


그림 B1.5 'SAVE' 클릭

● 코드 셀을 작성해서 실행하기

메뉴에서 'Insert'(① 그림 B1.6) → 'Code cell'을 선택한다(②).

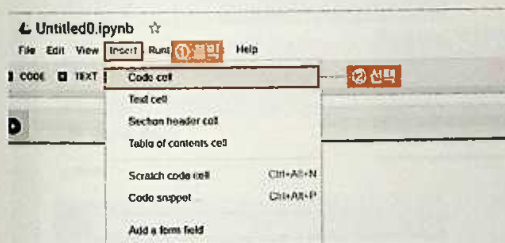


그림 B1.6 'Code cell' 선택

코드 입력 셀이 추가된다(그림 81.7).

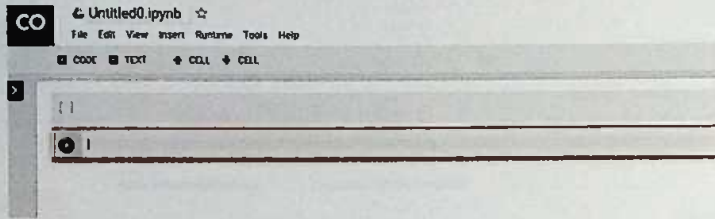


그림 81.7 코드 입력 셀이 추가된다.

코드를 입력하고(① 그림 81.8) 실행 버튼을 클릭하면(②), 그 결과가 아래에 표시된다(③).

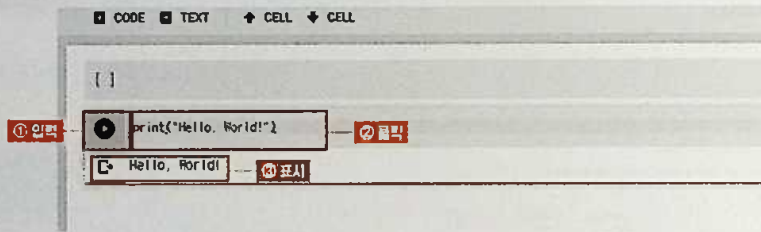


그림 81.8 코드 실행

● 텍스트 입력하기

메뉴에서 'Insert'(① 그림 81.9) → 'Text cell'을 선택한다(②).

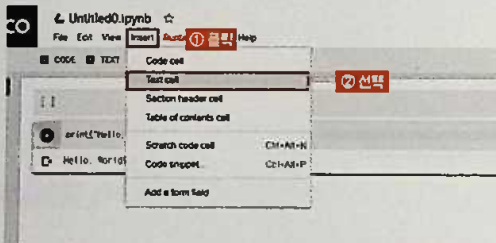


그림 81.9 'Text cell' 선택

텍스트를 입력하면(① 그림 B1.10), 오른쪽에 미리보기가 표시된다(②).

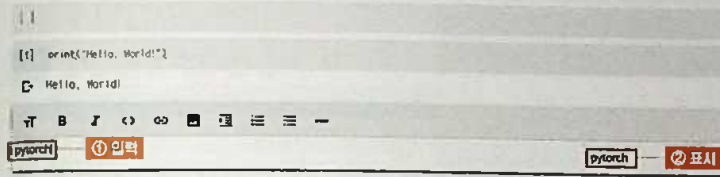


그림 B1.10 텍스트 입력

(Shift) + (Enter) 키를 누르면 입력한 내용이 확정된다(그림 B1.11).



그림 B1.11 입력 내용 확정

● 파일명 변경

왼쪽 위의 'Untitled0.ipynb'를 클릭하면 텍스트 내에 커서가 표시된다(그림 B1.12).

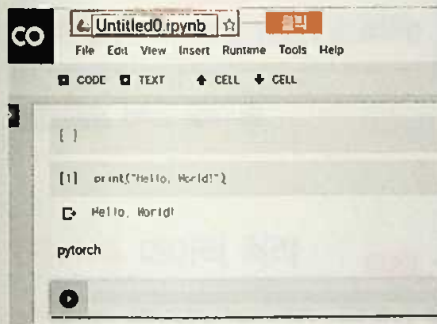


그림 B1.12 왼쪽 위의 파일명 클릭

파일명을 입력한다(여기서는 'Untitled0'를 'PyTorch'로 변경. 그림 B1.13).

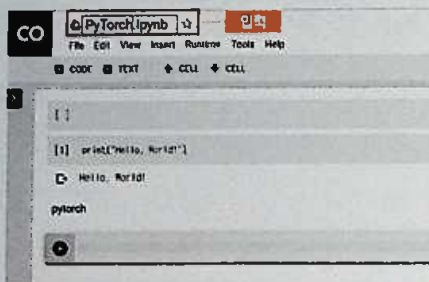


그림 B1.13 파일명 입력

(Shift) + (Enter) 키를 누르면 변경한 내용이 확정된다(그림 B1.14).

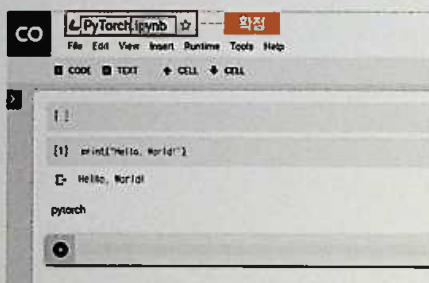


그림 B1.14 파일명 변경 확정

● 편리한 기능

메뉴의 'File' 아래에 있는 'CODE' 아이콘을 클릭하면(① 그림 B1.15), 코드 셀을 추가할 수 있다(②).

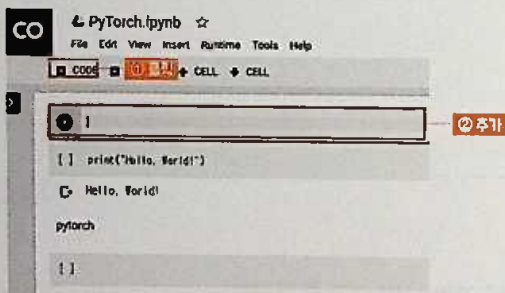


그림 B1.15 코드 셀 추가

메뉴의 'Edit' 아래에 있는 'TEXT' 아이콘을 클릭하면(① **그림 B1.16**), 텍스트 셀을 추가할 수 있다(②).

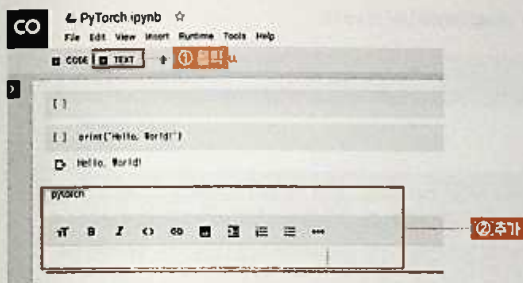


그림 B1.16 텍스트 셀 추가

메뉴의 'Insert' 아래에 있는 '↑CELL' 아이콘을 클릭하면(① **그림 B1.17**), 선택한 셀을 위로 이동할 수 있다(②). 메뉴의 'Runtime' 아래에 있는 '↓CELL' 아이콘을 클릭하면(③) 선택한 셀을 아래로 이동할 수 있다(④).

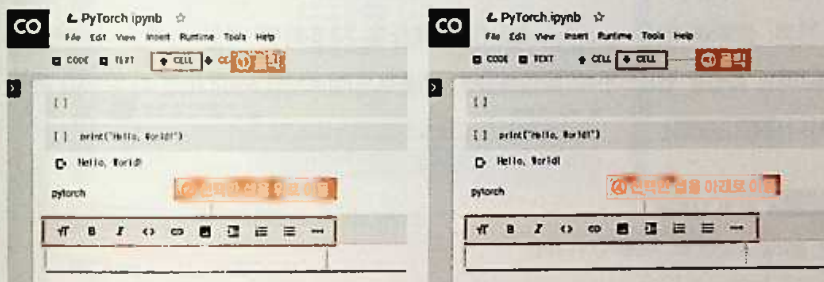


그림 B1.17 선택한 셀의 이동

B1.1.4 데이터 처리

4장에서는 웹상에서 데이터를 가져와야 할 때 `wget` 등의 명령을 사용한다. 예를 들어, 4.4절에서 사용하는 얼굴 데이터를 다운로드해서 압축 해제하는 과정은 **리스트 B1.1**과 같다.

리스트 B1.1 얼굴 데이터 다운로드(wget) 및 압축 해제(mv)



```
!wget http://vis-www.cs.umass.edu/lfw/lfw-deepfunneled.tgz
!tar xf lfw-deepfunneled.tgz
!mkdir lfw-deepfunneled/train
!mv lfw-deepfunneled/[A-W]* lfw-deepfunneled/train
!mkdir lfw-deepfunneled/test
!mv lfw-deepfunneled/[X-Z]* lfw-deepfunneled/test
```



```
--2018-07-03 07:58:07-- http://vis-www.cs.umass.edu/lfw/lfw-deepfunneled.tgz
Resolving vis-www.cs.umass.edu (vis-www.cs.umass.edu)... 128.119.244.95
Connecting to vis-www.cs.umass.edu (vis-www.cs.umass.edu)|128.119.244.95| →
:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 108761145 (104M) [application/x-gzip]
Saving to: 'lfw-deepfunneled.tgz'

lfw-deepfunneled.tg 100%[=====>] 103.72M 26.9MB/s in 5.1s

2018-07-03 07:58:12 (20.5 MB/s) - 'lfw-deepfunneled.tgz' saved `→
[108761145/108761145]
```

처음 기본 설치돼 있는 google.colab.files 모듈을 사용하면 파일 업로드나(여기서는 result.txt라는 파일을 업로드한다)([리스트 B1.2](#)) 다운로드(업로드한 result.txt를 다운로드) 등도 가능하다([리스트 B1.3](#)).

리스트 B1.2 파일 업로드



```
from google.colab import files

# 창이 표시되면 로컬 파일을 선택해서 업로드한다
uploaded = files.upload()
```



-- 파일 선택 선택한 파일 없음

Cancel upload

리스트 B1.3 파일 다운로드

```
# result.txt 다운로드
files.download("result.txt")
```

COLUMN

FUSE 사용

약간의 수고를 해야 하지만, 구글 드라이브(Google Drive)를 FUSE라는 리눅스 가상 파일 시스템 드라이버를 경유해서 사용하면 데이터를 구글 드라이브에서 직접 읽고 쓸 수 있다. 자세한 내용은 **그림 B1.18**의 주피터 노트북을 참고하자.



그림 B1.18 Drive FUSE example.ipynb

URL: <http://bit.ly/2XTxJhF>

진솔한 서평을 올려주세요!

이 책이나 이미 읽은 제이펍의 다른 책이 있다면, 책의 장단점을 잘 보여주는 솔직한 서평을 올려주세요. 매월 다섯 분을 선별하여 원하시는 제이펍 도서 1부씩을 선물해드리겠습니다.

■ 서평 이벤트 참여 방법

- 제이펍의 책을 읽고 자신의 블로그나 인터넷 서점에 서평을 올린다.
- 서평이 작성된 URL을 적어 아래의 계정으로 메일을 보낸다.

review.jpub@gmail.com

■ 서평 당선자 발표

매월 첫 주 제이펍 홈페이지(www.jpub.kr) 및 페이스북(www.facebook.com/jeipub)에 공지하고 당선된 분에게는 개별 연락을 드리겠습니다.

독자 여러분의 응원과 질타를 통해 더 나은 책을 만들 수 있도록 최선을 다하겠습니다.



숫자

102 Category Flower Dataset	90
1D(batch)	127
2D(batch dim)	127
3D(batch step dim)	127

A

abs	17
Activation Function	44
Adam(Adaptive Moment Estimation)	46
add_scalar	188, 190
add_scalars	190
Advanced-Indexing	115

B

backend.run	183
batch	127
Batch Normalization	50
BatchNorm	43
Bernoulli Distribution	24
Bilinear 보간법	87
BoW(Bag of Words)	107, 154
build_vocab	133

C

Caffe2	183
--------	-----

caffe2.python.onnx	183
Caffe2-android	185
cat	18
CHW	18
CIFAR-10	71
classifier.py	167
CNN	xvi, 59
CNN 회귀 모델	80
Colaboratory	193
Conv2d	83
Convolutional Neural Network	xvi, 59
ConvTransposed2d	83, 92
Corpus	107
cos	17
CountVectorizer	153
create_network	167
CUDA	3

D

DataLoader	48, 146
Dataset	48, 146
Dataset 클래스	111
DCGAN(Deep Convolutional Generative Adversarial Networks)	xvi, 89
dcgan-pytorch-example	91
Docker Documentation	173
Docker Hub	176
dot	19
Dropout	43, 50

E

eig	19
Elman Network	104
Embedding	108, 113
Encoder-Decoder 모델	xvi, 129

F

Factorization Machines	143
fake_data	89
fake_out	89
Fashion-MNIST	62
Feedforward형	45
FFT(Fast Fourier Transform)	3
Flask	164
forward 메서드	55
FUSE	203

G

GRU(Gated Recurrent Unit)	104
GAN(Generative Adversarial Network)	89
gesv	19
google.colab.files 모듈	202
GPU	3
Gradient Descent	26
Gunicorn	164

H

HTTPIe	171
--------	-----

I

ImageFolder	73
ImageNet	71
IMDb 리뷰 데이터	109
imdb.vocab	110
Inception	70

J

Jupyter Notebook	10, 181, 194
------------------	--------------

K

K 차원	147
------	-----

L

Labeled Faces in the Wild Home	80
Large Movie Review Dataset	109
LFW deep funneled images	80
Likelihood	24
Linear Regression	28
Linux	5
list2tensor	111
load_state_dict 메서드	163
log	17
Logistic Regression	35
Loss Function	26
LSTM(Long Short Term Memory)	104

M

MAE(Mean Absolute Error)	148
map_location	163
matmul	19
Matrix Factorization	143, 144
max	17
max_len	111
Maximum Likelihood Estimation	25
MaxPool2d	83
mean	17
Mean Error	148
Mean Squared Error	29
MF	143, 144
min	17
mini-batch	48
Miniconda	2, 5
MLP	43
nm	19
MovieLens	145

MovieLens 데이터	145
MovieLensDataset	155
Multi-Layer Perceptron	43
mv	19

N

n_tokens	111
ndarray 형식	183
NeuralMatrixFactorization	152
NeuralMatrixFactorization2	155
NeuralMF	153
NMF	145
nn.BCEWithLogitsLoss	115
nn.CrossEntropyLoss	127
nn.Embedding	108, 147, 158
nn.functional.l1_loss	148
nn.GRU	114
nn.L1Loss	148
nn.Linear	33
nn.LSTM	114
nn.Module	33, 163
nn.ReLU	45
nn.RNN	114
nn.Sequential	45
nn.utils.rnn.pack_padded_sequence 함수	118
Non-negative Matrix Factorization	145
Normalized	126
NumPy	xiv
NVIDIA	2, 174
NVIDIA Tesla K80	194
nvidia-384	3

O

ONNX(Open Neural Network Exchange)	180
onnx_graph_to_caffe2_net	184
optim 모듈	32
Oxford	90

P

PackedSequence	118
----------------	-----

padding	83
PIL 패키지	167
Practical PyTorch	121
Pre-trained	75
Pre-trained Embedding	108
Python 3.7	5

R

real_data	89
Recurrent Neural Networks	xvi
regularization	52
requires_grad	20
reshape 함수	18
ResNet	70
ResNet18	75
RNN	xvi, 109

S

scikit-learn	37, 117
SGD	27
Singular Value Decomposition	145
smart_getenv	170
SoC(System on Chip)	185
softmax 함수	126
split 메서드	123
sqrt	17
squeeze	94
stack	18
std	17
step	127
Stochastic Gradient Descent	27
sum	17
svd	19
SVD	145
SVMlite 형식	117
symlink	19

T

taco_burrito.prm	170
Tensor	13

TensorBoard	188
tensorboardX	188
text2ids	111
torch	12
torch.autograd	12
torch.multinomial	126
torch.nn	12, 32
torch.onnx	12
torch.onnx.export	183
torch.optim	12, 32
torch.save	97
torch.tensor 함수	13
torch.utils.data	12
torchvision	73
torchvision.utils.save_image	87
transpose	18
Transposed Convolution	83

U

Ubuntu 16.04	2
--------------	---

V

Variable	20
VGG	70
view	18, 127

W

words2tensor	133
--------------	-----

ㄱ

가변 길이	118
가상 환경	7
검증용	82
경사 하강법	26
제열 레이블링	109
과학습	50
기계 번역	103, 129

ㄴ

내부 상태	104
-------	-----

ㄷ

다중 분류	40
다층 퍼셉트론	xvi, 43
단계 수	114
대수 우도 함수	26
도커 허브	2, 176
동적 계획법	46

ㄹ

라이브러리	xvii
로지스틱 회귀	35
루프(반복)	33
라눅스	xiv

ㄴ

마스크 배열	15
머신러닝	50
모듈화	55
문장 분류	103
문장 생성	103
미니 배치	27
미니콘다	5
미리 학습 완료	108
미분	xiv

ㅁ

배치 수	114
배포	173
베르누이 분포	24
벡터	xiv
변환	107
보조 함수	132
부속 정보	153
브로드캐스트	17

비음수 행렬 인수분해	145
빅데이터	27

사

사전	107
사칙 연산	16
상속	56
선형 결합	35
선형 계층	104
선형 대수 계산	18
선형 모델	xvi
선형 회귀	28, 45
선형 회귀 모델	xvi, 28
세익스피어	122
소프트맥스 함수	41
손글씨 문자	45
손실 함수	26
수치	107
수치화	106
수학 함수	17
순환 구조	104
순환 신경망	xvi, 103
숨김층	44
시각화	104
시간 방향	104
시그모이드 함수	35
식별 모델	93
신경망	xvi, 50, 55
신경망 모델	155
신경망 정의	113
신경망 행렬 인수분해	151

ㅇ

애플리케이션	161
어휘집	133
엔디비아	2, 174
용어집	110
우도	24
우분투	2
웹 API	164, 176
이미지 분류	62
이미지 처리	59

인덱스	15
인스턴스	3
인코더-디코더 모델	130, 135

자

자동 미분	20, 46
자연어 처리	103
잠재 의미 분석	145
잠재적 특이 벡터 z	92
전이 학습	69
정규 분포	28
정규화	50
주피터 노트북	10
지수 함수	126
지유니콘	164
집합	107

차

체인 규칙	21
최대 우도 추정	25
최적화기	75, 138
추천 시스템	143

ㅋ

커널	60
커스텀 Dataset	154
커스텀 계층	55
컬래버레이터리	81, 195
코퍼스	107
콘솔 모드	3
쿠다	3
크로스 엔트로피 함수	40

텍

텍스트 데이터	106
텍스트 파일	122
텐서보드	188
텐서보드X	188

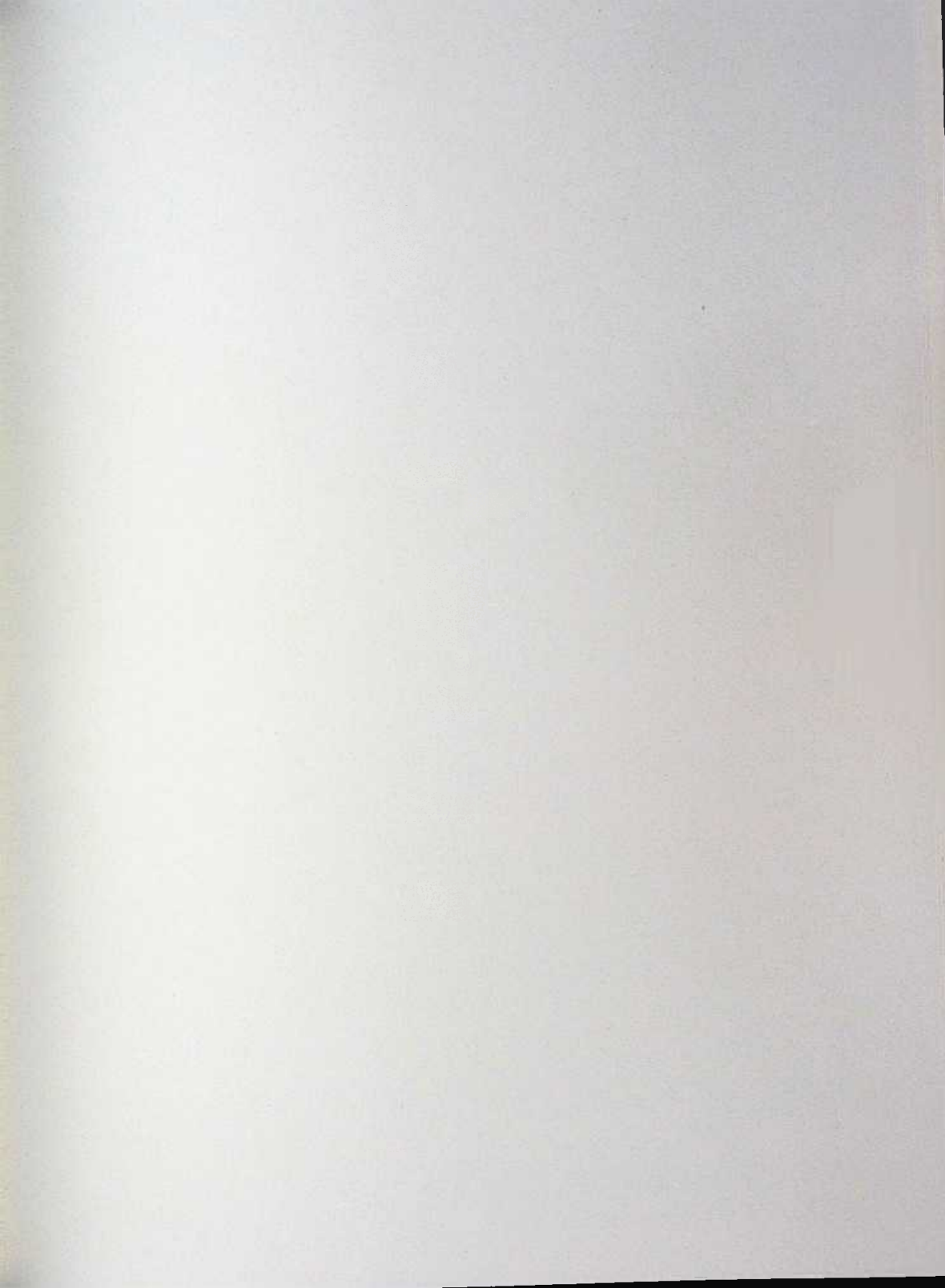
토큰화	106
특이값 분해	16, 145
특이 수	114

II

파이썬	xiv
파이토치	xiv
파이토치의 RNN 계열	114
파이토치를 사용한 DCGAN	91
파일 다운로드	202
파일 업로드	202
편미분	25
평가	115
평가 함수	148
평가 함수 eval_net	119
평균 절대 오차	148
평균 제곱 오차	29
플라스크	164
피드포워드형	45

III

학습률 파라미터	26
함수	xiv
합성곱 신경망	xvi, 59, 60
행렬	xiv
행렬 분해	143
행렬 인수분해	143, 144, 147
행렬곱	16
헬퍼 함수	156
형태소 분석	107
화물 모델	24, 29
확률적 경사 하강법	27
활성화 함수	44
회귀 계수	28
회귀 문제	28
훈련	115
훈련 함수	94
훈련용	82
최소 행렬 + 레이블	117





파이토치 첫걸음

실무에도 바로 활용할 수 있는 파이토치 입문서!

딥러닝의 파이썬 라이브러리로는 구글이 개발한 텐서플로(TensorFlow) 프레임워크가 가장 유명하지만, 심벌을 사용하는 프로그래밍 스타일 때문에 초보자가 접근하기 어렵다는 의견도 존재한다. 반면 이 책에서 다루는 파이토치는 페이스북을 중심으로 개발된 오픈 소스 프로젝트로 동적 네트워크라는 구조를 도입했으며, 일반적인 파이썬 프로그램과 같은 환경에서 간단하게 신경망을 구축할 수 있다는 점에서 많은 관심을 받고 있다. 특히, 해외 연구자들로부터 많은 지지를 받고 있어서 최신 연구들이 파이토치를 사용해 구현되는 중이다. 연구 결과들도 깃허브를 통해 빠르게 공개되는 것이 당연시되고 있다. 아직 한글 자료는 부족하지만, 사용하기 쉽고 최신 연구 결과를 바로 적용할 수 있어서 서비스에 딥러닝을 곧바로 적용하고 싶은 사람에게는 최적의 프레임워크가 될 것이다. 이 책을 통해서 독자 여러분이 신경망이나 딥러닝, 그리고 머신러닝 등에 흥미를 가지고 실제로 자신의 업무에 적용할 수 있게 되기를 바란다.

- '시작하며' 중에서

이 책의 대상 독자

- 인공지능을 배우고자 하는 프로그래머
- 머신러닝 및 딥러닝 엔지니어

난이도

분 야 인공지능 / 딥러닝

Jpub “오과 [이] 끝나는 세상”
www.jpub.kr



아름다운재단 출판 수익 및 역자 번역료의 일부는
아름다운재단의 사회영역기금에 기부됩니다.

정가 24,000 원



93000

9 791188 621590

ISBN 979-11-88621-59-0